APPENDIX A — COMPLETE SOURCE CODE

The full, readable source listing is supplied as the companion document “Maildesk Appendix A — Complete Source Code.” It contains the Java implementation and tests, MySQL schema, Windows scripts, Maven build file, Maven Wrapper, and CI workflow. The 40-page project report keeps this page as the source index; Appendix B contains the application outputs.

The companion source appendix is included with the project files in docs/report/Maildesk_Appendix_A_Source_Code.docx and is linked from the project README. Source is reproduced directly from the repository in monospace formatting. Local OAuth credentials, saved tokens, passwords, and private mailbox data are excluded.

APPENDIX A.1 — PRODUCTION JAVA

src/main/java/com/emailapp/App.java

```
package com.emailapp;

/** Member 2's launcher remains an alias for the single production entry point. */
public final class App {
    private App() { }
    public static void main(String[] args) { ems.App.main(args); }
}
```

src/main/java/com/emailapp/database/BlockedSenderRepository.java

```
package com.emailapp.database;

import ems.storage.JdbcMailStore;
import java.sql.SQLException;

/** Account-scoped sender blocks used by the local classifier. */
public final class BlockedSenderRepository {
    private final JdbcMailStore store;
    public BlockedSenderRepository() { this(new JdbcMailStore()); }
    public BlockedSenderRepository(JdbcMailStore store) { this.store = store; }
    public boolean isBlocked(long accountId, String senderEmail) throws SQLException {
        return store.isSenderBlocked(accountId, senderEmail);
    }
    public void block(long accountId, String senderEmail) throws SQLException {
        store.setSenderBlocked(accountId, senderEmail, true);
    }
    public void unblock(long accountId, String senderEmail) throws SQLException {
        store.setSenderBlocked(accountId, senderEmail, false);
    }
}
```

src/main/java/com/emailapp/database/DatabaseConnection.java

```
package com.emailapp.database;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
```

```
public final class DatabaseConnection {

    private DatabaseConnection() { }

    private static final String URL =
            System.getenv().getOrDefault(
                    "DB_URL",

"jdbc:mysql://localhost:3306/email_management?createDatabaseIfNotExist=true&connectionTimeZone=UTC&forceConnecti
onTimeZoneToSession=true&connectTimeout=5000&socketTimeout=15000"
            );

    private static final String USER =
            System.getenv().getOrDefault("DB_USER", "root");

    private static final String PASSWORD =
            System.getenv().getOrDefault("DB_PASSWORD", "");

    public static Connection getConnection()
            throws SQLException {

        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

src/main/java/com/emailapp/database/MessageRepository.java

```
package com.emailapp.database;

import ems.model.EmailMessage;
import ems.storage.JdbcMailStore;
import java.sql.SQLException;
import java.util.List;
import java.util.Set;

/** Member 3 compatibility facade; use MailStore for application operations. */
public final class MessageRepository {
    private final JdbcMailStore store;
    public MessageRepository() { this(new JdbcMailStore()); }
    public MessageRepository(JdbcMailStore store) { this.store = store; }
    public void save(EmailMessage message) throws SQLException {
        store.saveMessages(message.getAccountId(), List.of(message));
    }
    /** Saving the full snapshot keeps recipients and labels consistent atomically. */
    public void saveRecipients(EmailMessage message) throws SQLException { save(message); }
    public void saveLabels(EmailMessage message) throws SQLException { save(message); }
    public List<EmailMessage> findByAccount(long accountId) throws SQLException { return
store.findAll(accountId); }
    public List<String> findRecipients(long accountId, String messageId) throws SQLException {
        return store.findMessage(accountId, messageId).map(EmailMessage::getRecipientEmails).orElse(List.of());
    }
    public Set<String> findLabels(long accountId, String messageId) throws SQLException {
        return store.findMessage(accountId, messageId).map(EmailMessage::getGmailLabelIds).orElse(Set.of());
    }
    public void updateReadStatus(long accountId, String messageId, boolean read) throws SQLException {
        store.updateReadStatus(accountId, messageId, read);
    }
}
```

src/main/java/com/emailapp/database/SpamFeedbackRepository.java

```
package com.emailapp.database;

import ems.storage.JdbcMailStore;
```

```
import java.sql.SQLException;

/** User corrections take precedence over automatic results across refreshes. */
public final class SpamFeedbackRepository {
    private final JdbcMailStore store;
    public SpamFeedbackRepository() { this(new JdbcMailStore()); }
    public SpamFeedbackRepository(JdbcMailStore store) { this.store = store; }
    public void saveFeedback(long accountId, String messageId, boolean spam) throws SQLException {
        store.saveFeedback(accountId, messageId, spam);
    }
}
```

src/main/java/com/emailapp/database/SpamRepository.java

```
package com.emailapp.database;

import ems.model.EmailMessage;
import ems.model.SpamResult;
import ems.storage.JdbcMailStore;
import java.sql.SQLException;

/** Compatibility facade for Member 3's original repository API. */
public final class SpamRepository {
    private final JdbcMailStore store;
    public SpamRepository() { this(new JdbcMailStore()); }
    public SpamRepository(JdbcMailStore store) { this.store = store; }
    public void save(EmailMessage message, SpamResult result) throws SQLException {
        store.saveSpamResult(message.getAccountId(), message.getGmailMessageId(), result);
    }
    public SpamResult findByMessage(long accountId, String messageId) throws SQLException {
        if (store.findMessage(accountId, messageId).isEmpty()) return null;
        return store.getSpamResult(accountId, messageId);
    }
}
```

src/main/java/com/emailapp/spam/SpamClassifier.java

```
package com.emailapp.spam;

import ems.model.EmailMessage;
import ems.model.SpamResult;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.regex.Pattern;

/** Explainable local rules, independent of Gmail's own SPAM classification. */
public final class SpamClassifier {
    private static final Pattern LINK = Pattern.compile("https?://\\S+", Pattern.CASE_INSENSITIVE);
    private static final List<String> PHRASES = List.of(
            "claim your prize", "verify your account", "you have won",
            "urgent action required", "click here to claim");

    public SpamResult analyze(EmailMessage message) {
        return analyze(message, false);
    }

    /** The store supplies account-specific block information within its transaction. */
    public SpamResult analyze(EmailMessage message, boolean blockedSender) {
        int score = 0;
        List<String> reasons = new ArrayList<>();
        String sender = message.getSenderEmail().trim().toLowerCase(Locale.ROOT);
        String subject = message.getSubject().toLowerCase(Locale.ROOT);
        String body = message.getBodyText().toLowerCase(Locale.ROOT);
        if (blockedSender) {
            score += SpamRules.BLOCKED_SENDER;
            reasons.add("Sender is blocked");
```

```
        }
        String content = subject + " " + body;
        int phraseMatches = 0;
        for (String phrase : PHRASES) {
            if (content.contains(phrase)) {
                score += SpamRules.SUSPICIOUS_PHRASE;
                reasons.add("Suspicious phrase: " + phrase);
                // Distinct signals accumulate; one phrase alone should not classify a message.
                if (++phraseMatches == 3) break;
            }
        }
        if (LINK.matcher(body).results().limit(5).count() >= 5) {
            score += SpamRules.MANY_LINKS;
            reasons.add("Message contains many links");
        }
        if (subject.contains("urgent") || subject.contains("act now")) {
            score += SpamRules.URGENT_SUBJECT;
            reasons.add("Urgent subject wording");
        }
        if (sender.isBlank()) reasons.add("Sender address is missing");
        return new SpamResult(score, score >= SpamRules.SPAM_THRESHOLD, reasons);
    }
}
```

src/main/java/com/emailapp/spam/SpamRules.java

```
package com.emailapp.spam;

public final class SpamRules {

    private SpamRules() {
        // Prevent object creation
    }

    public static final int BLOCKED_SENDER = 50;
    public static final int SUSPICIOUS_PHRASE = 20;
    public static final int MANY_LINKS = 15;
    public static final int URGENT_SUBJECT = 10;

    public static final int SPAM_THRESHOLD = 50;
}
```

src/main/java/ems/App.java

```
package ems;

import ems.config.AppConfig;
import ems.exception.AppException;
import ems.service.GmailEmailService;
import ems.storage.JdbcMailStore;
import ems.ui.MainFrame;
import ems.ui.UiTheme;
import java.nio.file.Files;
import java.sql.DriverManager;
import javax.swing.JOptionPane;
import javax.swing.SwingUtilities;

/** Production entry point. The demo has its own explicitly labelled launcher. */
public final class App {
    private App() { }

    public static void main(String[] args) {
        if (args.length > 0 && args[0].equals("--help")) {
            System.out.println("Maildesk - Java Gmail client\n"
                    + "Run: java -jar target/maildesk.jar\n"
```

```
                    + "Check setup: java -jar target/maildesk.jar --check-setup\n"
                    + "Configuration: config/app.properties (or DB_URL, DB_USER, DB_PASSWORD, GMAIL_CREDENTIALS,
GMAIL_TOKEN_DIR).\n"
                    + "See README.md for Google Desktop OAuth and MySQL setup.");
            return;
        }
        if (args.length > 1 || args.length == 1 && !args[0].equals("--check-setup")) {
            System.err.println("Unknown option. Use --help or --check-setup.");
            System.exit(2);
        }
        try {
            AppConfig config = AppConfig.load();
            JdbcMailStore store = new JdbcMailStore(() -> DriverManager.getConnection(
                    config.getDatabaseUrl(), config.getDatabaseUser(), config.getDatabasePassword()));
            if (args.length == 1) {
                boolean credentials = Files.isRegularFile(config.getGmailCredentialsPath());
                System.out.println("Google Desktop OAuth JSON: " + (credentials ? "found" : "missing - follow
README.md setup"));
                boolean database = false;
                try { store.initialize(); database = true; System.out.println("MySQL: connected; schema
ready."); }
                catch (java.sql.SQLException exception) {
                    System.out.println("MySQL: unavailable - check db.url, db.user and db.password in
config/app.properties.");
                }
                System.out.println("Live Gmail access also requires the user to authorize the app in their
browser.");
                if (!credentials || !database) System.exit(1);
                return;
            }
            GmailEmailService service = GmailEmailService.createDefault();
            SwingUtilities.invokeLater(() -> {
                UiTheme.install();
                new MainFrame(service).setVisible(true);
            });
        } catch (AppException | RuntimeException exception) {
            String message = exception instanceof AppException ? exception.getMessage()
                    : "Maildesk could not start. Check config/app.properties and use --check-setup.";
            System.err.println(message);
            if (args.length == 0 && !java.awt.GraphicsEnvironment.isHeadless()) {
                SwingUtilities.invokeLater(() -> JOptionPane.showMessageDialog(null, message,
                        "Maildesk setup", JOptionPane.ERROR_MESSAGE));
            }
            System.exit(1);
        }
    }
}
```

src/main/java/ems/config/AppConfig.java

```
package ems.config;

import ems.exception.AppException;
import java.io.IOException;
import java.io.Reader;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Map;
import java.util.Properties;

/** Local-only configuration; environment values override an optional properties file. */
public final class AppConfig {
    private final Path home;
    private final Path configFile;
    private final Properties properties;
    private final Map<String, String> environment;

    private AppConfig(Path home, Path configFile, Properties properties, Map<String, String> environment) {
        this.home = home.toAbsolutePath().normalize();
        this.configFile = configFile.toAbsolutePath().normalize();
```

```
        this.properties = properties;
        this.environment = Map.copyOf(environment);
    }

    public static AppConfig load() throws AppException {
        return load(Path.of(System.getProperty("maildesk.home", ".")), System.getenv());
    }

    public static AppConfig load(Path home, Map<String, String> environment) throws AppException {
        Path normalized = home.toAbsolutePath().normalize();
        Path file = normalized.resolve("config/app.properties");
        Properties properties = new Properties();
        if (Files.exists(file)) {
            try (Reader reader = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
                properties.load(reader);
            } catch (IOException | IllegalArgumentException exception) {
                throw new AppException("Cannot read config/app.properties. Check the local file format and
permissions.", exception);
            }
        }
        return new AppConfig(normalized, file, properties, environment);
    }

    private String value(String environmentKey, String propertyKey, String fallback) {
        return environment.containsKey(environmentKey) ? environment.get(environmentKey)
                : properties.getProperty(propertyKey, fallback);
    }

    public String getDatabaseUrl() {
        return value("DB_URL", "db.url", "jdbc:mysql://localhost:3306/email_management"
                + "?createDatabaseIfNotExist=true&connectionTimeZone=UTC&forceConnectionTimeZoneToSession=true"
                + "&connectTimeout=5000&socketTimeout=15000").strip();
    }

    public String getDatabaseUser() { return value("DB_USER", "db.user", "root").strip(); }
    public String getDatabasePassword() { return value("DB_PASSWORD", "db.password", ""); }
    public Path getGmailCredentialsPath() {
        if (environment.containsKey("GMAIL_CREDENTIALS") || properties.containsKey("gmail.credentials")) {
            return localPath(value("GMAIL_CREDENTIALS", "gmail.credentials", "config/credentials.json"));
        }
        // Accept the filenames documented by Member 2, with the local config taking precedence.
        for (String candidate : new String[] {"config/credentials.json", "config/client_secret.json",
                "credentials.json", "client_secret.json", "client_secrets.json"}) {
            Path path = home.resolve(candidate);
            if (Files.isRegularFile(path)) return path;
        }
        return home.resolve("config/credentials.json");
    }
    public Path getTokenDirectory() {
        return localPath(value("GMAIL_TOKEN_DIR", "gmail.tokenDir", "config/tokens"));
    }
    public Path getConfigFile() { return configFile; }
    public Path getHome() { return home; }

    private Path localPath(String value) {
        Path path = Path.of(value.strip());
        return (path.isAbsolute() ? path : home.resolve(path)).normalize();
    }
}
```

src/main/java/ems/exception/AppException.java

```
package ems.exception;

/** A recoverable application failure whose message can be displayed to the user. */
public class AppException extends Exception {
    private static final long serialVersionUID = 1L;
```

```
    public AppException(String message) {
        super(message);
    }

    public AppException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

src/main/java/ems/gmail/AccessTokenProvider.java

```
package ems.gmail;

import ems.exception.AppException;

/** Separates authorization from HTTP mailbox operations and permits local-server tests. */
public interface AccessTokenProvider {
    void authorize() throws AppException;
    String accessToken() throws AppException;
    String refreshAccessToken() throws AppException;
    void disconnect();
}
```

src/main/java/ems/gmail/GmailApiClient.java

```
package ems.gmail;

import com.google.gson.Gson;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import ems.exception.AppException;
import ems.model.EmailMessage;
import java.io.IOException;
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.nio.file.Path;
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

/** Gmail REST client using an OAuth desktop session and standards-compliant MIME messages. */
public final class GmailApiClient implements GmailGateway {
    private static final URI GOOGLE_API = URI.create("https://gmail.googleapis.com/gmail/v1/users/me/");
    private static final Gson JSON = new Gson();
    private final URI baseUri;
    private final HttpClient http;
    private final AccessTokenProvider tokens;
    private final RetryDelay retryDelay;

    public GmailApiClient(Path credentialsFile, Path tokenDirectory) {
        this(GOOGLE_API, HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(20)).build(),
                new GmailAuth(credentialsFile, tokenDirectory));
    }

    /** The injected endpoint and token provider support deterministic local HTTP tests. */
    public GmailApiClient(URI baseUri, HttpClient http, AccessTokenProvider tokens) {
        this(baseUri, http, tokens, Thread::sleep);
    }
```

```
    GmailApiClient(URI baseUri, HttpClient http, AccessTokenProvider tokens, RetryDelay retryDelay) {
        if (!baseUri.toString().endsWith("/")) {
            throw new IllegalArgumentException("Gmail API base URI must end with '/'.");
        }
        this.baseUri = baseUri;
        this.http = http;
        this.tokens = tokens;
        this.retryDelay = retryDelay;
    }

    @Override public String connect() throws AppException {
        tokens.authorize();
        JsonObject profile = request("GET", "profile", null);
        return requiredString(profile, "emailAddress");
    }

    @Override public List<EmailMessage> fetchMessages(long accountId) throws AppException {
        Set<String> ids = new LinkedHashSet<>();
        Set<String> pageTokens = new HashSet<>();
        String pageToken = "";
        do {
            String path = "messages?maxResults=500&includeSpamTrash=true&q="
                    + encode("-in:trash -in:drafts");
            if (!pageToken.isEmpty()) path += "&pageToken=" + encode(pageToken);
            JsonObject page = request("GET", path, null);
            try {
                if (page.has("messages")) {
                    for (var value : page.getAsJsonArray("messages")) {
                        ids.add(requiredString(value.getAsJsonObject(), "id"));
                    }
                }
                pageToken = page.has("nextPageToken") ? requiredString(page, "nextPageToken") : "";
            } catch (RuntimeException malformedPage) {
                throw new AppException("Gmail returned an unreadable mailbox page. Please refresh again.",
malformedPage);
            }
            if (!pageToken.isEmpty() && !pageTokens.add(pageToken)) {
                throw new AppException("Gmail repeated a mailbox page. Please refresh again.");
            }
        } while (!pageToken.isEmpty());
        return readMessages(accountId, ids);
    }

    /** Bounded startup window; never presented as a complete mailbox snapshot. */
    @Override public List<EmailMessage> fetchRecentMessages(long accountId, int perMailbox) throws AppException
{
        int limit = Math.max(1, Math.min(perMailbox, 100));
        Set<String> ids = new LinkedHashSet<>();
        for (String label : List.of("INBOX", "SENT", "SPAM")) {
            JsonObject page = request("GET", "messages?maxResults=" + limit
                    + "&includeSpamTrash=true&labelIds=" + label + "&q=" + encode("-in:trash -in:drafts"),
null);
            try {
                if (page.has("messages")) {
                    int count = 0;
                    for (var item : page.getAsJsonArray("messages")) {
                        if (count++ >= limit) break;
                        ids.add(requiredString(item.getAsJsonObject(), "id"));
                    }
                }
            } catch (RuntimeException malformedPage) {
                throw new AppException("Gmail returned an unreadable recent-mail page. Click Refresh to try
again.", malformedPage);
            }
        }
        return readMessages(accountId, ids);
    }

    private List<EmailMessage> readMessages(long accountId, Set<String> ids) throws AppException {
        List<EmailMessage> result = new ArrayList<>();
```

```
        for (String id : ids) {
            try {
                result.add(getMessage(accountId, id));
            } catch (MessageUnavailableException deletedDuringSync) {
                // A message can disappear between the list and get calls.
            }
        }
        return result;
    }

    @Override public EmailMessage getMessage(long accountId, String id) throws AppException {
        JsonObject message = request("GET", "messages/" + messageId(id) + "?format=raw", null);
        Set<String> labels = new HashSet<>();
        try {
            if (message.has("labelIds")) {
                for (var label : message.getAsJsonArray("labelIds")) labels.add(label.getAsString());
            }
        } catch (RuntimeException malformedLabels) {
            throw new AppException("Gmail returned unreadable message labels. Please refresh again.",
malformedLabels);
        }
        String raw = requiredString(message, "raw");
        Instant received = null;
        if (message.has("internalDate")) {
            try { received = Instant.ofEpochMilli(message.get("internalDate").getAsLong()); }
            catch (RuntimeException invalidDate) {
                throw new AppException("Gmail returned an invalid message date.", invalidDate);
            }
        }
        return MimeMessages.parse(accountId, id, raw, received, labels);
    }

    @Override public String sendMessage(String senderEmail, String recipientEmail, String subject,
            String body) throws AppException {
        JsonObject message = new JsonObject();
        message.addProperty("raw", MimeMessages.compose(senderEmail, recipientEmail, subject, body));
        JsonObject response = request("POST", "messages/send", message);
        try {
            return requiredString(response, "id");
        } catch (AppException incompleteSendResponse) {
            throw new AppException("Gmail accepted the send request but returned no message ID. "
                    + "Check Sent before sending again.");
        }
    }

    @Override public void setRead(String id) throws AppException {
        modify(id, Set.of(), Set.of("UNREAD"));
    }

    @Override public void setSpam(String id, boolean spam) throws AppException {
        modify(id, spam ? Set.of("SPAM") : Set.of("INBOX"),
                spam ? Set.of("INBOX") : Set.of("SPAM"));
    }

    private void modify(String id, Set<String> add, Set<String> remove) throws AppException {
        JsonObject body = new JsonObject();
        body.add("addLabelIds", JSON.toJsonTree(add));
        body.add("removeLabelIds", JSON.toJsonTree(remove));
        request("POST", "messages/" + messageId(id) + "/modify", body);
    }

    @Override public void disconnect() { tokens.disconnect(); }

    private JsonObject request(String method, String path, JsonObject body) throws AppException {
        boolean sending = "POST".equals(method) && path.equals("messages/send");
        boolean refreshed = false;
        int retries = 0;
        while (true) {
            String token = tokens.accessToken();
            HttpRequest.Builder builder = HttpRequest.newBuilder(baseUri.resolve(path))
```

```
                .timeout(Duration.ofSeconds(60))
                .header("Authorization", "Bearer " + token)
                .header("Accept", "application/json");
        if (body == null) builder.GET();
        else builder.header("Content-Type", "application/json; charset=UTF-8")
                .method(method, HttpRequest.BodyPublishers.ofString(JSON.toJson(body)));
        try {
            HttpResponse<String> response = http.send(builder.build(),
                    HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8));
            int status = response.statusCode();
            if (status == 401 && !refreshed) {
                tokens.refreshAccessToken();
                refreshed = true;
                continue;
            }
            if (status < 200 || status >= 300) {
                Set<String> reasons = errorReasons(response.body());
                if (method.equals("GET") && retries < 6 && transientFailure(status, reasons)) {
                    retryDelay.pause(1000L << retries++);
                    continue;
                }
                if (sending && status >= 500) {
                    throw new AppException("Could not confirm delivery. Check Gmail Sent before retrying.");
                }
                throw apiFailure(status, reasons);
            }
            if (response.body().isBlank()) return new JsonObject();
            return JsonParser.parseString(response.body()).getAsJsonObject();
        } catch (IOException e) {
            throw new AppException(sending
                    ? "Could not confirm delivery. Check Gmail Sent before retrying."
                    : "Could not reach Gmail. Check your internet connection and try again.", e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new AppException(sending
                    ? "Could not confirm delivery. Check Gmail Sent before retrying."
                    : "The Gmail operation was interrupted.", e);
        } catch (RuntimeException e) {
            throw new AppException(sending
                    ? "Could not confirm delivery. Check Gmail Sent before retrying."
                    : "Gmail returned an unreadable response. Please try again.", e);
        }
    }
}

private static boolean transientFailure(int status, Set<String> reasons) {
    return status == 429 || status == 500 || status == 502 || status == 503 || status == 504
            || status == 403 && (reasons.contains("rateLimitExceeded")
            || reasons.contains("userRateLimitExceeded"));
}

/** Read only machine error codes; never echo Google's response or account details. */
private static Set<String> errorReasons(String body) {
    Set<String> result = new HashSet<>();
    try {
        JsonObject error = JsonParser.parseString(body).getAsJsonObject().getAsJsonObject("error");
        for (String field : List.of("errors", "details")) {
            if (error != null && error.has(field) && error.get(field).isJsonArray()) {
                for (var item : error.getAsJsonArray(field)) {
                    if (item.isJsonObject() && item.getAsJsonObject().has("reason")) {
                        result.add(item.getAsJsonObject().get("reason").getAsString());
                    }
                }
            }
        }
    } catch (RuntimeException ignored) { /* Fall back to a safe HTTP-status explanation. */ }
    return result;
}

private static AppException apiFailure(int status, Set<String> reasons) {
    if (status == 403) {
        if (reasons.contains("dailyLimitExceeded") || reasons.contains("quotaExceeded")) {
```

```
                return new AppException("The Google Cloud project's Gmail API quota is exhausted. "
                        + "Check Gmail API quotas in Google Cloud or wait for the quota to reset.");
            }
            if (transientFailure(status, reasons)) {
                return new AppException("Gmail temporarily limited requests. Your sign-in is still valid. "
                        + "Wait a moment, then click Refresh.");
            }
            if (reasons.contains("domainPolicy")) {
                return new AppException("Your Google Workspace administrator blocks Gmail access for this app. "
                        + "Ask your administrator to allow it.");
            }
            if (reasons.contains("insufficientPermissions") ||
reasons.contains("ACCESS_TOKEN_SCOPE_INSUFFICIENT")) {
                return new AppException("The Google sign-in did not grant the required Gmail permission. "
                        + "Disconnect, connect again, and allow Maildesk to manage Gmail.");
            }
        }
        return switch (status) {
            case 401 -> new AppException("Gmail authorization expired. Disconnect and connect again.");
            case 403 -> new AppException("Gmail denied access. Enable the Gmail API in your Google Cloud "
                    + "project, add your account as a test user, and grant Gmail permissions when connecting.");
            case 404 -> new MessageUnavailableException();
            case 429 -> new AppException("Gmail's request limit was reached. Wait a moment and refresh again.");
            default -> new AppException("Gmail could not complete the request (HTTP " + status + "). Please try
again.");
        };
    }

    @FunctionalInterface interface RetryDelay { void pause(long milliseconds) throws InterruptedException; }

    private static String messageId(String id) throws AppException {
        if (id == null || !id.matches("[A-Za-z0-9_-]+")) throw new AppException("Invalid Gmail message ID.");
        return id;
    }

    private static String encode(String value) {
        return URLEncoder.encode(value, StandardCharsets.UTF_8);
    }

    private static String requiredString(JsonObject object, String key) throws AppException {
        if (!object.has(key) || !object.get(key).isJsonPrimitive()
                || !object.get(key).getAsJsonPrimitive().isString()
                || object.get(key).getAsString().isBlank()) {
            throw new AppException("Gmail returned an incomplete response. Please refresh again.");
        }
        return object.get(key).getAsString();
    }

    private static final class MessageUnavailableException extends AppException {
        private static final long serialVersionUID = 1L;
        private MessageUnavailableException() { super("This message is no longer available in Gmail. Refresh the
mailbox."); }
    }
}
```

src/main/java/ems/gmail/GmailAuth.java

```
package ems.gmail;

import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import ems.exception.AppException;
import java.awt.Desktop;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.URLDecoder;
```

```
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.time.Duration;
import java.time.Instant;
import java.util.Base64;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

/** OAuth Authorization Code + PKCE using the system browser and a loopback-only callback. */
public final class GmailAuth implements AccessTokenProvider {
    static final String SCOPE = "https://www.googleapis.com/auth/gmail.modify";
    private static final URI GOOGLE_AUTHORIZE = URI.create("https://accounts.google.com/o/oauth2/v2/auth");
    private static final URI GOOGLE_TOKEN = URI.create("https://oauth2.googleapis.com/token");
    private final Path credentialsFile;
    private final TokenStore store;
    private final HttpClient http;
    private final URI authorizeEndpoint;
    private final URI tokenEndpoint;
    private final BrowserOpener browser;
    private final Duration timeout;
    private String clientId;
    private String clientSecret;
    private String accessToken;
    private String refreshToken;
    private Instant expiresAt = Instant.EPOCH;
    private boolean ignoreSavedTokens;

    public GmailAuth(Path credentialsFile, Path tokenDirectory) {
        this(credentialsFile, tokenDirectory,
                HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(20)).build(),
                GOOGLE_AUTHORIZE, GOOGLE_TOKEN, GmailAuth::openBrowser, Duration.ofMinutes(3));
    }

    GmailAuth(Path credentialsFile, Path tokenDirectory, HttpClient http,
            URI authorizeEndpoint, URI tokenEndpoint, BrowserOpener browser, Duration timeout) {
        this.credentialsFile = credentialsFile;
        this.store = new TokenStore(tokenDirectory);
        this.http = http;
        this.authorizeEndpoint = authorizeEndpoint;
        this.tokenEndpoint = tokenEndpoint;
        this.browser = browser;
        this.timeout = timeout;
    }

    @Override public synchronized void authorize() throws AppException {
        readCredentials();
        if (accessToken != null && !expiresAt.isBefore(Instant.now().plusSeconds(60))) return;
        if (!ignoreSavedTokens && accessToken == null) loadTokens();
        if (accessToken != null && !expiresAt.isBefore(Instant.now().plusSeconds(60))) return;
        if (refreshToken != null) {
            try {
                refreshAccessToken();
                return;
            } catch (AuthorizationExpired expired) {
                // Revoked/expired grants require a new explicit authorization in the browser.
            }
        }
        authorizeInBrowser();
    }
```

```
    @Override public synchronized String accessToken() throws AppException {
        if (accessToken == null) throw new AppException("Connect a Gmail account first.");
        if (expiresAt.isBefore(Instant.now().plusSeconds(60))) return refreshAccessToken();
        return accessToken;
    }

    @Override public synchronized String refreshAccessToken() throws AppException {
        if (refreshToken == null) {
            forgetTokens();
            throw new AuthorizationExpired();
        }
        Map<String, String> form = clientFields();
        form.put("grant_type", "refresh_token");
        form.put("refresh_token", refreshToken);
        applyTokens(exchangeToken(form));
        return accessToken;
    }

    @Override public synchronized void disconnect() { forgetTokens(); }

    private void readCredentials() throws AppException {
        if (!Files.isRegularFile(credentialsFile)) {
            throw new AppException("Gmail setup is required. Enable the Gmail API, create a Desktop app "
                    + "OAuth client in Google Cloud, and save its downloaded JSON as "
                    + credentialsFile.toAbsolutePath() + ". See README.md for setup steps.");
        }
        try {
            JsonObject root = JsonParser.parseString(Files.readString(credentialsFile,
                    StandardCharsets.UTF_8)).getAsJsonObject();
            if (!root.has("installed")) {
                throw new AppException("Use Google OAuth credentials for a Desktop app, not a Web
application.");
            }
            JsonObject installed = root.getAsJsonObject("installed");
            clientId = installed.get("client_id").getAsString();
            clientSecret = installed.has("client_secret") ? installed.get("client_secret").getAsString() : "";
            if (clientId.isBlank()) throw new IllegalArgumentException("Missing client ID");
        } catch (AppException e) {
            throw e;
        } catch (IOException | RuntimeException e) {
            throw new AppException("The Gmail credentials file could not be read. Download new Desktop app
credentials.", e);
        }
    }

    private void loadTokens() throws AppException {
        try {
            JsonObject data = store.load();
            if (data == null || !clientId.equals(string(data, "client_id"))) return;
            accessToken = string(data, "access_token");
            refreshToken = string(data, "refresh_token");
            expiresAt = data.has("expires_at") ? Instant.ofEpochMilli(data.get("expires_at").getAsLong())
                    : Instant.EPOCH;
        } catch (IOException | RuntimeException e) {
            throw new AppException("Could not read the protected Gmail sign-in cache. Check permissions on the
token directory.", e);
        }
    }

    private void authorizeInBrowser() throws AppException {
        HttpServer server = null;
        try {
            String state = randomToken();
            String verifier = randomToken();
            String challenge = Base64.getUrlEncoder().withoutPadding().encodeToString(
                    MessageDigest.getInstance("SHA-256").digest(verifier.getBytes(StandardCharsets.US_ASCII)));
            server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
            URI redirect = URI.create("http://127.0.0.1:" + server.getAddress().getPort() + "/oauth2callback");
            CompletableFuture<String> code = new CompletableFuture<>();
            server.createContext("/oauth2callback", exchange -> handleCallback(exchange, state, code));
            server.start();
```

```
            Map<String, String> form = new LinkedHashMap<>();
            form.put("client_id", clientId);
            form.put("redirect_uri", redirect.toString());
            form.put("response_type", "code");
            form.put("scope", SCOPE);
            form.put("access_type", "offline");
            form.put("prompt", "consent select_account");
            form.put("state", state);
            form.put("code_challenge", challenge);
            form.put("code_challenge_method", "S256");
            browser.open(URI.create(authorizeEndpoint + "?" + formEncode(form)));
            String authCode = code.get(timeout.toMillis(), TimeUnit.MILLISECONDS);
            Map<String, String> tokenForm = clientFields();
            tokenForm.put("grant_type", "authorization_code");
            tokenForm.put("code", authCode);
            tokenForm.put("redirect_uri", redirect.toString());
            tokenForm.put("code_verifier", verifier);
            applyTokens(exchangeToken(tokenForm));
        } catch (AppException e) {
            throw e;
        } catch (TimeoutException e) {
            throw new AppException("Google sign-in timed out. Click Connect Gmail and complete sign-in within
three minutes.", e);
        } catch (ExecutionException e) {
            if (e.getCause() instanceof AppException app) throw app;
            throw new AppException("Google sign-in could not be completed. Please try again.", e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new AppException("Google sign-in was interrupted.", e);
        } catch (Exception e) {
            throw new AppException("Could not open Google sign-in. Check that a default web browser is available
"
                    + "and local loopback connections are allowed.", e);
        } finally {
            if (server != null) server.stop(0);
        }
    }

    private static void handleCallback(HttpExchange exchange, String state, CompletableFuture<String> result)
            throws IOException {
        int status = 400;
        String message = "Invalid sign-in response. Return to the application and try again.";
        String authorizationCode = null;
        AppException authorizationFailure = null;
        try {
            Map<String, String> query = parseQuery(exchange.getRequestURI().getRawQuery());
            String returnedState = query.getOrDefault("state", "");
            boolean valid = exchange.getRequestMethod().equals("GET")
                    && exchange.getRequestURI().getPath().equals("/oauth2callback")
                    && MessageDigest.isEqual(state.getBytes(StandardCharsets.UTF_8),
                            returnedState.getBytes(StandardCharsets.UTF_8));
            if (valid && query.containsKey("error")) {
                authorizationFailure = new AppException("Google sign-in was cancelled or access was denied.");
                message = "Sign-in was not completed. You can close this tab and return to the application.";
            } else if (valid && !query.getOrDefault("code", "").isBlank()) {
                status = 200;
                message = "Authorization received. You can close this tab and return to the application.";
                authorizationCode = query.get("code");
            }
        } catch (IllegalArgumentException malformedQuery) {
            // Reject malformed callbacks without revealing query data or cancelling a legitimate response.
        }
        byte[] page = ("<!doctype html><html><head><meta charset='utf-8'><title>Gmail connection</title>"
                + "</head><body><p>" + message + "</p></body></html>").getBytes(StandardCharsets.UTF_8);
        exchange.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
        exchange.getResponseHeaders().set("Cache-Control", "no-store");
        exchange.getResponseHeaders().set("Content-Security-Policy", "default-src 'none'");
        try {
            exchange.sendResponseHeaders(status, page.length);
            try (var output = exchange.getResponseBody()) { output.write(page); }
        } finally {
            // Complete only after the browser response is written; otherwise a fast denial
            // can stop the listener while it is still displaying the return-to-app page.
```

```
            if (authorizationCode != null) result.complete(authorizationCode);
            else if (authorizationFailure != null) result.completeExceptionally(authorizationFailure);
        }
    }

    private JsonObject exchangeToken(Map<String, String> form) throws AppException {
        HttpRequest request = HttpRequest.newBuilder(tokenEndpoint).timeout(Duration.ofSeconds(45))
                .header("Content-Type", "application/x-www-form-urlencoded")
                .POST(HttpRequest.BodyPublishers.ofString(formEncode(form))).build();
        try {
            HttpResponse<String> response = http.send(request,
                    HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8));
            if (response.statusCode() < 200 || response.statusCode() >= 300) {
                if (response.body().contains("invalid_grant")) {
                    forgetTokens();
                    throw new AuthorizationExpired();
                }
                throw new AppException("Google could not authorize Gmail (HTTP " + response.statusCode()
                        + "). Check your OAuth client configuration and test-user access.");
            }
            return JsonParser.parseString(response.body()).getAsJsonObject();
        } catch (IOException e) {
            throw new AppException("Could not reach Google's sign-in server. Check your internet connection.",
e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new AppException("Google sign-in was interrupted.", e);
        } catch (RuntimeException e) {
            throw new AppException("Google returned an unreadable sign-in response.", e);
        }
    }

    private void applyTokens(JsonObject data) throws AppException {
        try {
            String newToken = string(data, "access_token");
            if (newToken == null || newToken.isBlank()) throw new IllegalArgumentException("Missing token");
            if (data.has("scope") && !java.util.Arrays.asList(data.get("scope").getAsString().split(" "))
                    .contains(SCOPE)) {
                throw new AppException("Gmail permission was not granted. Connect again and allow Gmail
access.");
            }
            accessToken = newToken;
            String newRefresh = string(data, "refresh_token");
            if (newRefresh != null && !newRefresh.isBlank()) refreshToken = newRefresh;
            long expires = data.has("expires_in") ? data.get("expires_in").getAsLong() : 3600;
            expiresAt = Instant.now().plusSeconds(Math.max(1, expires));
            JsonObject cache = new JsonObject();
            cache.addProperty("client_id", clientId);
            cache.addProperty("access_token", accessToken);
            if (refreshToken != null) cache.addProperty("refresh_token", refreshToken);
            cache.addProperty("expires_at", expiresAt.toEpochMilli());
            store.save(cache);
            ignoreSavedTokens = false;
        } catch (AppException e) {
            throw e;
        } catch (IOException e) {
            throw new AppException("Gmail authorized, but the protected sign-in cache could not be saved. "
                    + "Check permissions on the token directory.", e);
        } catch (RuntimeException e) {
            throw new AppException("Google returned incomplete sign-in credentials. Connect again.", e);
        }
    }

    private Map<String, String> clientFields() {
        Map<String, String> form = new LinkedHashMap<>();
        form.put("client_id", clientId);
        if (clientSecret != null && !clientSecret.isBlank()) form.put("client_secret", clientSecret);
        return form;
    }

    private void forgetTokens() {
        accessToken = null;
```

```
        refreshToken = null;
        expiresAt = Instant.EPOCH;
        ignoreSavedTokens = true;
        try { store.clear(); } catch (IOException ignored) {
            // The current process will never reuse a cache after disconnect, even if deletion fails.
        }
    }

    static Map<String, String> parseQuery(String query) {
        Map<String, String> values = new LinkedHashMap<>();
        if (query == null || query.isEmpty()) return values;
        for (String pair : query.split("&")) {
            String[] parts = pair.split("=", 2);
            String key = URLDecoder.decode(parts[0], StandardCharsets.UTF_8);
            String value = URLDecoder.decode(parts.length == 2 ? parts[1] : "", StandardCharsets.UTF_8);
            if (values.putIfAbsent(key, value) != null) throw new IllegalArgumentException("Duplicate
parameter");
        }
        return values;
    }

    static String formEncode(Map<String, String> form) {
        return form.entrySet().stream().map(entry -> URLEncoder.encode(entry.getKey(), StandardCharsets.UTF_8)
                + "=" + URLEncoder.encode(entry.getValue(), StandardCharsets.UTF_8))
                .collect(java.util.stream.Collectors.joining("&"));
    }

    private static String string(JsonObject object, String key) {
        return object.has(key) && !object.get(key).isJsonNull() ? object.get(key).getAsString() : null;
    }

    private static String randomToken() {
        byte[] bytes = new byte[32];
        new SecureRandom().nextBytes(bytes);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(bytes);
    }

    private static void openBrowser(URI uri) throws IOException {
        if (!Desktop.isDesktopSupported() || !Desktop.getDesktop().isSupported(Desktop.Action.BROWSE)) {
            throw new IOException("No default browser is available.");
        }
        Desktop.getDesktop().browse(uri);
    }

    @FunctionalInterface
    interface BrowserOpener { void open(URI uri) throws Exception; }

    private static final class AuthorizationExpired extends AppException {
        private static final long serialVersionUID = 1L;
        private AuthorizationExpired() { super("Gmail authorization expired or was revoked. Connect Gmail
again."); }
    }
}
```

src/main/java/ems/gmail/GmailGateway.java

```
package ems.gmail;

import ems.exception.AppException;
import ems.model.EmailMessage;
import java.util.List;

/** Remote Gmail operations. Implementations must not expose tokens in errors. */
public interface GmailGateway {
    String connect() throws AppException;
    List<EmailMessage> fetchMessages(long accountId) throws AppException;
    default List<EmailMessage> fetchRecentMessages(long accountId, int perMailbox) throws AppException {
```

```
        return fetchMessages(accountId);
    }
    EmailMessage getMessage(long accountId, String gmailMessageId) throws AppException;
    String sendMessage(String senderEmail, String recipientEmail, String subject, String body)
            throws AppException;
    void setRead(String gmailMessageId) throws AppException;
    void setSpam(String gmailMessageId, boolean spam) throws AppException;
    void disconnect();
}
```

src/main/java/ems/gmail/MimeMessages.java

```
package ems.gmail;

import ems.exception.AppException;
import ems.model.EmailMessage;
import jakarta.mail.Address;
import jakarta.mail.Message;
import jakarta.mail.Multipart;
import jakarta.mail.Part;
import jakarta.mail.Session;
import jakarta.mail.internet.InternetAddress;
import jakarta.mail.internet.MimeMessage;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.StringReader;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.ArrayList;
import java.util.Base64;
import java.util.Date;
import java.util.List;
import java.util.Properties;
import java.util.Set;
import javax.swing.text.MutableAttributeSet;
import javax.swing.text.html.HTML;
import javax.swing.text.html.HTMLEditorKit;
import javax.swing.text.html.parser.ParserDelegator;

/** RFC 822/MIME conversion. HTML is converted to inert text, never rendered or fetched. */
final class MimeMessages {
    private MimeMessages() {}

    static EmailMessage parse(long accountId, String id, String raw, Instant received,
            Set<String> labels) throws AppException {
        try {
            byte[] bytes = Base64.getUrlDecoder().decode(raw);
            MimeMessage mime = new MimeMessage(Session.getInstance(new Properties()),
                    new ByteArrayInputStream(bytes));
            Address[] from = mime.getFrom();
            String sender = from == null || from.length == 0 ? "" : address(from[0]);
            List<String> recipients = new ArrayList<>();
            Address[] allRecipients = mime.getAllRecipients();
            if (allRecipients != null) {
                for (Address recipient : allRecipients) recipients.add(address(recipient));
            }
            Date sentDate = mime.getSentDate();
            Instant date = received != null ? received
                    : sentDate == null ? Instant.EPOCH : sentDate.toInstant();
            return new EmailMessage(accountId, id, sender, recipients,
                    mime.getSubject() == null ? "" : mime.getSubject(),
                    readText(mime, 0), date, labels);
        } catch (Exception e) {
            throw new AppException("Gmail returned a message that could not be read.", e);
        }
    }

    static String compose(String sender, String recipient, String subject, String body)
            throws AppException {
```

```
    try {
        MimeMessage mime = new MimeMessage(Session.getInstance(new Properties()));
        mime.setFrom(validAddress(sender));
        mime.setRecipient(Message.RecipientType.TO, validAddress(recipient));
        if (subject == null || subject.contains("\r") || subject.contains("\n")) {
            throw new AppException("The subject must be a single line.");
        }
        mime.setSubject(subject, StandardCharsets.UTF_8.name());
        mime.setText(body == null ? "" : body, StandardCharsets.UTF_8.name());
        mime.setSentDate(new Date());
        mime.saveChanges();
        ByteArrayOutputStream bytes = new ByteArrayOutputStream();
        mime.writeTo(bytes);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(bytes.toByteArray());
    } catch (AppException e) {
        throw e;
    } catch (Exception e) {
        throw new AppException("Enter a valid sender and recipient email address.", e);
    }
}

private static InternetAddress validAddress(String value) throws Exception {
    if (value == null || value.contains("\r") || value.contains("\n")) {
        throw new AppException("Enter one valid recipient email address.");
    }
    InternetAddress[] addresses = InternetAddress.parse(value, true);
    if (addresses.length != 1) throw new AppException("Enter one recipient email address.");
    addresses[0].validate();
    return addresses[0];
}

private static String address(Address address) {
    return address instanceof InternetAddress internet ? internet.getAddress() : address.toString();
}

private static String readText(Part part, int depth) throws Exception {
    if (depth > 30 || Part.ATTACHMENT.equalsIgnoreCase(part.getDisposition())
            || (part.getFileName() != null && !part.getFileName().isBlank())) return "";
    if (part.isMimeType("text/plain")) return contentString(part);
    if (part.isMimeType("text/html")) return htmlToText(contentString(part));
    if (part.isMimeType("multipart/*")) {
        Multipart multi = (Multipart) part.getContent();
        // Prefer the text/plain alternative instead of displaying duplicate HTML text.
        if (part.isMimeType("multipart/alternative")) {
            String fallback = "";
            for (int i = 0; i < multi.getCount(); i++) {
                Part child = multi.getBodyPart(i);
                String text = readText(child, depth + 1);
                if (child.isMimeType("text/plain") && !text.isBlank()) return text;
                if (!text.isBlank()) fallback = text;
            }
            return fallback;
        }
        List<String> pieces = new ArrayList<>();
        for (int i = 0; i < multi.getCount(); i++) {
            String text = readText(multi.getBodyPart(i), depth + 1);
            if (!text.isBlank()) pieces.add(text);
        }
        return String.join("\n\n", pieces);
    }
    return "";
}

private static String contentString(Part part) throws Exception {
    Object content = part.getContent();
    return content instanceof String text ? text : "";
}

private static String htmlToText(String html) throws IOException {
    StringBuilder text = new StringBuilder();
    new ParserDelegator().parse(new StringReader(html), new HTMLEditorKit.ParserCallback() {
```

```
            private int hiddenDepth;
            @Override public void handleStartTag(HTML.Tag tag, MutableAttributeSet attrs, int pos) {
                if (tag == HTML.Tag.SCRIPT || tag == HTML.Tag.STYLE) hiddenDepth++;
                if (hiddenDepth == 0 && tag.isBlock()) newline();
            }
            @Override public void handleEndTag(HTML.Tag tag, int pos) {
                if (tag == HTML.Tag.SCRIPT || tag == HTML.Tag.STYLE) {
                    hiddenDepth = Math.max(0, hiddenDepth - 1);
                }
                if (hiddenDepth == 0 && tag.isBlock()) newline();
            }
            @Override public void handleSimpleTag(HTML.Tag tag, MutableAttributeSet attrs, int pos) {
                if (tag == HTML.Tag.BR) newline();
            }
            @Override public void handleText(char[] data, int pos) {
                if (hiddenDepth == 0) text.append(data);
            }
            private void newline() {
                if (text.length() > 0 && text.charAt(text.length() - 1) != '\n') text.append('\n');
            }
        }, true);
        return text.toString().strip();
    }
}
```

src/main/java/ems/gmail/SimpleJson.java

```
package ems.gmail;

import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.Strictness;
import com.google.gson.ToNumberPolicy;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Member 2 JSON helper API backed by the project's strict JSON parser.
 * Parses JSON into Java Maps, Lists, Strings, Numbers, and Booleans.
 */
public final class SimpleJson {
    private static final Gson JSON = new GsonBuilder().setStrictness(Strictness.STRICT)
            .setObjectToNumberStrategy(ToNumberPolicy.LONG_OR_DOUBLE).create();

    private SimpleJson() {
    }

    @SuppressWarnings("unchecked")
    public static Map<String, Object> parseObject(String json) {
        Object result = parse(json);
        if (result instanceof Map<?, ?> map) {
            return (Map<String, Object>) map;
        }
        return new LinkedHashMap<>();
    }

    public static Object parse(String json) {
        if (json == null || json.isBlank()) return null;
        return JSON.fromJson(json, Object.class);
    }

    public static String getString(Map<String, Object> map, String key) {
        if (map == null) return null;
        Object val = map.get(key);
        return val != null ? val.toString() : null;
    }
```

```
    public static String getStringOrDefault(Map<String, Object> map, String key, String defaultValue) {
        String val = getString(map, key);
        return val != null ? val : defaultValue;
    }

    @SuppressWarnings("unchecked")
    public static List<Object> getList(Map<String, Object> map, String key) {
        if (map == null) return List.of();
        Object val = map.get(key);
        if (val instanceof List<?> list) {
            return (List<Object>) list;
        }
        return List.of();
    }

    @SuppressWarnings("unchecked")
    public static Map<String, Object> getObject(Map<String, Object> map, String key) {
        if (map == null) return Map.of();
        Object val = map.get(key);
        if (val instanceof Map<?, ?> obj) {
            return (Map<String, Object>) obj;
        }
        return Map.of();
    }

    public static String escapeString(String raw) {
        if (raw == null) return "";
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < raw.length(); i++) {
            char c = raw.charAt(i);
            switch (c) {
                case '"' -> sb.append("\\\"");
                case '\\' -> sb.append("\\\\");
                case '\b' -> sb.append("\\b");
                case '\f' -> sb.append("\\f");
                case '\n' -> sb.append("\\n");
                case '\r' -> sb.append("\\r");
                case '\t' -> sb.append("\\t");
                default -> {
                    if (c < 0x20) {
                        sb.append(String.format("\\u%04x", (int) c));
                    } else {
                        sb.append(c);
                    }
                }
            }
        }
        return sb.toString();
    }

}
```

src/main/java/ems/gmail/TokenStore.java

```
package ems.gmail;

import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.LinkOption;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.nio.file.attribute.AclEntry;
import java.nio.file.attribute.AclEntryFlag;
import java.nio.file.attribute.AclEntryPermission;
import java.nio.file.attribute.AclEntryType;
import java.nio.file.attribute.AclFileAttributeView;
import java.nio.file.attribute.PosixFileAttributeView;
```

```
import java.nio.file.attribute.PosixFilePermissions;
import java.nio.file.attribute.UserPrincipal;
import java.util.EnumSet;
import java.util.List;

/** Owner-only OAuth cache. Tokens never belong in source control or the message database. */
final class TokenStore {
    private final Path directory;
    private final Path file;

    TokenStore(Path directory) {
        this.directory = directory.toAbsolutePath().normalize();
        this.file = this.directory.resolve("gmail-token.json");
    }

    JsonObject load() throws IOException {
        if (!Files.exists(file, LinkOption.NOFOLLOW_LINKS)) return null;
        rejectSymlink(file);
        secure(directory, true);
        secure(file, false);
        try {
            return JsonParser.parseString(Files.readString(file, StandardCharsets.UTF_8)).getAsJsonObject();
        } catch (RuntimeException corrupt) {
            clear();
            return null;
        }
    }

    void save(JsonObject data) throws IOException {
        Files.createDirectories(directory);
        rejectSymlink(directory);
        secure(directory, true);
        Path temp = Files.createTempFile(directory, "oauth-", ".tmp");
        try {
            secure(temp, false);
            Files.writeString(temp, data.toString(), StandardCharsets.UTF_8);
            try {
                Files.move(temp, file, StandardCopyOption.REPLACE_EXISTING, StandardCopyOption.ATOMIC_MOVE);
            } catch (java.nio.file.AtomicMoveNotSupportedException unsupported) {
                Files.move(temp, file, StandardCopyOption.REPLACE_EXISTING);
            }
            secure(file, false);
        } finally {
            Files.deleteIfExists(temp);
        }
    }

    void clear() throws IOException { Files.deleteIfExists(file); }

    private static void rejectSymlink(Path path) throws IOException {
        if (Files.isSymbolicLink(path)) throw new IOException("OAuth cache must not be a symbolic link.");
    }

    private static void secure(Path path, boolean directory) throws IOException {
        if (Files.getFileAttributeView(path, PosixFileAttributeView.class) != null) {
            Files.setPosixFilePermissions(path, PosixFilePermissions.fromString(directory ? "rwx------" : "rw---
----"));
            return;
        }
        AclFileAttributeView acl = Files.getFileAttributeView(path, AclFileAttributeView.class);
        if (acl != null) {
            UserPrincipal owner = path.getFileSystem().getUserPrincipalLookupService()
                    .lookupPrincipalByName(System.getProperty("user.name"));
            AclEntry.Builder entry = AclEntry.newBuilder().setType(AclEntryType.ALLOW)
                    .setPrincipal(owner).setPermissions(EnumSet.allOf(AclEntryPermission.class));
            if (directory) entry.setFlags(AclEntryFlag.DIRECTORY_INHERIT, AclEntryFlag.FILE_INHERIT);
            acl.setAcl(List.of(entry.build()));
            return;
        }
        throw new IOException("This filesystem cannot restrict access to OAuth credentials.");
```

```
    }
}
```

src/main/java/ems/model/Account.java

```
package ems.model;

import java.util.Objects;

/** A locally identified Gmail account. OAuth credentials are never part of this model. */
public final class Account {
    private final long accountId;
    private final String emailAddress;

    public Account(long accountId, String emailAddress) {
        this.accountId = accountId;
        this.emailAddress = Objects.requireNonNull(emailAddress, "emailAddress");
    }

    public long getAccountId() {
        return accountId;
    }

    public String getEmailAddress() {
        return emailAddress;
    }
}
```

src/main/java/ems/model/EmailMessage.java

```
package ems.model;

import java.time.Instant;
import java.util.List;
import java.util.Objects;
import java.util.Set;

/**
 * An immutable snapshot of a Gmail message. Gmail IDs are opaque strings.
 * Empty subjects and plain-text bodies are valid. Dates are instants; the UI
 * formats them in the user's local time zone.
 */
public final class EmailMessage {
    private final long accountId;
    private final String gmailMessageId;
    private final String senderEmail;
    private final List<String> recipientEmails;
    private final String subject;
    private final String bodyText;
    private final Instant receivedAt;
    private final Set<String> gmailLabelIds;

    public EmailMessage(
            long accountId,
            String gmailMessageId,
            String senderEmail,
            List<String> recipientEmails,
            String subject,
            String bodyText,
            Instant receivedAt,
            Set<String> gmailLabelIds) {
        this.accountId = accountId;
        this.gmailMessageId = Objects.requireNonNull(gmailMessageId, "gmailMessageId");
        this.senderEmail = Objects.requireNonNull(senderEmail, "senderEmail");
        this.recipientEmails = List.copyOf(recipientEmails);
        this.subject = Objects.requireNonNull(subject, "subject");
```

```
        this.bodyText = Objects.requireNonNull(bodyText, "bodyText");
        this.receivedAt = Objects.requireNonNull(receivedAt, "receivedAt");
        this.gmailLabelIds = Set.copyOf(gmailLabelIds);
    }

    public long getAccountId() {
        return accountId;
    }

    public String getGmailMessageId() {
        return gmailMessageId;
    }

    public String getSenderEmail() {
        return senderEmail;
    }

    public List<String> getRecipientEmails() {
        return recipientEmails;
    }

    public String getSubject() {
        return subject;
    }

    public String getBodyText() {
        return bodyText;
    }

    public Instant getReceivedAt() {
        return receivedAt;
    }

    public Set<String> getGmailLabelIds() {
        return gmailLabelIds;
    }
}
```

src/main/java/ems/model/Mailbox.java

```
package ems.model;

/** Mailbox views; SUSPECTED is the local detector's view rather than a Gmail label. */
public enum Mailbox {
    INBOX,
    SENT,
    SPAM,
    SUSPECTED
}
```

src/main/java/ems/model/SpamResult.java

```
package ems.model;

import java.util.List;

/**
 * The local detector's result. The score is a rule score, not a probability.
 * Gmail's own SPAM label remains separate from this classification.
 */
public final class SpamResult {
    private final int score;
    private final boolean suspectedSpam;
    private final List<String> reasons;
```

```
    public SpamResult(int score, boolean suspectedSpam, List<String> reasons) {
        this.score = score;
        this.suspectedSpam = suspectedSpam;
        this.reasons = List.copyOf(reasons);
    }

    public int getScore() {
        return score;
    }

    public boolean isSuspectedSpam() {
        return suspectedSpam;
    }

    public List<String> getReasons() {
        return reasons;
    }
}
```

src/main/java/ems/service/EmailService.java

```
package ems.service;

import ems.exception.AppException;
import ems.model.Account;
import ems.model.EmailMessage;
import ems.model.Mailbox;
import ems.model.SpamResult;
import java.util.List;

/**
 * Shared boundary between the Swing interface and Gmail/database integrations.
 * Implementations support one connected account at a time and scope every
 * mailbox operation to that account. Operations can block, so the UI calls them
 * on background workers rather than the Swing event-dispatch thread.
 *
 * <p>Implementations should turn recoverable provider/database errors into
 * {@link AppException} with useful, non-sensitive messages. A failed operation
 * must not be presented to the UI as a successful update.
 */
public interface EmailService {
    /** Authorizes Gmail access through Google and returns the connected account. */
    Account connectGmail() throws AppException;

    /** Ends the application's local connected session; does not delete the Gmail account. */
    void disconnect();

    /** Fetches Gmail changes, updates the local cache, and runs local spam analysis. */
    void refresh() throws AppException;
    default void refreshRecent() throws AppException { refresh(); }

    /**
     * Returns visible cached messages, newest first, for the selected mailbox.
     * An empty query returns all messages in that view; searches match subject,
     * body, sender, or recipient addresses. Returns an empty list if none match.
     */
    List<EmailMessage> getMessages(Mailbox mailbox, String query) throws AppException;

    /**
     * Opens an accessible message for the connected account and marks it read
     * in Gmail. Returns the updated snapshot after a successful operation.
     */
    EmailMessage openMessage(String gmailMessageId) throws AppException;

    /** Sends to one real recipient address and returns Gmail's new message ID. */
```

```
    String sendMessage(String recipientEmail, String subject, String body) throws AppException;

    /** Returns the local detector's result for a message in the connected account. */
    SpamResult getSpamResult(String gmailMessageId) throws AppException;

    /** Applies Gmail's spam classification and persists the user's correction locally. */
    void markAsSpam(String gmailMessageId) throws AppException;

    /** Removes Gmail's spam classification and persists the user's correction locally. */
    void markAsNotSpam(String gmailMessageId) throws AppException;

    /** Non-sensitive notice for a successful operation with a recoverable cache issue. */
    default String getLastWarning() { return ""; }
}
```

src/main/java/ems/service/GmailEmailService.java

```
package ems.service;

import ems.exception.AppException;
import ems.config.AppConfig;
import ems.gmail.GmailApiClient;
import ems.storage.JdbcMailStore;
import java.sql.DriverManager;
import ems.gmail.GmailGateway;
import ems.model.*;
import ems.storage.MailStore;
import jakarta.mail.internet.AddressException;
import jakarta.mail.internet.InternetAddress;
import java.sql.SQLException;
import java.time.Instant;
import java.util.*;

/** Connects the Swing contract to Gmail and the persistent account-scoped cache. */
public final class GmailEmailService implements EmailService {
    private final GmailGateway gmail;
    private final MailStore store;
    private Account account;
    private String lastWarning = "";

    /** Shared production wiring for both team launchers; constructors stay injectable for tests. */
    public static GmailEmailService createDefault() throws AppException {
        AppConfig config = AppConfig.load();
        return new GmailEmailService(new GmailApiClient(config.getGmailCredentialsPath(),
                config.getTokenDirectory()), new JdbcMailStore(() -> DriverManager.getConnection(
                config.getDatabaseUrl(), config.getDatabaseUser(), config.getDatabasePassword())));
    }

    public GmailEmailService(GmailGateway gmail, MailStore store) {
        this.gmail = Objects.requireNonNull(gmail, "gmail");
        this.store = Objects.requireNonNull(store, "store");
    }

    @Override public synchronized Account connectGmail() throws AppException {
        lastWarning = "";
        if (account != null) return account;
        try { store.initialize(); }
        catch (SQLException exception) {
            throw databaseError("Cannot connect to MySQL. Check db.url, db.user and db.password in
config/app.properties.", exception);
        }
        String email = gmail.connect();
        try {
            account = store.findOrCreateAccount(email);
            return account;
        } catch (SQLException exception) {
            gmail.disconnect();
```

```
            throw databaseError("Google sign-in succeeded, but the local account could not be saved. Check MySQL
and try again.", exception);
        }
    }

    @Override public synchronized void disconnect() {
        try { gmail.disconnect(); }
        finally { account = null; lastWarning = ""; }
    }

    @Override public synchronized void refresh() throws AppException {
        Account current = connected(); lastWarning = "";
        // A failed network fetch must never be mistaken for an empty mailbox.
        List<EmailMessage> snapshot = gmail.fetchRecentMessages(current.getAccountId(), 6)
                .stream().limit(18).toList();
        verifyAccount(current, snapshot);
        try { store.replaceSnapshot(current.getAccountId(), snapshot); }
        catch (SQLException exception) {
            throw databaseError("Gmail was read, but the local mailbox could not be updated. Check MySQL and
refresh again.", exception);
        }
    }

    @Override public synchronized List<EmailMessage> getMessages(Mailbox mailbox, String query) throws
AppException {
        Account current = connected();
        if (mailbox == null) throw new AppException("Choose a mailbox first.");
        try { return store.getMessages(current.getAccountId(), mailbox, query == null ? "" : query.strip()); }
        catch (SQLException exception) { throw databaseError("Cannot read cached messages. Check MySQL and try
again.", exception); }
    }

    @Override public synchronized void refreshRecent() throws AppException {
        refresh();
    }

    @Override public synchronized EmailMessage openMessage(String gmailMessageId) throws AppException {
        Account current = connected(); lastWarning = "";
        requireCached(current, gmailMessageId);
        EmailMessage message = gmail.getMessage(current.getAccountId(), gmailMessageId);
        verifyAccount(current, List.of(message));
        if (message.getGmailLabelIds().contains("UNREAD")) {
            gmail.setRead(gmailMessageId);
            Set<String> labels = new HashSet<>(message.getGmailLabelIds()); labels.remove("UNREAD");
            message = withLabels(message, labels);
        }
        try { store.saveMessages(current.getAccountId(), List.of(message)); }
        catch (SQLException exception) {
            throw databaseError("The message was opened in Gmail, but its local copy could not be updated.
Refresh after checking MySQL.", exception);
        }
        return message;
    }

    @Override public synchronized String sendMessage(String recipientEmail, String subject, String body) throws
AppException {
        Account current = connected(); lastWarning = "";
        String recipient = validateRecipient(recipientEmail);
        String cleanSubject = subject == null ? "" : subject.strip();
        if (cleanSubject.isBlank() || cleanSubject.length() > 200 || containsLineBreak(subject)) {
            throw new AppException("Enter a subject between 1 and 200 characters on one line.");
        }
        if (body == null || body.isBlank() || body.length() > 10_000) {
            throw new AppException("Enter a message between 1 and 10,000 characters.");
        }
        String id = gmail.sendMessage(current.getEmailAddress(), recipient, cleanSubject, body);
        // Gmail has accepted this email. Never turn a later cache error into 'send failed'.
        EmailMessage sent = new EmailMessage(current.getAccountId(), id, current.getEmailAddress(),
                List.of(recipient), cleanSubject, body, Instant.now(),
                recipient.equalsIgnoreCase(current.getEmailAddress()) ? Set.of("SENT", "INBOX") :
Set.of("SENT"));
```

```
        try { sent = gmail.getMessage(current.getAccountId(), id); verifyAccount(current, List.of(sent)); }
        catch (AppException exception) {
            lastWarning = "Email was sent. Its latest Gmail copy could not be loaded; refresh Sent before
sending it again.";
        }
        try { store.saveMessages(current.getAccountId(), List.of(sent)); }
        catch (SQLException exception) {
            lastWarning = "Email was sent, but its local copy could not be saved. Check Gmail Sent before
sending it again.";
        }
        return id;
    }

    @Override public synchronized SpamResult getSpamResult(String gmailMessageId) throws AppException {
        Account current = connected(); requireCached(current, gmailMessageId);
        try { return store.getSpamResult(current.getAccountId(), gmailMessageId); }
        catch (SQLException exception) { throw databaseError("Cannot read this message's spam review. Check
MySQL and try again.", exception); }
    }

    @Override public synchronized void markAsSpam(String gmailMessageId) throws AppException {
correct(gmailMessageId, true); }
    @Override public synchronized void markAsNotSpam(String gmailMessageId) throws AppException {
correct(gmailMessageId, false); }

    private void correct(String id, boolean spam) throws AppException {
        Account current = connected(); lastWarning = "";
        EmailMessage original = requireCached(current, id);
        if (original.getGmailLabelIds().contains("SENT") && !original.getGmailLabelIds().contains("INBOX")
                && !original.getGmailLabelIds().contains("SPAM")) {
            throw new AppException("Spam corrections apply to received messages.");
        }
        gmail.setSpam(id, spam);
        Set<String> labels = new HashSet<>(original.getGmailLabelIds());
        if (spam) { labels.add("SPAM"); labels.remove("INBOX"); }
        else { labels.remove("SPAM"); labels.add("INBOX"); }
        EmailMessage updated = withLabels(original, labels);
        try { updated = gmail.getMessage(current.getAccountId(), id); verifyAccount(current, List.of(updated));
}
        catch (AppException exception) {
            lastWarning = "Gmail was updated. Refresh to load its latest message details.";
        }
        try {
            store.saveMessages(current.getAccountId(), List.of(updated));
            store.saveFeedback(current.getAccountId(), id, spam);
        } catch (SQLException exception) {
            throw databaseError("Gmail's spam label was updated, but the local correction could not be saved.
Check MySQL and refresh.", exception);
        }
    }

    @Override public synchronized String getLastWarning() { return lastWarning; }

    private Account connected() throws AppException {
        if (account == null) throw new AppException("Connect your Gmail account first.");
        return account;
    }

    private EmailMessage requireCached(Account current, String id) throws AppException {
        if (id == null || id.isBlank()) throw new AppException("Choose an available message first.");
        try {
            return store.findMessage(current.getAccountId(), id)
                    .orElseThrow(() -> new AppException("This message is unavailable in the connected account.
Refresh your mailbox."));
        } catch (SQLException exception) { throw databaseError("Cannot access this message. Check MySQL and try
again.", exception); }
    }

    private static void verifyAccount(Account current, List<EmailMessage> messages) throws AppException {
        if (messages == null || messages.stream().anyMatch(message -> message == null
```

```
                || message.getAccountId() != current.getAccountId())) {
            throw new AppException("The mailbox response did not match the connected account. Disconnect and
sign in again.");
        }
    }

    private static String validateRecipient(String input) throws AppException {
        if (input == null || input.isBlank() || input.length() > 254 || containsLineBreak(input)) {
            throw new AppException("Enter one valid recipient email address.");
        }
        try {
            InternetAddress[] addresses = InternetAddress.parse(input.strip(), true);
            if (addresses.length != 1 || addresses[0].isGroup()) throw new AddressException();
            addresses[0].validate();
            String address = addresses[0].getAddress();
            if (!address.contains("@")) throw new AddressException();
            return address;
        } catch (AddressException exception) { throw new AppException("Enter one valid recipient email
address."); }
    }

    private static boolean containsLineBreak(String value) { return value != null && (value.indexOf('\r') >= 0
|| value.indexOf('\n') >= 0); }
    private static EmailMessage withLabels(EmailMessage message, Set<String> labels) {
        return new EmailMessage(message.getAccountId(), message.getGmailMessageId(), message.getSenderEmail(),
                message.getRecipientEmails(), message.getSubject(), message.getBodyText(),
message.getReceivedAt(), labels);
    }
    private static AppException databaseError(String message, SQLException cause) { return new
AppException(message, cause); }
}
```

src/main/java/ems/storage/ConnectionFactory.java

```
package ems.storage;


import java.sql.Connection;
import java.sql.SQLException;


/** Opens one owned connection per store operation. */
@FunctionalInterface
public interface ConnectionFactory {
    Connection open() throws SQLException;
}
```

src/main/java/ems/storage/JdbcMailStore.java

```
package ems.storage;


import com.emailapp.database.DatabaseConnection;
import com.emailapp.spam.SpamClassifier;
import ems.model.Account;
import ems.model.EmailMessage;
import ems.model.Mailbox;
import ems.model.SpamResult;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Timestamp;
import java.time.Instant;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Collection;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
```

```
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TimeZone;

/** JDBC cache with atomic snapshot replacement and persistent account-scoped feedback. */
public final class JdbcMailStore implements MailStore {
    private final ConnectionFactory connections;
    private final SpamClassifier classifier = new SpamClassifier();

    public JdbcMailStore() {
        this(DatabaseConnection::getConnection);
    }

    public JdbcMailStore(ConnectionFactory connections) {
        this.connections = java.util.Objects.requireNonNull(connections);
    }

    @Override
    public void initialize() throws SQLException {
        try (Connection connection = connections.open(); Statement statement = connection.createStatement()) {
            for (String sql : Schema.TABLES) statement.execute(sql);
        }
    }

    @Override
    public Account findOrCreateAccount(String email) throws SQLException {
        String normalized = email.trim().toLowerCase(Locale.ROOT);
        if (normalized.isEmpty()) throw new SQLException("The Gmail account has no email address.");
        try (Connection connection = connections.open()) {
            Optional<Account> existing = findAccount(connection, normalized);
            if (existing.isPresent()) return existing.get();
            try (PreparedStatement insert = connection.prepareStatement(
                    "INSERT INTO accounts (email_address) VALUES (?)", Statement.RETURN_GENERATED_KEYS)) {
                insert.setString(1, normalized);
                insert.executeUpdate();
                try (ResultSet keys = insert.getGeneratedKeys()) {
                    if (keys.next()) return new Account(keys.getLong(1), normalized);
                }
            } catch (SQLException exception) {
                // Another connection may have created this account since the SELECT.
                Optional<Account> concurrent = findAccount(connection, normalized);
                if (concurrent.isPresent()) return concurrent.get();
                throw exception;
            }
            return findAccount(connection, normalized).orElseThrow(() -> new SQLException("Account ID was not
returned."));
        }
    }

    private Optional<Account> findAccount(Connection connection, String email) throws SQLException {
        try (PreparedStatement query = connection.prepareStatement(
                "SELECT account_id, email_address FROM accounts WHERE email_address = ?")) {
            query.setString(1, email);
            try (ResultSet rows = query.executeQuery()) {
                return rows.next() ? Optional.of(new Account(rows.getLong(1), rows.getString(2))) :
Optional.empty();
            }
        }
    }

    @Override
    public void saveMessages(long accountId, List<EmailMessage> messages) throws SQLException {
        checkAccount(accountId, messages);
        transaction(connection -> saveBatch(connection, accountId, messages));
    }

    @Override
```

```
    public void replaceSnapshot(long accountId, List<EmailMessage> messages) throws SQLException {
        checkAccount(accountId, messages);
        transaction(connection -> {
            saveBatch(connection, accountId, messages);
            Set<String> present = new LinkedHashSet<>();
            for (EmailMessage message : messages) present.add(message.getGmailMessageId());
            List<String> removed = new ArrayList<>();
            try (PreparedStatement query = connection.prepareStatement("SELECT gmail_message_id FROM messages
WHERE account_id=?")) {
                query.setLong(1, accountId);
                try (ResultSet rows = query.executeQuery()) {
                    while (rows.next()) if (!present.contains(rows.getString(1)))
removed.add(rows.getString(1));
                }
            }
            deleteBatch(connection, accountId, removed);
        });
    }

    private static void checkAccount(long accountId, List<EmailMessage> messages) throws SQLException {
        for (EmailMessage message : messages) {
            if (message.getAccountId() != accountId) throw new SQLException("Message belongs to another
account.");
        }
    }

    private void saveBatch(Connection connection, long accountId, List<EmailMessage> messages) throws
SQLException {
        Set<String> blocked = blockedSenders(connection, accountId);
        for (EmailMessage message : messages) {
            upsertMessage(connection, message);
            replaceChildren(connection, message, "message_recipients", "recipient_email",
                    distinctRecipients(message.getRecipientEmails()));
            replaceChildren(connection, message, "message_labels", "gmail_label_id",
message.getGmailLabelIds());
            SpamResult automatic = classifier.analyze(message,
                    blocked.contains(message.getSenderEmail().trim().toLowerCase(Locale.ROOT)));
            saveSpamResult(connection, accountId, message.getGmailMessageId(), automatic);
        }
    }

    private static Collection<String> distinctRecipients(List<String> recipients) {
        // Gmail may repeat a recipient in To/Cc with different casing. MySQL's default
        // collation also treats those addresses as equal; retain the first spelling.
        Map<String, String> distinct = new LinkedHashMap<>();
        for (String recipient : recipients) {
            distinct.putIfAbsent(recipient.toLowerCase(Locale.ROOT), recipient);
        }
        return distinct.values();
    }

    private void upsertMessage(Connection connection, EmailMessage message) throws SQLException {
        String fields = "sender_email=?, subject=?, body_text=?, received_at=?, is_read=?";
        try (PreparedStatement update = connection.prepareStatement(
                "UPDATE messages SET " + fields + " WHERE account_id=? AND gmail_message_id=?")) {
            bindMessage(update, message);
            if (update.executeUpdate() > 0) return;
        }
        // MySQL may return zero when an UPDATE has identical values (useAffectedRows=true).
        if (messageExists(connection, message.getAccountId(), message.getGmailMessageId())) return;
        try (PreparedStatement insert = connection.prepareStatement("INSERT INTO messages "
                + "(sender_email,subject,body_text,received_at,is_read,account_id,gmail_message_id) VALUES
(?,?,?,?,?,?,?)")) {
            bindMessage(insert, message);
            insert.executeUpdate();
        }
    }

    private static void bindMessage(PreparedStatement statement, EmailMessage message) throws SQLException {
        statement.setString(1, message.getSenderEmail());
        statement.setString(2, message.getSubject());
```

```
        statement.setString(3, message.getBodyText());
        statement.setTimestamp(4, Timestamp.from(message.getReceivedAt()), utc());
        statement.setBoolean(5, !message.getGmailLabelIds().contains("UNREAD"));
        statement.setLong(6, message.getAccountId());
        statement.setString(7, message.getGmailMessageId());
    }

    private void replaceChildren(Connection connection, EmailMessage message, String table, String column,
            Collection<String> values) throws SQLException {
        deleteOne(connection, table, message.getAccountId(), message.getGmailMessageId());
        if (values.isEmpty()) return;
        try (PreparedStatement insert = connection.prepareStatement("INSERT INTO " + table
                + " (account_id,gmail_message_id," + column + ") VALUES (?,?,?)")) {
            for (String value : values) {
                insert.setLong(1, message.getAccountId());
                insert.setString(2, message.getGmailMessageId());
                insert.setString(3, value);
                insert.addBatch();
            }
            insert.executeBatch();
        }
    }

    @Override
    public List<EmailMessage> getMessages(long accountId, Mailbox mailbox, String query) throws SQLException {
        String base = "SELECT m.account_id,m.gmail_message_id,m.sender_email,m.subject,m.body_text,m.received_at
"
                + "FROM messages m WHERE m.account_id=? AND NOT EXISTS (SELECT 1 FROM message_labels t "
                + "WHERE t.account_id=m.account_id AND t.gmail_message_id=m.gmail_message_id AND
t.gmail_label_id='TRASH') ";
        if (mailbox == Mailbox.SUSPECTED) {
            // Review only incoming mail that Gmail has not already placed in Spam.
            base += "AND NOT EXISTS (SELECT 1 FROM message_labels excluded WHERE
excluded.account_id=m.account_id "
                    + "AND excluded.gmail_message_id=m.gmail_message_id AND excluded.gmail_label_id IN
('SPAM','SENT')) "
                    + "AND ((EXISTS (SELECT 1 FROM spam_feedback f WHERE f.account_id=m.account_id "
                    + "AND f.gmail_message_id=m.gmail_message_id AND f.user_marked_spam=TRUE)) OR "
                    + "(NOT EXISTS (SELECT 1 FROM spam_feedback f WHERE f.account_id=m.account_id "
                    + "AND f.gmail_message_id=m.gmail_message_id) AND EXISTS (SELECT 1 FROM spam_results s "
                    + "WHERE s.account_id=m.account_id AND s.gmail_message_id=m.gmail_message_id AND
s.suspected_spam=TRUE))) ";
        } else {
            base += "AND EXISTS (SELECT 1 FROM message_labels l WHERE l.account_id=m.account_id "
                    + "AND l.gmail_message_id=m.gmail_message_id AND l.gmail_label_id=?) ";
            if (mailbox == Mailbox.INBOX) base += "AND NOT EXISTS (SELECT 1 FROM message_labels s "
                    + "WHERE s.account_id=m.account_id AND s.gmail_message_id=m.gmail_message_id AND
s.gmail_label_id='SPAM') ";
        }
        String normalized = query == null ? "" : query.trim().toLowerCase(Locale.ROOT);
        if (!normalized.isEmpty()) base += "AND (LOWER(m.sender_email) LIKE ? ESCAPE '!' "
                + "OR LOWER(m.subject) LIKE ? ESCAPE '!' OR LOWER(m.body_text) LIKE ? ESCAPE '!' "
                + "OR EXISTS (SELECT 1 FROM message_recipients r WHERE r.account_id=m.account_id "
                + "AND r.gmail_message_id=m.gmail_message_id AND LOWER(r.recipient_email) LIKE ? ESCAPE '!')) ";
        base += "ORDER BY m.received_at DESC,m.gmail_message_id DESC";
        try (Connection connection = connections.open(); PreparedStatement statement =
connection.prepareStatement(base)) {
            int index = 1;
            statement.setLong(index++, accountId);
            if (mailbox != Mailbox.SUSPECTED) statement.setString(index++, mailbox.name());
            if (!normalized.isEmpty()) {
                String pattern = "%" + normalized.replace("!", "!!").replace("%", "!%").replace("_", "!_") +
"%";
                for (int i = 0; i < 4; i++) statement.setString(index++, pattern);
            }
            return readMessages(connection, statement);
        }
    }

    @Override
    public Optional<EmailMessage> findMessage(long accountId, String gmailMessageId) throws SQLException {
```

```
        try (Connection connection = connections.open(); PreparedStatement statement =
connection.prepareStatement(
                "SELECT account_id,gmail_message_id,sender_email,subject,body_text,received_at FROM messages "
                        + "WHERE account_id=? AND gmail_message_id=?")) {
            statement.setLong(1, accountId);
            statement.setString(2, gmailMessageId);
            return readMessages(connection, statement).stream().findFirst();
        }
    }

    /** Legacy repository support, with bounded bulk child reads rather than N+1 connections. */
    public List<EmailMessage> findAll(long accountId) throws SQLException {
        try (Connection connection = connections.open(); PreparedStatement statement =
connection.prepareStatement(
                "SELECT account_id,gmail_message_id,sender_email,subject,body_text,received_at FROM messages "
                        + "WHERE account_id=? ORDER BY received_at DESC,gmail_message_id DESC")) {
            statement.setLong(1, accountId);
            return readMessages(connection, statement);
        }
    }

    private List<EmailMessage> readMessages(Connection connection, PreparedStatement query) throws SQLException
{
        List<EmailMessage> bases = new ArrayList<>();
        try (ResultSet rows = query.executeQuery()) {
            while (rows.next()) bases.add(new EmailMessage(rows.getLong("account_id"),
rows.getString("gmail_message_id"),
                    rows.getString("sender_email"), List.of(), rows.getString("subject"),
rows.getString("body_text"),
                    rows.getTimestamp("received_at", utc()).toInstant(), Set.of()));
        }
        if (bases.isEmpty()) return List.of();
        Map<String, List<String>> recipients = readChildren(connection, bases, "message_recipients",
"recipient_email");
        Map<String, List<String>> labels = readChildren(connection, bases, "message_labels", "gmail_label_id");
        List<EmailMessage> result = new ArrayList<>(bases.size());
        for (EmailMessage base : bases) result.add(new EmailMessage(base.getAccountId(),
base.getGmailMessageId(),
                base.getSenderEmail(), recipients.getOrDefault(base.getGmailMessageId(), List.of()),
                base.getSubject(), base.getBodyText(), base.getReceivedAt(),
                new LinkedHashSet<>(labels.getOrDefault(base.getGmailMessageId(), List.of()))));
        return result;
    }

    private Map<String, List<String>> readChildren(Connection connection, List<EmailMessage> messages,
            String table, String column) throws SQLException {
        Map<String, List<String>> values = new LinkedHashMap<>();
        for (int start = 0; start < messages.size(); start += 500) {
            List<EmailMessage> batch = messages.subList(start, Math.min(start + 500, messages.size()));
            String placeholders = String.join(",", Collections.nCopies(batch.size(), "?"));
            try (PreparedStatement query = connection.prepareStatement("SELECT gmail_message_id," + column
                    + " FROM " + table + " WHERE account_id=? AND gmail_message_id IN (" + placeholders + ")
ORDER BY " + column)) {
                query.setLong(1, messages.get(0).getAccountId());
                for (int i = 0; i < batch.size(); i++) query.setString(i + 2, batch.get(i).getGmailMessageId());
                try (ResultSet rows = query.executeQuery()) {
                    while (rows.next()) values.computeIfAbsent(rows.getString(1), unused -> new
ArrayList<>()).add(rows.getString(2));
                }
            }
        }
        return values;
    }

    @Override
    public SpamResult getSpamResult(long accountId, String gmailMessageId) throws SQLException {
        try (Connection connection = connections.open()) {
            Optional<Boolean> feedback = feedback(connection, accountId, gmailMessageId);
            if (feedback.isPresent()) return corrected(feedback.get());
            try (PreparedStatement query = connection.prepareStatement(
                    "SELECT score,suspected_spam,reasons FROM spam_results WHERE account_id=? AND
gmail_message_id=?")) {
```

```
                query.setLong(1, accountId);
                query.setString(2, gmailMessageId);
                try (ResultSet rows = query.executeQuery()) {
                    if (rows.next()) return new SpamResult(rows.getInt(1), rows.getBoolean(2),
ReasonJson.decode(rows.getString(3)));
                }
            }
            return new SpamResult(0, false, List.of("This message has not been analyzed yet."));
        }
    }

    @Override
    public void saveFeedback(long accountId, String gmailMessageId, boolean spam) throws SQLException {
        transaction(connection -> {
            if (!messageExists(connection, accountId, gmailMessageId)) throw new SQLException("Message was not
found in this account.");
            try (PreparedStatement update = connection.prepareStatement("UPDATE spam_feedback SET
user_marked_spam=?,corrected_at=? "
                    + "WHERE account_id=? AND gmail_message_id=?")) {
                bindFeedback(update, accountId, gmailMessageId, spam);
                if (update.executeUpdate() > 0) return;
            }
            if (feedback(connection, accountId, gmailMessageId).isPresent()) return;
            try (PreparedStatement insert = connection.prepareStatement("INSERT INTO spam_feedback "
                    + "(user_marked_spam,corrected_at,account_id,gmail_message_id) VALUES (?,?,?,?)")) {
                bindFeedback(insert, accountId, gmailMessageId, spam);
                insert.executeUpdate();
            }
        });
    }

    private static void bindFeedback(PreparedStatement statement, long accountId, String id, boolean spam)
throws SQLException {
        statement.setBoolean(1, spam);
        statement.setTimestamp(2, Timestamp.from(Instant.now()), utc());
        statement.setLong(3, accountId);
        statement.setString(4, id);
    }

    private static Optional<Boolean> feedback(Connection connection, long accountId, String id) throws
SQLException {
        try (PreparedStatement query = connection.prepareStatement(
                "SELECT user_marked_spam FROM spam_feedback WHERE account_id=? AND gmail_message_id=?")) {
            query.setLong(1, accountId);
            query.setString(2, id);
            try (ResultSet rows = query.executeQuery()) {
                return rows.next() ? Optional.of(rows.getBoolean(1)) : Optional.empty();
            }
        }
    }

    private static SpamResult corrected(boolean spam) {
        return new SpamResult(spam ? 100 : 0, spam, List.of(spam ? "You marked this message as spam." : "You
marked this message as not spam."));
    }

    /** Compatibility entry point for Member 3's repository; automatic scores never overwrite feedback. */
    public void saveSpamResult(long accountId, String id, SpamResult result) throws SQLException {
        transaction(connection -> saveSpamResult(connection, accountId, id, result));
    }

    private void saveSpamResult(Connection connection, long accountId, String id, SpamResult result) throws
SQLException {
        String reasons = ReasonJson.encode(result.getReasons());
        try (PreparedStatement update = connection.prepareStatement("UPDATE spam_results SET
score=?,suspected_spam=?,reasons=?,analyzed_at=? "
                + "WHERE account_id=? AND gmail_message_id=?")) {
            bindSpam(update, accountId, id, result, reasons);
            if (update.executeUpdate() > 0) return;
        }
```

```
        try (PreparedStatement exists = connection.prepareStatement("SELECT 1 FROM spam_results WHERE
account_id=? AND gmail_message_id=?")) {
            exists.setLong(1, accountId);
            exists.setString(2, id);
            try (ResultSet rows = exists.executeQuery()) { if (rows.next()) return; }
        }
        try (PreparedStatement insert = connection.prepareStatement("INSERT INTO spam_results "
                + "(score,suspected_spam,reasons,analyzed_at,account_id,gmail_message_id) VALUES
(?,?,?,?,?,?)")) {
            bindSpam(insert, accountId, id, result, reasons);
            insert.executeUpdate();
        }
    }

    private static void bindSpam(PreparedStatement statement, long accountId, String id, SpamResult result,
String reasons) throws SQLException {
        statement.setInt(1, result.getScore());
        statement.setBoolean(2, result.isSuspectedSpam());
        statement.setString(3, reasons);
        statement.setTimestamp(4, Timestamp.from(Instant.now()), utc());
        statement.setLong(5, accountId);
        statement.setString(6, id);
    }

    @Override
    public void deleteMessages(long accountId, Collection<String> gmailMessageIds) throws SQLException {
        transaction(connection -> deleteBatch(connection, accountId, gmailMessageIds));
    }

    private void deleteBatch(Connection connection, long accountId, Collection<String> gmailMessageIds) throws
SQLException {
        for (String id : gmailMessageIds) {
            for (String table : List.of("message_recipients", "message_labels", "spam_results", "messages"))
                deleteOne(connection, table, accountId, id);
            // Feedback deliberately survives cache eviction and later re-downloads.
        }
    }

    public boolean isSenderBlocked(long accountId, String email) throws SQLException {
        try (Connection connection = connections.open()) {
            return blockedSenders(connection, accountId).contains(email.trim().toLowerCase(Locale.ROOT));
        }
    }

    public void setSenderBlocked(long accountId, String email, boolean blocked) throws SQLException {
        String normalized = email.trim().toLowerCase(Locale.ROOT);
        if (normalized.isEmpty()) throw new SQLException("A sender email is required.");
        transaction(connection -> {
            try (PreparedStatement delete = connection.prepareStatement(
                    "DELETE FROM blocked_senders WHERE account_id=? AND sender_email=?")) {
                delete.setLong(1, accountId);
                delete.setString(2, normalized);
                delete.executeUpdate();
            }
            if (blocked) try (PreparedStatement insert = connection.prepareStatement(
                    "INSERT INTO blocked_senders (account_id,sender_email) VALUES (?,?)")) {
                insert.setLong(1, accountId);
                insert.setString(2, normalized);
                insert.executeUpdate();
            }
        });
    }

    private Set<String> blockedSenders(Connection connection, long accountId) throws SQLException {
        Set<String> senders = new LinkedHashSet<>();
        try (PreparedStatement query = connection.prepareStatement("SELECT sender_email FROM blocked_senders
WHERE account_id=?")) {
            query.setLong(1, accountId);
            try (ResultSet rows = query.executeQuery()) {
                while (rows.next()) senders.add(rows.getString(1).trim().toLowerCase(Locale.ROOT));
            }
```

```
        }
        return senders;
    }

    public void updateReadStatus(long accountId, String id, boolean read) throws SQLException {
        transaction(connection -> {
            try (PreparedStatement update = connection.prepareStatement("UPDATE messages SET is_read=? WHERE
account_id=? AND gmail_message_id=?")) {
                update.setBoolean(1, read);
                update.setLong(2, accountId);
                update.setString(3, id);
                if (update.executeUpdate() == 0 && !messageExists(connection, accountId, id)) return;
            }
            try (PreparedStatement delete = connection.prepareStatement("DELETE FROM message_labels WHERE
account_id=? AND gmail_message_id=? AND gmail_label_id='UNREAD'")) {
                delete.setLong(1, accountId);
                delete.setString(2, id);
                delete.executeUpdate();
            }
            if (!read) try (PreparedStatement insert = connection.prepareStatement("INSERT INTO message_labels
(account_id,gmail_message_id,gmail_label_id) VALUES (?,?,'UNREAD')")) {
                insert.setLong(1, accountId);
                insert.setString(2, id);
                insert.executeUpdate();
            }
        });
    }

    private static boolean messageExists(Connection connection, long accountId, String id) throws SQLException {
        try (PreparedStatement query = connection.prepareStatement("SELECT 1 FROM messages WHERE account_id=?
AND gmail_message_id=?")) {
            query.setLong(1, accountId);
            query.setString(2, id);
            try (ResultSet rows = query.executeQuery()) { return rows.next(); }
        }
    }

    private static void deleteOne(Connection connection, String table, long accountId, String id) throws
SQLException {
        try (PreparedStatement delete = connection.prepareStatement("DELETE FROM " + table + " WHERE
account_id=? AND gmail_message_id=?")) {
            delete.setLong(1, accountId);
            delete.setString(2, id);
            delete.executeUpdate();
        }
    }

    @FunctionalInterface private interface Transaction { void run(Connection connection) throws SQLException; }

    private void transaction(Transaction action) throws SQLException {
        try (Connection connection = connections.open()) {
            connection.setAutoCommit(false);
            try {
                action.run(connection);
                connection.commit();
            } catch (SQLException | RuntimeException exception) {
                try { connection.rollback(); } catch (SQLException rollback) {
exception.addSuppressed(rollback); }
                throw exception;
            }
        }
    }

    private static Calendar utc() { return Calendar.getInstance(TimeZone.getTimeZone("UTC")); }
}
```

src/main/java/ems/storage/MailStore.java

```
package ems.storage;
```

```
import ems.model.Account;
import ems.model.EmailMessage;
import ems.model.Mailbox;
import ems.model.SpamResult;
import java.sql.SQLException;
import java.util.Collection;
import java.util.List;
import java.util.Optional;

/** Account-scoped local mailbox cache. Gmail is the source of mailbox truth. */
public interface MailStore {
    void initialize() throws SQLException;
    Account findOrCreateAccount(String email) throws SQLException;
    void saveMessages(long accountId, List<EmailMessage> messages) throws SQLException;
    void replaceSnapshot(long accountId, List<EmailMessage> messages) throws SQLException;
    List<EmailMessage> getMessages(long accountId, Mailbox mailbox, String query) throws SQLException;
    Optional<EmailMessage> findMessage(long accountId, String gmailMessageId) throws SQLException;
    SpamResult getSpamResult(long accountId, String gmailMessageId) throws SQLException;
    void saveFeedback(long accountId, String gmailMessageId, boolean spam) throws SQLException;
    void deleteMessages(long accountId, Collection<String> gmailMessageIds) throws SQLException;
}
```

src/main/java/ems/storage/ReasonJson.java

```
package ems.storage;

import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

/** Minimal JSON string-array codec; avoids storing Java serialization in the database. */
final class ReasonJson {
    private ReasonJson() { }

    static String encode(List<String> values) {
        StringBuilder result = new StringBuilder("[");
        for (String value : values) {
            if (result.length() > 1) result.append(',');
            result.append('"');
            for (int i = 0; i < value.length(); i++) {
                char c = value.charAt(i);
                if (c == '"' || c == '\\') result.append('\\').append(c);
                else if (c < 32) result.append(String.format(java.util.Locale.ROOT, "\\u%04x", (int) c));
                else result.append(c);
            }
            result.append('"');
        }
        return result.append(']').toString();
    }

    static List<String> decode(String json) throws SQLException {
        List<String> values = new ArrayList<>();
        if (json == null || json.isBlank()) return values;
        String text = json.trim();
        if (!text.startsWith("[") || !text.endsWith("]")) throw new SQLException("Invalid stored spam
reasons.");
        int i = 1;
        while (i < text.length() - 1) {
            if (Character.isWhitespace(text.charAt(i)) || text.charAt(i) == ',') { i++; continue; }
            if (text.charAt(i++) != '"') throw new SQLException("Invalid stored spam reason.");
            StringBuilder value = new StringBuilder();
            boolean closed = false;
            while (i < text.length() - 1) {
                char c = text.charAt(i++);
                if (c == '"') { closed = true; break; }
                if (c != '\\') { value.append(c); continue; }
                if (i >= text.length() - 1) throw new SQLException("Invalid stored spam reason escape.");
                char escaped = text.charAt(i++);
```

```
                switch (escaped) {
                    case '"', '\\', '/' -> value.append(escaped);
                    case 'n' -> value.append('\n');
                    case 'r' -> value.append('\r');
                    case 't' -> value.append('\t');
                    case 'b' -> value.append('\b');
                    case 'f' -> value.append('\f');
                    case 'u' -> {
                        if (i + 4 > text.length() - 1) throw new SQLException("Invalid unicode escape in spam
reason.");
                        try { value.append((char) Integer.parseInt(text.substring(i, i + 4), 16)); }
                        catch (NumberFormatException bad) { throw new SQLException("Invalid unicode escape in
spam reason.", bad); }
                        i += 4;
                    }
                    default -> throw new SQLException("Invalid stored spam reason escape.");
                }
            }
            if (!closed) throw new SQLException("Unclosed stored spam reason.");
            values.add(value.toString());
        }
        return values;
    }
}
```

src/main/java/ems/storage/Schema.java

```
package ems.storage;

/** Keep table definitions aligned with sql/schema.sql. Database creation is an installation step. */
final class Schema {
    private Schema() { }
    static final String[] TABLES = {
        """
        CREATE TABLE IF NOT EXISTS accounts (
            account_id BIGINT AUTO_INCREMENT PRIMARY KEY,
            email_address VARCHAR(254) NOT NULL UNIQUE
        )
        """,
        """
        CREATE TABLE IF NOT EXISTS messages (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            sender_email VARCHAR(320) NOT NULL,
            subject TEXT NOT NULL,
            body_text LONGTEXT NOT NULL,
            received_at DATETIME(6) NOT NULL,
            is_read BOOLEAN NOT NULL DEFAULT TRUE,
            PRIMARY KEY (account_id, gmail_message_id),
            FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        )
        """,
        """
        CREATE TABLE IF NOT EXISTS message_recipients (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            recipient_email VARCHAR(320) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id, recipient_email),
            FOREIGN KEY (account_id, gmail_message_id) REFERENCES messages(account_id, gmail_message_id) ON
DELETE CASCADE
        )
        """,
        """
        CREATE TABLE IF NOT EXISTS message_labels (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            gmail_label_id VARCHAR(128) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id, gmail_label_id),
            FOREIGN KEY (account_id, gmail_message_id) REFERENCES messages(account_id, gmail_message_id) ON
DELETE CASCADE
        )
        """,
```

```
        """
        CREATE TABLE IF NOT EXISTS spam_results (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            score INT NOT NULL,
            suspected_spam BOOLEAN NOT NULL,
            reasons TEXT NOT NULL,
            analyzed_at DATETIME(6) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id),
            FOREIGN KEY (account_id, gmail_message_id) REFERENCES messages(account_id, gmail_message_id) ON
DELETE CASCADE
        )
        """,
        """
        CREATE TABLE IF NOT EXISTS spam_feedback (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            user_marked_spam BOOLEAN NOT NULL,
            corrected_at DATETIME(6) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id),
            FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        )
        """,
        """
        CREATE TABLE IF NOT EXISTS blocked_senders (
            account_id BIGINT NOT NULL,
            sender_email VARCHAR(320) NOT NULL,
            PRIMARY KEY (account_id, sender_email),
            FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        )
        """
    };
}
```

src/main/java/ems/ui/ComposePanel.java

```
package ems.ui;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.KeyEvent;
import java.util.Objects;
import java.util.function.Consumer;
import java.util.regex.Pattern;
import javax.swing.AbstractAction;
import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JComponent;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.KeyStroke;

/**
 * Reusable compose form. Validation stays local; the owner performs sending
 * through its service on a background worker and reports failures with showError.
 */
public final class ComposePanel extends JPanel {
    private static final long serialVersionUID = 1L;
    private static final Color ERROR = new Color(162, 51, 43);
    private static final Pattern ADDRESS = Pattern.compile(
            "^[^\\s@,;<>]+@[^\\s@,;<>.]+(?:\\.[^\\s@,;<>.]+)+$");
```

```
/** Validated input, ready for the owner to submit through EmailService. */
public static record Draft(String recipientEmail, String subject, String body) { }

private final JTextField toField;
private final JTextField subjectField;
private final JTextArea bodyArea;
private final JButton sendButton;
private final JButton cancelButton;
private final JLabel errorLabel;
private final Consumer<Draft> onSend;
private final Runnable onCancel;
private boolean busy;

public ComposePanel(
        String senderEmail,
        boolean demoMode,
        Consumer<Draft> onSend,
        Runnable onCancel) {
    this.onSend = Objects.requireNonNull(onSend, "onSend");
    this.onCancel = Objects.requireNonNull(onCancel, "onCancel");
    Objects.requireNonNull(senderEmail, "senderEmail");

    setName("composePanel");
    setLayout(new BorderLayout(0, 18));
    setBackground(UiTheme.SURFACE);
    setBorder(BorderFactory.createEmptyBorder(24, 28, 24, 28));
    setPreferredSize(new Dimension(660, 560));

    JPanel heading = column();
    heading.add(UiTheme.label("New message", Font.BOLD, 24, UiTheme.TEXT));
    heading.add(Box.createVerticalStrut(6));
    JLabel from = UiTheme.label("From: " + senderEmail, Font.PLAIN, 13, UiTheme.MUTED);
    from.putClientProperty("html.disable", Boolean.TRUE);
    heading.add(from);
    if (demoMode) {
        heading.add(Box.createVerticalStrut(12));
        JLabel notice = UiTheme.label(
                "DEMO MODE  ·  Messages stay in the demo. Nothing is sent through Gmail.",
                Font.PLAIN, 12, UiTheme.ACCENT);
        notice.setBorder(BorderFactory.createEmptyBorder(10, 12, 10, 12));
        notice.setOpaque(true);
        notice.setBackground(UiTheme.BACKGROUND);
        notice.setAlignmentX(LEFT_ALIGNMENT);
        heading.add(notice);
    }
    add(heading, BorderLayout.NORTH);

    JPanel fields = new JPanel(new BorderLayout(0, 14));
    fields.setOpaque(false);
    JPanel shortFields = column();
    toField = UiTheme.field("recipient@example.com");
    toField.setName("composeTo");
    toField.getAccessibleContext().setAccessibleName("Recipient email address");
    toField.getAccessibleContext().setAccessibleDescription("Enter one recipient email address.");
    addField(shortFields, "To", toField, KeyEvent.VK_T);
    shortFields.add(Box.createVerticalStrut(12));

    subjectField = UiTheme.field("Add a subject");
    subjectField.setName("composeSubject");
    subjectField.getAccessibleContext().setAccessibleName("Subject");
    subjectField.getAccessibleContext().setAccessibleDescription("Required, up to 200 characters.");
    addField(shortFields, "Subject", subjectField, KeyEvent.VK_S);
    fields.add(shortFields, BorderLayout.NORTH);

    JPanel bodyPanel = new JPanel(new BorderLayout(0, 7));
    bodyPanel.setOpaque(false);
    bodyArea = new JTextArea();
    bodyArea.setName("composeBody");
    bodyArea.setFont(UiTheme.font(Font.PLAIN, 14));
    bodyArea.setForeground(UiTheme.TEXT);
```

```
        bodyArea.setBackground(UiTheme.SURFACE);
        bodyArea.setCaretColor(UiTheme.ACCENT);
        bodyArea.setLineWrap(true);
        bodyArea.setWrapStyleWord(true);
        bodyArea.setBorder(BorderFactory.createEmptyBorder(10, 12, 10, 12));
        bodyArea.getAccessibleContext().setAccessibleName("Message body");
        bodyArea.getAccessibleContext().setAccessibleDescription("Required, up to 10,000 characters.");
        JLabel bodyLabel = UiTheme.label("Message", Font.BOLD, 13, UiTheme.TEXT);
        bodyLabel.setLabelFor(bodyArea);
        bodyLabel.setDisplayedMnemonic(KeyEvent.VK_M);
        bodyPanel.add(bodyLabel, BorderLayout.NORTH);
        JScrollPane bodyScroll = new JScrollPane(bodyArea);
        bodyScroll.setBorder(BorderFactory.createLineBorder(UiTheme.BORDER));
        bodyScroll.setPreferredSize(new Dimension(560, 180));
        bodyPanel.add(bodyScroll, BorderLayout.CENTER);
        fields.add(bodyPanel, BorderLayout.CENTER);
        add(fields, BorderLayout.CENTER);

        JPanel footer = new JPanel(new BorderLayout(0, 10));
        footer.setOpaque(false);
        errorLabel = UiTheme.label(" ", Font.PLAIN, 12, ERROR);
        errorLabel.setName("composeError");
        errorLabel.putClientProperty("html.disable", Boolean.TRUE);
        errorLabel.getAccessibleContext().setAccessibleName("Compose status");
        footer.add(errorLabel, BorderLayout.NORTH);
        JPanel actions = new JPanel(new BorderLayout());
        actions.setOpaque(false);
        actions.add(UiTheme.label("Ctrl+Enter to send", Font.PLAIN, 12, UiTheme.MUTED),
                BorderLayout.WEST);
        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
        buttons.setOpaque(false);
        cancelButton = UiTheme.secondaryButton("Cancel");
        cancelButton.setName("cancelComposeButton");
        cancelButton.addActionListener(event -> cancel());
        sendButton = UiTheme.primaryButton(demoMode ? "Send demo message" : "Send message");
        sendButton.setName("sendButton");
        sendButton.addActionListener(event -> submit());
        buttons.add(cancelButton);
        buttons.add(sendButton);
        actions.add(buttons, BorderLayout.EAST);
        footer.add(actions, BorderLayout.CENTER);
        add(footer, BorderLayout.SOUTH);

        getInputMap(JComponent.WHEN_ANCESTOR_OF_FOCUSED_COMPONENT).put(
                KeyStroke.getKeyStroke(KeyEvent.VK_ENTER, KeyEvent.CTRL_DOWN_MASK), "sendDraft");
        getActionMap().put("sendDraft", new AbstractAction() {
            @Override
            public void actionPerformed(ActionEvent event) {
                submit();
            }
        });
        getInputMap(JComponent.WHEN_ANCESTOR_OF_FOCUSED_COMPONENT).put(
                KeyStroke.getKeyStroke(KeyEvent.VK_ESCAPE, 0), "cancelCompose");
        getActionMap().put("cancelCompose", new AbstractAction() {
            @Override
            public void actionPerformed(ActionEvent event) {
                cancel();
            }
        });
    }

    /** The owner calls this on the Swing event-dispatch thread while sending. */
    public void setBusy(boolean busy) {
        this.busy = busy;
        toField.setEnabled(!busy);
        subjectField.setEnabled(!busy);
        bodyArea.setEnabled(!busy);
        sendButton.setEnabled(!busy);
        cancelButton.setEnabled(!busy);
    }

    /** Shows a recoverable send error inline without discarding the draft. */
```

```
    public void showError(String message) {
        String text = message == null || message.isBlank() ? " " : message;
        errorLabel.setText(text);
        errorLabel.setToolTipText(text.isBlank() ? null : "Status: " + text);
    }

    private void submit() {
        if (busy) {
            return;
        }
        String rawRecipient = toField.getText();
        String recipient = rawRecipient.trim();
        String rawSubject = subjectField.getText();
        String subject = rawSubject.trim();
        String body = bodyArea.getText();
        if (rawRecipient.indexOf('\r') >= 0 || rawRecipient.indexOf('\n') >= 0
                || recipient.length() > 254 || !ADDRESS.matcher(recipient).matches()) {
            showError("Enter one email address, such as name@example.com.");
            toField.requestFocusInWindow();
            return;
        }
        if (subject.isBlank() || subject.length() > 200
                || rawSubject.indexOf('\r') >= 0 || rawSubject.indexOf('\n') >= 0) {
            showError("Enter a subject between 1 and 200 characters.");
            subjectField.requestFocusInWindow();
            return;
        }
        if (body.isBlank() || body.length() > 10_000) {
            showError("Enter a message between 1 and 10,000 characters.");
            bodyArea.requestFocusInWindow();
            return;
        }
        showError(null);
        onSend.accept(new Draft(recipient, subject, body));
    }

    private void cancel() {
        if (!busy) {
            onCancel.run();
        }
    }

    private static JPanel column() {
        JPanel panel = new JPanel();
        panel.setOpaque(false);
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
        return panel;
    }

    private static void addField(JPanel panel, String title, JTextField field, int mnemonic) {
        JLabel label = UiTheme.label(title, Font.BOLD, 13, UiTheme.TEXT);
        label.setLabelFor(field);
        label.setDisplayedMnemonic(mnemonic);
        label.setAlignmentX(LEFT_ALIGNMENT);
        field.setAlignmentX(LEFT_ALIGNMENT);
        field.setMaximumSize(new Dimension(Integer.MAX_VALUE, 38));
        field.setPreferredSize(new Dimension(560, 38));
        panel.add(label);
        panel.add(Box.createVerticalStrut(6));
        panel.add(field);
    }
}
```

src/main/java/ems/ui/MailAppPanel.java

```
package ems.ui;


import ems.model.*;
import ems.service.EmailService;
import java.awt.*;
```

```
import java.awt.event.*;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.EnumMap;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.CancellationException;
import java.util.concurrent.ExecutionException;
import java.util.function.Consumer;
import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.DefaultTableCellRenderer;

/** Member 1 UI. All service calls execute off the Swing event-dispatch thread. */
public final class MailAppPanel extends JPanel {
    private static final DateTimeFormatter DATE = DateTimeFormatter
            .ofPattern("dd MMM yyyy, HH:mm").withZone(ZoneId.systemDefault());
    private final EmailService service;
    private final boolean demo;
    private final CardLayout cards = new CardLayout();
    private final JPanel pages = UiTheme.panel(cards, UiTheme.BACKGROUND);
    private final EnumMap<Mailbox, JButton> navigation = new EnumMap<>(Mailbox.class);
    private final MailTableModel tableModel = new MailTableModel();
    private final JTable table = new JTable(tableModel);
    private final JTextField search = UiTheme.field("Search sender, subject or message");
    private final JLabel status = UiTheme.label("Ready to connect", Font.PLAIN, 12, UiTheme.MUTED);
    private final JLabel title = UiTheme.label("Inbox", Font.BOLD, 28, UiTheme.TEXT);
    private final JLabel subtitle = UiTheme.label("", Font.PLAIN, 12, UiTheme.MUTED);
    private final JLabel accountLabel = UiTheme.label("", Font.PLAIN, 11, new Color(0xB9C9D4));
    private final JButton connect = UiTheme.primaryButton("Connect Gmail");
    private final JButton refresh = UiTheme.secondaryButton("Refresh");
    private final JButton compose = UiTheme.primaryButton("+  Compose");
    private final JButton disconnect = UiTheme.secondaryButton("Disconnect");
    private final JButton markSpam = UiTheme.secondaryButton("Mark as spam");
    private final JButton notSpam = UiTheme.secondaryButton("Not spam");
    private final JButton searchButton = UiTheme.secondaryButton("Search");
    private final JPanel reader = UiTheme.panel(new BorderLayout(), UiTheme.SURFACE);
    private final JPanel listCards = UiTheme.panel(new CardLayout(), UiTheme.SURFACE);
    private final JLabel emptyTitle = UiTheme.label("No messages here", Font.BOLD, 18, UiTheme.TEXT);
    private final JLabel emptyDetail = UiTheme.label("", Font.PLAIN, 12, UiTheme.MUTED);
    private Account account;
    private Mailbox mailbox = Mailbox.INBOX;
    private boolean busy;
    private boolean updatingTable;
    private boolean connected;
    private EmailMessage openedMessage;

    public MailAppPanel(EmailService service) { this(service, false); }

    public MailAppPanel(EmailService service, boolean demoMode) {
        super(new BorderLayout());
        this.service = java.util.Objects.requireNonNull(service, "service");
        demo = demoMode;
        setBackground(UiTheme.BACKGROUND);
        setBorder(new EmptyBorder(0, 0, 0, 0));
        if (demo) add(demoBanner(), BorderLayout.NORTH);
        pages.add(connectionPage(), "connect");
        pages.add(mailPage(), "mail");
        add(pages, BorderLayout.CENTER);
        JPanel footer = UiTheme.panel(new BorderLayout(), UiTheme.SURFACE);
        footer.setBorder(new CompoundBorder(new MatteBorder(1, 0, 0, 0, UiTheme.BORDER),
                new EmptyBorder(9, 20, 9, 20)));
        status.setName("statusLabel");
        footer.add(status);
        footer.add(UiTheme.label("MAILDESK", Font.BOLD, 10, UiTheme.MUTED), BorderLayout.EAST);
        add(footer, BorderLayout.SOUTH);
        wireActions();
        updateControls();
    }

    private JPanel demoBanner() {
        JPanel banner = UiTheme.panel(new FlowLayout(FlowLayout.LEFT, 20, 7), new Color(0xFFF3D6));
```

```
        banner.add(UiTheme.label("DEMO MODE", Font.BOLD, 11, new Color(0x805A16)));
        banner.add(UiTheme.label("Sample messages only. Nothing is sent to Gmail.", Font.PLAIN, 12,
                new Color(0x805A16)));
        return banner;
    }

    private JPanel connectionPage() {
        JPanel page = UiTheme.panel(new GridBagLayout(), UiTheme.BACKGROUND);
        JPanel card = UiTheme.panel(new BorderLayout(50, 0), UiTheme.SURFACE);
        card.setBorder(new CompoundBorder(new LineBorder(UiTheme.BORDER), new EmptyBorder(46, 46, 46, 46)));
        JPanel copy = UiTheme.panel(new GridBagLayout(), UiTheme.SURFACE);
        GridBagConstraints c = new GridBagConstraints();
        c.gridx = 0; c.anchor = GridBagConstraints.WEST; c.fill = GridBagConstraints.HORIZONTAL;
        c.weightx = 1; c.insets = new Insets(0, 0, 22, 0);
        addRow(copy, UiTheme.label("MAILDESK", Font.BOLD, 15, UiTheme.ACCENT), c, 0);
        addRow(copy, UiTheme.label("Make room for", Font.BOLD, 35, UiTheme.TEXT), c, 1);
        c.insets = new Insets(-20, 0, 24, 0);
        addRow(copy, UiTheme.label("what matters.", Font.BOLD, 35, UiTheme.TEXT), c, 2);
        c.insets = new Insets(0, 0, 12, 0);
        addRow(copy, UiTheme.label("Your mail, a focused workspace, and a second", Font.PLAIN, 14,
UiTheme.MUTED), c, 3);
        addRow(copy, UiTheme.label("look at messages that seem suspicious.", Font.PLAIN, 14, UiTheme.MUTED), c,
4);
        c.insets = new Insets(18, 0, 15, 0);
        connect.setName("connectButton");
        connect.setText(demo ? "Open demo mailbox  →" : "Connect Gmail  →");
        addRow(copy, connect, c, 5);
        c.insets = new Insets(0, 0, 6, 0);
        addRow(copy, UiTheme.label(demo ? "Try the screens with fictional messages." : "Sign in securely in your
browser with Google.",
                Font.PLAIN, 12, UiTheme.MUTED), c, 6);
        addRow(copy, UiTheme.label(demo ? "No account or internet connection needed." : "Your Gmail password is
never entered here.",
                Font.PLAIN, 12, UiTheme.MUTED), c, 7);
        card.add(copy, BorderLayout.CENTER);
        JPanel features = UiTheme.panel(new GridLayout(3, 1, 0, 16), UiTheme.SURFACE);
        features.setPreferredSize(new Dimension(280, 360));
        features.add(feature("01", "One clear inbox", "Read, search and write with less clutter."));
        features.add(feature("02", "A second look", "See why a message was flagged for review."));
        features.add(feature("03", "You're in control", "Correct spam decisions when needed."));
        card.add(features, BorderLayout.EAST);
        page.add(card);
        return page;
    }

    private JPanel feature(String number, String heading, String detail) {
        JPanel panel = UiTheme.panel(new BorderLayout(0, 8), UiTheme.BACKGROUND);
        panel.setBorder(new EmptyBorder(16, 18, 16, 18));
        panel.add(UiTheme.label(number + "  /  " + heading, Font.BOLD, 14, UiTheme.TEXT), BorderLayout.NORTH);
        JTextArea text = new JTextArea(detail);
        text.setFont(UiTheme.font(Font.PLAIN, 12)); text.setForeground(UiTheme.MUTED);
        text.setOpaque(false); text.setEditable(false); text.setLineWrap(true); text.setWrapStyleWord(true);
        panel.add(text);
        return panel;
    }

    private JPanel mailPage() {
        JPanel page = UiTheme.panel(new BorderLayout(), UiTheme.BACKGROUND);
        page.add(sidebar(), BorderLayout.WEST);
        JPanel workspace = UiTheme.panel(new BorderLayout(0, 20), UiTheme.BACKGROUND);
        workspace.setBorder(new EmptyBorder(25, 24, 20, 24));
        JPanel header = UiTheme.panel(new BorderLayout(12, 0), UiTheme.BACKGROUND);
        JPanel heading = UiTheme.panel(new GridLayout(2, 1, 0, 5), UiTheme.BACKGROUND);
        heading.add(title); heading.add(subtitle); header.add(heading);
        JPanel actions = UiTheme.panel(new FlowLayout(FlowLayout.RIGHT, 8, 0), UiTheme.BACKGROUND);
        refresh.setName("refreshButton"); actions.add(refresh); header.add(actions, BorderLayout.EAST);
        JPanel top = UiTheme.panel(new BorderLayout(0, 18), UiTheme.BACKGROUND);
        top.add(header, BorderLayout.NORTH);
        JPanel searchRow = UiTheme.panel(new BorderLayout(8, 0), UiTheme.BACKGROUND);
        search.setName("searchField"); searchButton.setName("searchButton");
        searchRow.add(search); searchRow.add(searchButton, BorderLayout.EAST);
        top.add(searchRow); workspace.add(top, BorderLayout.NORTH);
```

```
        configureTable();
        JScrollPane scroll = new JScrollPane(table);
        // Install explicitly so headless previews and native windows have the same header.
        scroll.setColumnHeaderView(table.getTableHeader());
        scroll.setBorder(BorderFactory.createEmptyBorder());
scroll.getViewport().setBackground(UiTheme.SURFACE);
        listCards.add(scroll, "messages");
        JPanel empty = UiTheme.panel(new GridBagLayout(), UiTheme.SURFACE);
        JPanel emptyCopy = UiTheme.panel(new GridLayout(2, 1, 0, 12), UiTheme.SURFACE);
        emptyCopy.add(emptyTitle); emptyCopy.add(emptyDetail); empty.add(emptyCopy);
        listCards.add(empty, "empty");
        JPanel list = UiTheme.panel(new BorderLayout(), UiTheme.SURFACE);
        list.setBorder(new LineBorder(UiTheme.BORDER));
        JLabel listTitle = UiTheme.label("MESSAGES", Font.BOLD, 10, UiTheme.MUTED);
        listTitle.setBorder(new EmptyBorder(15, 14, 12, 14)); list.add(listTitle, BorderLayout.NORTH);
        list.add(listCards);
        list.setMinimumSize(new Dimension(360, 300));
        reader.setMinimumSize(new Dimension(350, 300)); reader.setBorder(new LineBorder(UiTheme.BORDER));
        JSplitPane split = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT, list, reader);
        split.setBorder(BorderFactory.createEmptyBorder()); split.setContinuousLayout(true);
        split.setResizeWeight(0.48); split.setDividerSize(9); split.setBackground(UiTheme.BACKGROUND);
        workspace.add(split); page.add(workspace); showReaderPlaceholder("Select a message", "Its contents and
spam review will appear here.");
        return page;
    }

    private JPanel sidebar() {
        JPanel sidebar = UiTheme.panel(new BorderLayout(0, 24), UiTheme.SIDEBAR);
        sidebar.setPreferredSize(new Dimension(212, 700));
        sidebar.setBorder(new EmptyBorder(30, 17, 18, 17));
        JPanel top = UiTheme.panel(new GridBagLayout(), UiTheme.SIDEBAR);
        GridBagConstraints c = new GridBagConstraints();
        c.gridx = 0; c.weightx = 1; c.fill = GridBagConstraints.HORIZONTAL; c.anchor = GridBagConstraints.WEST;
        c.insets = new Insets(0, 9, 5, 0);
        addRow(top, UiTheme.label("MAILDESK", Font.BOLD, 20, Color.WHITE), c, 0);
        addRow(top, UiTheme.label("A little more clarity.", Font.PLAIN, 11, new Color(0xB9C9D4)), c, 1);
        c.insets = new Insets(28, 0, 26, 0); compose.setName("composeButton"); addRow(top, compose, c, 2);
        c.insets = new Insets(0, 9, 12, 0);
        addRow(top, UiTheme.label("WORKSPACE", Font.BOLD, 10, new Color(0x91ACBC)), c, 3);
        int row = 4;
        for (Mailbox box : Mailbox.values()) {
            JButton button = new JButton(mailboxName(box));
            button.setHorizontalAlignment(SwingConstants.LEFT); button.setFont(UiTheme.font(Font.BOLD, 13));
            button.setOpaque(true); button.setFocusPainted(true);
            button.setBorder(new EmptyBorder(13, 12, 13, 8));
            button.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
            button.setName(switch (box) { case INBOX -> "inboxButton"; case SENT -> "sentButton";
                case SPAM -> "spamButton"; case SUSPECTED -> "suspectedButton"; });
            button.addActionListener(event -> changeMailbox(box));
            navigation.put(box, button); c.insets = new Insets(0, 0, 5, 0); addRow(top, button, c, row++);
        }
        sidebar.add(top, BorderLayout.NORTH);
        JPanel bottom = UiTheme.panel(new GridLayout(3, 1, 0, 9), UiTheme.SIDEBAR);
        bottom.add(UiTheme.label("CONNECTED ACCOUNT", Font.BOLD, 10, new Color(0x91ACBC)));
        bottom.add(accountLabel); disconnect.setName("disconnectButton"); bottom.add(disconnect);
        sidebar.add(bottom, BorderLayout.SOUTH);
        return sidebar;
    }

    private void configureTable() {
        table.setName("messageTable"); table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        table.setRowHeight(58); table.setShowGrid(false); table.setIntercellSpacing(new Dimension(0, 0));
        table.setBackground(UiTheme.SURFACE); table.setForeground(UiTheme.TEXT);
        table.setSelectionBackground(UiTheme.SELECTED); table.setSelectionForeground(UiTheme.TEXT);
        table.getTableHeader().setReorderingAllowed(false);
        table.getTableHeader().setBackground(new Color(0xF7F9FA));
        table.getTableHeader().setForeground(UiTheme.MUTED);
        table.getTableHeader().setFont(UiTheme.font(Font.BOLD, 10));
        table.getTableHeader().setPreferredSize(new Dimension(400, 35));
        table.setDefaultRenderer(Object.class, new DefaultTableCellRenderer() {
            @Override public Component getTableCellRendererComponent(JTable t, Object value,
                    boolean selected, boolean focused, int row, int column) {
                // Plain text only: mail content must not trigger Swing HTML rendering.
```

```
                putClientProperty("html.disable", Boolean.TRUE);
                super.getTableCellRendererComponent(t, value, selected, focused, row, column);
                setBorder(new CompoundBorder(new MatteBorder(0, 0, 1, 0, UiTheme.BORDER),
                        new EmptyBorder(0, 12, 0, 9)));
                setFont(UiTheme.font(tableModel.isUnread(row) ? Font.BOLD : Font.PLAIN, column == 2 ? 10 : 12));
                setForeground(column == 2 ? UiTheme.MUTED : UiTheme.TEXT);
                setBackground(selected ? UiTheme.SELECTED : UiTheme.SURFACE);
                setToolTipText("Message: " + value);
                return this;
            }
        });
        setColumnWidths();
        table.getSelectionModel().addListSelectionListener(event -> {
            if (!event.getValueIsAdjusting() && !updatingTable && !busy && table.getSelectedRow() >= 0) {
                open(tableModel.getMessageAt(table.getSelectedRow()).getGmailMessageId());
            }
        });
    }

    private void setColumnWidths() {
        table.getColumnModel().getColumn(0).setPreferredWidth(150);
        table.getColumnModel().getColumn(1).setPreferredWidth(215);
        table.getColumnModel().getColumn(2).setPreferredWidth(100);
    }

    private void wireActions() {
        connect.addActionListener(event -> runTask("Connecting…", () -> {
            Account value = service.connectGmail();
            return new Connected(value, service.getMessages(Mailbox.INBOX, ""), service.getLastWarning());
        }, result -> {
            account = result.account(); connected = true; mailbox = Mailbox.INBOX;
            accountLabel.setText(account.getEmailAddress()); accountLabel.setToolTipText("Account: " +
account.getEmailAddress());
            cards.show(pages, "mail"); applyMessages(result.messages());
            showNotice(withWarning(demo ? "Demo mailbox open. No Gmail connection is active."
                    : "Gmail connected. Inbox updated.", result.warning()), false);
            // Sign-in and synchronization are separate: a failed refresh must not erase
            // a valid Google grant or prevent the user from accessing the cached mailbox.
            runTask("Loading recent Gmail messages…", () -> {
                service.refreshRecent();
                return service.getMessages(Mailbox.INBOX, "");
            }, messages -> {
                applyMessages(messages);
                showNotice(demo ? "Demo mailbox open. No Gmail connection is active."
                        : "Sample mailbox loaded: up to 18 messages total (6 each from Inbox, Sent and Spam).",
false);
            });
        }));
        refresh.addActionListener(event -> load(true));
        searchButton.addActionListener(event -> load(false));
        search.addActionListener(event -> load(false));
        compose.addActionListener(event -> showCompose());
        disconnect.addActionListener(event -> close(() -> {
            account = null; connected = false; search.setText(""); tableModel.setMessages(List.of());
            openedMessage = null; cards.show(pages, "connect"); updateControls();
            showNotice("Disconnected. You can connect again when ready.", false);
        }));
        markSpam.setName("markSpamButton"); notSpam.setName("notSpamButton");
        markSpam.addActionListener(event -> correctSpam(true));
        notSpam.addActionListener(event -> correctSpam(false));
    }

    private void changeMailbox(Mailbox next) {
        if (busy || !connected) return;
        runTask("Loading " + mailboxName(next) + "…", () -> service.getMessages(next, ""), messages -> {
            mailbox = next; search.setText(""); applyMessages(messages);
            showNotice("Showing " + mailboxName(next).toLowerCase() + ".", false);
        });
        // The selected mailbox and query change only after the service succeeds.
    }

    private void load(boolean sync) {
```

```
        if (busy || !connected) return;
        String query = search.getText().strip();
        runTask(sync ? "Refreshing mailbox…" : "Searching messages…", () -> {
            if (sync) service.refresh();
            return service.getMessages(mailbox, query);
        }, messages -> {
            applyMessages(messages);
            showNotice(messages.size() + (messages.size() == 1 ? " message" : " messages")
                    + (query.isEmpty() ? " in this view." : " matching your search."), false);
        });
    }

    private void applyMessages(List<EmailMessage> messages) {
        updatingTable = true;
        tableModel.setSent(mailbox == Mailbox.SENT); tableModel.setMessages(messages); setColumnWidths();
        table.clearSelection(); updatingTable = false;
        openedMessage = null;
        title.setText(mailboxName(mailbox));
        subtitle.setText(switch (mailbox) {
            case INBOX -> "A clear view of what has arrived.";
            case SENT -> "Messages you have sent.";
            case SPAM -> "Messages classified as spam in Gmail.";
            case SUSPECTED -> "Our detector's suggestions. Review before deciding.";
        });
        ((CardLayout) listCards.getLayout()).show(listCards, messages.isEmpty() ? "empty" : "messages");
        emptyTitle.setText(search.getText().isBlank() ? "All clear here" : "No matching messages");
        emptyDetail.setText(search.getText().isBlank() ? "Refresh to check for new messages." : "Try another
search or clear the search field.");
        showReaderPlaceholder("Select a message", "Read it here without leaving your inbox.");
        updateControls(); revalidate(); repaint();
    }

    private void open(String id) {
        Mailbox currentMailbox = mailbox;
        String query = search.getText().strip();
        openedMessage = null;
        showReaderPlaceholder("Opening message…", "Loading the message and its review.");
        runTask("Opening message…", () -> new Opened(service.openMessage(id), service.getSpamResult(id),
                service.getMessages(currentMailbox, query)), result -> {
            updatingTable = true; tableModel.setMessages(result.messages());
            for (int row = 0; row < tableModel.getRowCount(); row++) {
                if (tableModel.getMessageAt(row).getGmailMessageId().equals(id))
table.setRowSelectionInterval(row, row);
            }
            updatingTable = false; openedMessage = result.message();
            showMessage(result.message(), result.result());
            showNotice("Message opened.", false);
        }, error -> {
            updatingTable = true; table.clearSelection(); updatingTable = false;
            showReaderPlaceholder("Couldn't open this message", "Select it again to retry, or refresh your
mailbox.");
            showNotice(error, true);
        });
    }

    private void showReaderPlaceholder(String heading, String detail) {
        reader.removeAll();
        JPanel center = UiTheme.panel(new GridBagLayout(), UiTheme.SURFACE);
        JPanel text = UiTheme.panel(new GridLayout(3, 1, 0, 12), UiTheme.SURFACE);
        text.add(UiTheme.label("○", Font.PLAIN, 42, UiTheme.ACCENT));
        text.add(UiTheme.label(heading, Font.BOLD, 20, UiTheme.TEXT));
        text.add(UiTheme.label(detail, Font.PLAIN, 12, UiTheme.MUTED)); center.add(text); reader.add(center);
        reader.revalidate(); reader.repaint();
    }

    private void showMessage(EmailMessage message, SpamResult result) {
        reader.removeAll();
        JPanel content = UiTheme.panel(new BorderLayout(0, 18), UiTheme.SURFACE);
        content.setBorder(new EmptyBorder(24, 24, 20, 24));
        ReaderArticle article = new ReaderArticle();
        JTextArea subject = plainText(message.getSubject().isBlank() ? "(No subject)" : message.getSubject(),
22, Font.BOLD);
```

```
        subject.setName("messageSubject"); article.add(subject);
        article.add(plainText("From  " + message.getSenderEmail(), 12, Font.BOLD));
        JTextArea recipients = plainText("To  " + String.join(", ", message.getRecipientEmails()), 12,
Font.PLAIN);
        recipients.setForeground(UiTheme.MUTED); article.add(recipients);
        article.add(UiTheme.label(DATE.format(message.getReceivedAt()), Font.PLAIN, 11, UiTheme.MUTED));
        if (mailbox != Mailbox.SENT) {
            ReaderColumn review = new ReaderColumn(6);
            review.setBackground(result.isSuspectedSpam() ? new Color(0xFFF5DE) : new Color(0xEDF5F2));
            review.setBorder(new EmptyBorder(12, 12, 12, 12));
            review.add(UiTheme.label(result.isSuspectedSpam() ? "Review suggested · Score " + result.getScore()
                    : "No rule flag · Score " + result.getScore(), Font.BOLD, 12, UiTheme.TEXT));
            String reasons = result.getReasons().isEmpty() ? "A rule score is not a guarantee of safety."
                    : String.join("\n", result.getReasons());
            JTextArea explanation = plainText(reasons, 11, Font.PLAIN); explanation.setName("spamReasons");
            review.add(explanation); article.add(review);
        }
        article.add(Box.createVerticalStrut(8));
        JTextArea body = plainText(message.getBodyText().isBlank() ? "(This message has no plain-text content.)"
: message.getBodyText(), 14, Font.PLAIN);
        body.setName("messageBody"); body.setBorder(new EmptyBorder(4, 0, 12, 8));
        article.add(body);
        JScrollPane scroll = new JScrollPane(article); scroll.setBorder(BorderFactory.createEmptyBorder());
        scroll.setHorizontalScrollBarPolicy(ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);
        scroll.getVerticalScrollBar().setUnitIncrement(18);
        scroll.getViewport().setBackground(UiTheme.SURFACE); content.add(scroll);
        JPanel actions = UiTheme.panel(new FlowLayout(FlowLayout.LEFT, 6, 0), UiTheme.SURFACE);
        if (mailbox != Mailbox.SENT) { actions.add(markSpam); actions.add(notSpam); }
        content.add(actions, BorderLayout.SOUTH); reader.add(content); reader.revalidate(); reader.repaint();
    }

    private void correctSpam(boolean spam) {
        if (busy || openedMessage == null) return;
        String id = openedMessage.getGmailMessageId();
        Mailbox currentMailbox = mailbox;
        String query = search.getText().strip();
        runTask(spam ? "Marking as spam…" : "Saving correction…", () -> {
            if (spam) service.markAsSpam(id); else service.markAsNotSpam(id);
            return new UpdatedMailbox(service.getMessages(currentMailbox, query), service.getLastWarning());
        }, result -> {
            applyMessages(result.messages());
            showNotice(withWarning(spam ? "Marked as spam. Your correction was saved."
                    : "Marked as not spam. Your correction was saved.", result.warning()), false);
        });
    }

    private void showCompose() {
        if (busy || account == null) return;
        JDialog dialog = new JDialog(SwingUtilities.getWindowAncestor(this), "New message",
Dialog.ModalityType.APPLICATION_MODAL);
        ComposePanel[] editor = new ComposePanel[1];
        editor[0] = new ComposePanel(account.getEmailAddress(), demo, draft -> {
            editor[0].setBusy(true);
            runTask(demo ? "Sending demo message…" : "Sending message…",
                    () -> new SentMessage(service.sendMessage(draft.recipientEmail(), draft.subject(),
draft.body()),
                            service.getLastWarning()), result -> {
                        dialog.dispose();
                        String notice = withWarning(demo ? "Demo message added to Sent. Nothing was sent to
Gmail."
                                : "Message sent successfully.", result.warning());
                        showNotice(notice, false);
                        if (mailbox == Mailbox.SENT) {
                            String sentQuery = search.getText().strip();
                            runTask("Updating Sent…", () -> service.getMessages(Mailbox.SENT, sentQuery),
                                    messages -> { applyMessages(messages); showNotice(notice, false); },
                                    error -> showNotice(notice + " " + error, true));
                        }
                    }, error -> {
                        editor[0].setBusy(false); editor[0].showError(error); showNotice(error, true);
                    });
        }, dialog::dispose);
        dialog.setContentPane(editor[0]); dialog.setDefaultCloseOperation(WindowConstants.DO_NOTHING_ON_CLOSE);
```

```
        dialog.addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent event) { if (!busy) dialog.dispose(); }
        });
        dialog.pack(); dialog.setMinimumSize(new Dimension(600, 530));
        dialog.setLocationRelativeTo(this); dialog.setVisible(true);
    }

    public boolean isBusy() { return busy; }

    public void showNotice(String message, boolean error) {
        status.setText(message); status.setForeground(error ? new Color(0xAA342A) : UiTheme.MUTED);
        status.setToolTipText("Status: " + message);
    }

    /** Finishes disconnection on a worker before invoking the callback on the EDT. */
    public void close(Runnable afterClose) {
        if (busy) return;
        runTask("Disconnecting…", () -> { service.disconnect(); return Boolean.TRUE; }, result -> {
            connected = false; afterClose.run();
        });
    }

    private void updateControls() {
        connect.setEnabled(!busy);
        for (var entry : navigation.entrySet()) {
            entry.getValue().setEnabled(connected && !busy);
            entry.getValue().setBackground(entry.getKey() == mailbox ? new Color(0x2D4C59) : UiTheme.SIDEBAR);
            entry.getValue().setForeground(entry.getKey() == mailbox ? Color.WHITE : new Color(0xC0CFD8));
        }
        for (JComponent control : new JComponent[] {refresh, compose, disconnect, search, searchButton, table})
{
            control.setEnabled(connected && !busy);
        }
        boolean canCorrect = connected && !busy && openedMessage != null
                && mailbox != Mailbox.SENT;
        markSpam.setEnabled(canCorrect); notSpam.setEnabled(canCorrect);
        setCursor(Cursor.getPredefinedCursor(busy ? Cursor.WAIT_CURSOR : Cursor.DEFAULT_CURSOR));
    }

    private <T> void runTask(String activity, Callable<T> work, Consumer<T> success) {
        runTask(activity, work, success, error -> showNotice(error, true));
    }

    private <T> void runTask(String activity, Callable<T> work, Consumer<T> success, Consumer<String> failure) {
        if (busy) return;
        busy = true; updateControls(); showNotice(activity, false);
        new SwingWorker<T, Void>() {
            @Override protected T doInBackground() throws Exception { return work.call(); }
            @Override protected void done() {
                busy = false;
                try {
                    T result = get();
                    success.accept(result);
                } catch (InterruptedException exception) {
                    Thread.currentThread().interrupt(); failure.accept("The operation was interrupted. Please
try again.");
                } catch (ExecutionException exception) {
                    Throwable cause = exception.getCause();
                    String message = cause instanceof ems.exception.AppException ? cause.getMessage()
                            : "The operation could not finish. Please try again.";
                    failure.accept(message == null || message.isBlank() ? "The operation could not finish." :
message);
                } catch (CancellationException exception) {
                    failure.accept("The operation was cancelled.");
                } finally { updateControls(); }
            }
        }.execute();
    }

    private static JTextArea plainText(String text, int size, int style) {
```

```
    JTextArea area = new JTextArea(text);
    area.setFont(UiTheme.font(style, size)); area.setForeground(UiTheme.TEXT);
    area.setOpaque(false); area.setEditable(false); area.setLineWrap(true); area.setWrapStyleWord(true);
    area.setCaretPosition(0); return area;
}

private static String mailboxName(Mailbox box) {
    return switch (box) { case INBOX -> "Inbox"; case SENT -> "Sent"; case SPAM -> "Gmail Spam";
        case SUSPECTED -> "Suspected Spam"; };
}

private static void addRow(JPanel panel, JComponent component, GridBagConstraints c, int row) {
    c.gridy = row; panel.add(component, c);
}

private static String withWarning(String message, String warning) {
    return warning == null || warning.isBlank() ? message : message + " " + warning;
}

private record Connected(Account account, List<EmailMessage> messages, String warning) { }
private record UpdatedMailbox(List<EmailMessage> messages, String warning) { }
private record SentMessage(String id, String warning) { }
private record Opened(EmailMessage message, SpamResult result, List<EmailMessage> messages) { }

/**
 * Measures wrapped text at its allocated width before computing scroll height.
 * Standard BorderLayout/BoxLayout preferences can use a text area's old width,
 * cutting off its final lines after a narrow viewport is assigned.
 */
private static class ReaderColumn extends JPanel {
    private final int gap;

    ReaderColumn(int gap) {
        super(null);
        this.gap = gap;
        setBackground(UiTheme.SURFACE);
    }

    @Override public Dimension getPreferredSize() {
        int width = getParent() instanceof JViewport viewport
                ? viewport.getExtentSize().width : getWidth();
        if (width <= 0) width = 420;
        return new Dimension(width, heightForWidth(width));
    }

    private int heightForWidth(int width) {
        Insets insets = getInsets();
        int available = Math.max(1, width - insets.left - insets.right);
        int height = insets.top + insets.bottom;
        boolean first = true;
        for (Component child : getComponents()) {
            if (!child.isVisible()) continue;
            if (!first) height += gap;
            height += childHeight(child, available);
            first = false;
        }
        return height;
    }

    @Override public void doLayout() {
        Insets insets = getInsets();
        int width = Math.max(1, getWidth() - insets.left - insets.right);
        int y = insets.top;
        boolean first = true;
        for (Component child : getComponents()) {
            if (!child.isVisible()) continue;
            if (!first) y += gap;
            int height = childHeight(child, width);
            child.setBounds(insets.left, y, width, height);
```

```
                y += height;
                first = false;
            }
        }

        private static int childHeight(Component child, int width) {
            if (child instanceof ReaderColumn column) return column.heightForWidth(width);
            if (child instanceof JTextArea text) {
                Insets insets = text.getInsets();
                javax.swing.text.View view = text.getUI().getRootView(text);
                view.setSize(Math.max(1, width - insets.left - insets.right), Integer.MAX_VALUE);
                return (int) Math.ceil(view.getPreferredSpan(javax.swing.text.View.Y_AXIS))
                        + insets.top + insets.bottom;
            }
            return child.getPreferredSize().height;
        }
    }

    /** Incoming headers, spam reasons and body share one width-tracking scroll area. */
    private static final class ReaderArticle extends ReaderColumn implements Scrollable {
        ReaderArticle() { super(10); }
        @Override public Dimension getPreferredScrollableViewportSize() { return new Dimension(420, 500); }
        @Override public int getScrollableUnitIncrement(Rectangle r, int orientation, int direction) { return
18; }
        @Override public int getScrollableBlockIncrement(Rectangle r, int orientation, int direction) { return
Math.max(18, r.height - 36); }
        @Override public boolean getScrollableTracksViewportWidth() { return true; }
        @Override public boolean getScrollableTracksViewportHeight() { return false; }
    }
}
```

src/main/java/ems/ui/MailTableModel.java

```
package ems.ui;

import ems.model.EmailMessage;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.List;
import javax.swing.table.AbstractTableModel;

/** Table keeps message IDs with their rows; the UI never identifies mail by subject. */
public final class MailTableModel extends AbstractTableModel {
    private static final DateTimeFormatter DATE = DateTimeFormatter.ofPattern("dd MMM, HH:mm")
            .withZone(ZoneId.systemDefault());
    private List<EmailMessage> messages = List.of();
    private boolean sent;

    public void setMessages(List<EmailMessage> messages) {
        this.messages = List.copyOf(messages);
        fireTableDataChanged();
    }

    public void setSent(boolean sent) {
        this.sent = sent;
        fireTableStructureChanged();
    }

    public EmailMessage getMessageAt(int row) { return messages.get(row); }
    public boolean isUnread(int row) {
        return messages.get(row).getGmailLabelIds().contains("UNREAD");
    }
    @Override public int getRowCount() { return messages.size(); }
    @Override public int getColumnCount() { return 3; }
    @Override public String getColumnName(int column) {
        return switch (column) {
            case 0 -> sent ? "TO" : "FROM";
            case 1 -> "SUBJECT";
```

```
            default -> sent ? "SENT AT" : "RECEIVED";
        };
    }
    @Override public Object getValueAt(int row, int column) {
        EmailMessage message = messages.get(row);
        return switch (column) {
            case 0 -> sent ? String.join(", ", message.getRecipientEmails()) : message.getSenderEmail();
            case 1 -> message.getSubject().isBlank() ? "(No subject)" : message.getSubject();
            default -> DATE.format(message.getReceivedAt());
        };
    }
}
```

src/main/java/ems/ui/MainFrame.java

```
package ems.ui;

import ems.service.EmailService;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

/** Production integration: construct on the EDT with Member 2's EmailService. */
public final class MainFrame extends JFrame {
    private final MailAppPanel appPanel;

    public MainFrame(EmailService service) { this(service, false); }

    public MainFrame(EmailService service, boolean demoMode) {
        super(demoMode ? "Maildesk — Interface demo" : "Maildesk — Email management");
        if (!SwingUtilities.isEventDispatchThread()) {
            throw new IllegalStateException("Create MainFrame on the Swing event-dispatch thread.");
        }
        appPanel = new MailAppPanel(service, demoMode);
        setContentPane(appPanel);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        Rectangle available = GraphicsEnvironment.getLocalGraphicsEnvironment().getMaximumWindowBounds();
        setMinimumSize(new Dimension(Math.min(1100, available.width), Math.min(700, available.height)));
        setSize(Math.min(1360, available.width), Math.min(860, available.height));
        setLocationRelativeTo(null);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent event) {
                if (appPanel.isBusy()) {
                    appPanel.showNotice("Please wait for the current operation to finish.", true);
                    return;
                }
                appPanel.close(MainFrame.this::dispose);
            }
        });
    }

    public MailAppPanel getAppPanel() { return appPanel; }
}
```

src/main/java/ems/ui/UiTheme.java

```
package ems.ui;

import java.awt.*;
import javax.swing.*;
import javax.swing.border.*;

/** Small, dependency-free style helpers shared by Member 1's screens. */
public final class UiTheme {
    public static final Color BACKGROUND = new Color(0xF4F6F8);
    public static final Color SURFACE = Color.WHITE;
    public static final Color TEXT = new Color(0x182C3C);
```

```
public static final Color MUTED = new Color(0x667684);
public static final Color ACCENT = new Color(0x117568);
public static final Color BORDER = new Color(0xDFE5E9);
public static final Color SIDEBAR = new Color(0x172C3C);
public static final Color SELECTED = new Color(0xEAF4F1);

private UiTheme() { }

public static void install() {
    try {
        UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
    } catch (Exception ignored) {
        // Swing's installed look and feel remains usable.
    }
    Font base = font(Font.PLAIN, 13);
    for (String key : new String[] {"Label.font", "Button.font", "TextField.font",
            "TextArea.font", "Table.font", "TableHeader.font", "OptionPane.messageFont",
            "OptionPane.buttonFont", "ToolTip.font"}) {
        UIManager.put(key, base);
    }
    UIManager.put("Panel.background", BACKGROUND);
    UIManager.put("ScrollBar.width", 11);
    UIManager.put("TextField.selectionBackground", new Color(0xC4E4DE));
}

public static Font font(int style, int size) {
    return new Font("SansSerif", style, size);
}

public static JLabel label(String text, int style, int size, Color color) {
    JLabel label = new JLabel();
    label.putClientProperty("html.disable", Boolean.TRUE);
    label.setText(text);
    label.setFont(font(style, size));
    label.setForeground(color);
    return label;
}

public static JButton primaryButton(String text) {
    return button(text, ACCENT, Color.WHITE);
}

public static JButton secondaryButton(String text) {
    return button(text, SURFACE, TEXT);
}

private static JButton button(String text, Color background, Color foreground) {
    JButton button = new JButton(text);
    button.setFont(font(Font.BOLD, 13));
    button.setBackground(background);
    button.setForeground(foreground);
    button.setOpaque(true);
    button.setContentAreaFilled(true);
    button.setFocusPainted(true);
    button.setBorder(new CompoundBorder(new LineBorder(
            background.equals(SURFACE) ? BORDER : background, 1, true),
            new EmptyBorder(10, 16, 10, 16)));
    button.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    return button;
}

public static JTextField field(String placeholder) {
    JTextField field = new JTextField() {
        @Override protected void paintComponent(Graphics graphics) {
            super.paintComponent(graphics);
            if (getText().isEmpty() && !isFocusOwner()) {
                Graphics2D g = (Graphics2D) graphics.create();
                g.setFont(getFont());
                g.setColor(MUTED);
                g.drawString(placeholder, getInsets().left,
```

```
                        (getHeight() - g.getFontMetrics().getHeight()) / 2
                                + g.getFontMetrics().getAscent());
                    g.dispose();
                }
            }
        };
        field.setFont(font(Font.PLAIN, 13));
        field.setForeground(TEXT);
        field.setBackground(SURFACE);
        field.setBorder(new CompoundBorder(new LineBorder(BORDER, 1, true),
                new EmptyBorder(10, 12, 10, 12)));
        field.getAccessibleContext().setAccessibleName(placeholder);
        return field;
    }

    static JPanel panel(java.awt.LayoutManager layout, Color color) {
        JPanel panel = new JPanel(layout);
        panel.setBackground(color);
        return panel;
    }
}
```

APPENDIX A.2 — DEMONSTRATION JAVA

src/demo/java/ems/demo/DemoApp.java

```
package ems.demo;

import ems.ui.MainFrame;
import ems.ui.UiTheme;

import javax.swing.SwingUtilities;

/** Starts the interface with fictional, nonpersistent local data. */
public final class DemoApp {
    private DemoApp() {
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            UiTheme.install();
            MainFrame frame = new MainFrame(new DemoEmailService(), true);
            frame.setVisible(true);
        });
    }
}
```

src/demo/java/ems/demo/DemoEmailService.java

```
package ems.demo;

import ems.exception.AppException;
import ems.model.Account;
import ems.model.EmailMessage;
import ems.model.Mailbox;
import ems.model.SpamResult;
import ems.service.EmailService;

import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
```

```
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.regex.Pattern;

/**
 * In-memory demonstration only. This class never authenticates with Google,
 * contacts a mail server, sends an actual email, or stores data on disk.
 */
public final class DemoEmailService implements EmailService {
    private static final Account ACCOUNT = new Account(1L, "student@example.test");
    private static final Pattern EMAIL_ADDRESS = Pattern.compile(
            "^[^\\s@,;]+@[^\\s@,;]+\\.[^\\s@,;]+$");
    private static final Instant SAMPLE_TIME = Instant.parse("2026-10-20T06:00:00Z");

    private final Map<String, EmailMessage> messages = new LinkedHashMap<>();
    private final Map<String, SpamResult> spamResults = new LinkedHashMap<>();
    private final Map<String, Boolean> spamOverrides = new LinkedHashMap<>();
    private boolean connected;
    private long nextSentId = 1L;

    public DemoEmailService() {
        seedMessages();
    }

    @Override
    public synchronized Account connectGmail() {
        connected = true;
        return ACCOUNT;
    }

    @Override
    public synchronized void disconnect() {
        connected = false;
    }

    @Override
    public synchronized void refresh() throws AppException {
        requireConnected();
        // Demo data is local. Explicit corrections survive refresh/reconnect.
        for (String messageId : messages.keySet()) {
            Boolean override = spamOverrides.get(messageId);
            if (override != null) {
                spamResults.put(messageId, correctionResult(override));
            }
        }
    }

    @Override
    public synchronized List<EmailMessage> getMessages(Mailbox mailbox, String query)
            throws AppException {
        requireConnected();
        if (mailbox == null) {
            throw new AppException("Choose a mailbox first.");
        }
        String search = query == null ? "" : query.strip().toLowerCase(Locale.ROOT);
        List<EmailMessage> result = new ArrayList<>();
        for (EmailMessage message : messages.values()) {
            if (belongsToMailbox(message, mailbox) && matchesSearch(message, search)) {
                result.add(message);
            }
        }
        result.sort(Comparator.comparing(EmailMessage::getReceivedAt)
                .reversed().thenComparing(EmailMessage::getGmailMessageId));
        return List.copyOf(result);
    }

    @Override
    public synchronized EmailMessage openMessage(String gmailMessageId) throws AppException {
        EmailMessage message = requireMessage(gmailMessageId);
```

```
    if (message.getGmailLabelIds().contains("UNREAD")) {
        Set<String> labels = new LinkedHashSet<>(message.getGmailLabelIds());
        labels.remove("UNREAD");
        message = withLabels(message, labels);
        messages.put(gmailMessageId, message);
    }
    return message;
}

@Override
public synchronized String sendMessage(String recipientEmail, String subject, String body)
        throws AppException {
    requireConnected();
    String recipient = recipientEmail == null ? "" : recipientEmail.strip();
    String cleanSubject = subject == null ? "" : subject.strip();
    if (recipient.length() > 254 || !EMAIL_ADDRESS.matcher(recipient).matches()) {
        throw new AppException("Enter one valid recipient email address.");
    }
    if (cleanSubject.isEmpty() || cleanSubject.length() > 200) {
        throw new AppException("The subject must contain 1 to 200 characters.");
    }
    if (body == null || body.isBlank() || body.length() > 10_000) {
        throw new AppException("The message must contain 1 to 10,000 characters.");
    }
    String messageId = "demo-sent-" + nextSentId++;
    Set<String> labels = new LinkedHashSet<>();
    labels.add("SENT");
    if (recipient.equalsIgnoreCase(ACCOUNT.getEmailAddress())) {
        labels.add("INBOX");
    }
    EmailMessage message = new EmailMessage(ACCOUNT.getAccountId(), messageId,
            ACCOUNT.getEmailAddress(), List.of(recipient), cleanSubject, body,
            Instant.now(), Set.copyOf(labels));
    messages.put(messageId, message);
    spamResults.put(messageId, new SpamResult(0, false,
            List.of("Created locally in demo mode; no email was delivered.")));
    return messageId;
}

@Override
public synchronized SpamResult getSpamResult(String gmailMessageId) throws AppException {
    requireMessage(gmailMessageId);
    return spamResults.get(gmailMessageId);
}

@Override
public synchronized void markAsSpam(String gmailMessageId) throws AppException {
    EmailMessage message = requireReceivedMessage(gmailMessageId);
    Set<String> labels = new LinkedHashSet<>(message.getGmailLabelIds());
    labels.remove("INBOX");
    labels.add("SPAM");
    messages.put(gmailMessageId, withLabels(message, labels));
    spamOverrides.put(gmailMessageId, true);
    spamResults.put(gmailMessageId, correctionResult(true));
}

@Override
public synchronized void markAsNotSpam(String gmailMessageId) throws AppException {
    EmailMessage message = requireReceivedMessage(gmailMessageId);
    Set<String> labels = new LinkedHashSet<>(message.getGmailLabelIds());
    labels.remove("SPAM");
    labels.add("INBOX");
    messages.put(gmailMessageId, withLabels(message, labels));
    spamOverrides.put(gmailMessageId, false);
    spamResults.put(gmailMessageId, correctionResult(false));
}

private void requireConnected() throws AppException {
    if (!connected) {
        throw new AppException("Connect the demo account before opening your mailbox.");
    }
```

```
}

private EmailMessage requireMessage(String messageId) throws AppException {
    requireConnected();
    EmailMessage message = messages.get(messageId);
    if (message == null || message.getAccountId() != ACCOUNT.getAccountId()) {
        throw new AppException("This message is unavailable in the connected account.");
    }
    return message;
}

private EmailMessage requireReceivedMessage(String messageId) throws AppException {
    EmailMessage message = requireMessage(messageId);
    if (message.getGmailLabelIds().contains("SENT")
            && !message.getGmailLabelIds().contains("INBOX")
            && !message.getGmailLabelIds().contains("SPAM")) {
        throw new AppException("Spam corrections apply to received messages.");
    }
    return message;
}

private boolean belongsToMailbox(EmailMessage message, Mailbox mailbox) {
    Set<String> labels = message.getGmailLabelIds();
    return switch (mailbox) {
        case INBOX -> labels.contains("INBOX") && !labels.contains("SPAM");
        case SENT -> labels.contains("SENT");
        case SPAM -> labels.contains("SPAM");
        case SUSPECTED -> !labels.contains("SPAM") && !labels.contains("SENT")
                && spamResults.get(message.getGmailMessageId()).isSuspectedSpam();
    };
}

private boolean matchesSearch(EmailMessage message, String query) {
    if (query.isEmpty()) {
        return true;
    }
    String searchable = message.getSenderEmail() + "\n"
            + String.join("\n", message.getRecipientEmails()) + "\n"
            + message.getSubject() + "\n" + message.getBodyText();
    return searchable.toLowerCase(Locale.ROOT).contains(query);
}

private static EmailMessage withLabels(EmailMessage message, Set<String> labels) {
    return new EmailMessage(message.getAccountId(), message.getGmailMessageId(),
            message.getSenderEmail(), message.getRecipientEmails(), message.getSubject(),
            message.getBodyText(), message.getReceivedAt(), Set.copyOf(labels));
}

private static SpamResult correctionResult(boolean spam) {
    return new SpamResult(spam ? 100 : 0, spam,
            List.of(spam ? "You marked this message as spam."
                    : "You marked this message as not spam."));
}

private void seedMessages() {
    add("demo-meeting", "maya@campus.test", "Project catch-up at 4 PM",
            "Hi! Let's meet in the library at 4 PM to review the email management project.\n\n"
                    + "Please bring your screen sketches and the shared method contract.\n\nMaya",
            0, Set.of("INBOX", "UNREAD"), 0, false,
            "A familiar project update with no matching warning rules.");
    add("demo-build", "build@team.test", "The interface branch is ready to review",
            "The new mailbox screens are ready for review.\n\n"
                    + "Please check keyboard navigation, compose validation, and the empty state.\n\n"
                    + "This fictional notification is part of the demo mailbox.",
            45, Set.of("INBOX", "UNREAD"), 0, false,
            "No suspicious demo rules matched.");
    add("demo-invoice", "billing@invoice-review.test", "Action needed: verify your payment today",
            "Your payment is on hold. Verify your billing details immediately to avoid suspension.\n\n"
                    + "Open https://billing-check.example.test/verify and enter your account details.\n\n"
                    + "This is a fictional suspicious message for testing. Do not enter real details.",
```

```
                110, Set.of("INBOX", "UNREAD"), 78, true,
                "Urgent payment language.", "Requests sensitive billing details.",
                "Sender and link domains differ.");
        add("demo-library", "library@campus.test", "Your book reservation is available",
                "Your reserved database systems book is available at the circulation desk.\n\n"
                        + "Please collect it by Friday. Your library card is all you need.",
                180, Set.of("INBOX"), 0, false,
                "No suspicious demo rules matched.");
        add("demo-internship", "careers@campus.test", "Campus internship briefing — Thursday",
                "The careers team is holding a briefing on Thursday at 11 AM.\n\n"
                        + "We will cover application timelines and interview preparation. Attendance is
optional.",
                260, Set.of("INBOX", "UNREAD"), 5, false,
                "An informational campus announcement.");
        add("demo-password", "security@account-alert.test", "Your account will close in 24 hours",
                "Confirm your password now or your mailbox will be closed.\n\n"
                        + "Reply with your password and verification code to keep access.\n\n"
                        + "Fictional phishing example: legitimate services should not request passwords by
email.",
                370, Set.of("INBOX"), 94, true,
                "Requests a password and verification code.", "Threatens account closure.",
                "Uses urgency to pressure the recipient.");
        add("demo-newsletter", "newsletter@coding-club.test", "This week at the coding club",
                "This week's events:\n\nMonday: Java study group\nWednesday: SQL practice\n"
                        + "Friday: student project demonstrations\n\nSee you there!",
                500, Set.of("INBOX"), 0, false,
                "An ordinary fictional newsletter.");
        add("demo-prize", "offers@prize-draw.test", "You won! Claim a surprise reward",
                "You have been selected for a large cash reward. Pay a small processing fee to claim it
today.\n\n"
                        + "This fictional spam example is already filed in the demo Gmail Spam folder.",
                600, Set.of("SPAM", "UNREAD"), 96, true,
                "Unsolicited prize claim.", "Requests an upfront processing fee.");
        add("demo-travel", "deals@travel-promo.test", "Last chance: exclusive holiday discount",
                "A limited offer is waiting. Book immediately to unlock your special discount.\n\n"
                        + "This fictional promotion is already in the demo Gmail Spam folder.",
                750, Set.of("SPAM", "UNREAD"), 62, true,
                "Promotional urgency.", "Unsolicited offer from an unfamiliar sender.");
        add("demo-welcome", "hello@maildesk.test", "Welcome to your demo mailbox",
                "Welcome! This is an in-memory preview of the email management interface.\n\n"
                        + "All messages are fictional. Sending saves a message in demo Sent only; "
                        + "no email leaves this application. No Google sign-in, database, or network is
used.\n\n"
                        + "Try searching, composing, and reviewing the two suspected spam examples.",
                1_440, Set.of("INBOX"), 0, false,
                "A local demonstration message.");
        addSent("demo-sent-report", "maya@campus.test", "Project progress and next steps",
                "Hi Maya,\n\nThe member assignments and shared method signatures are ready. "
                        + "I will finish the Swing screens next.\n\nThanks!", 200);
        addSent("demo-sent-review", "dev@team.test", "Please review the compose screen",
                "The compose form now checks recipient, subject, and body before sending.\n\n"
                        + "Please try the keyboard shortcuts when you review it.", 1_600);
    }

    private void add(String id, String sender, String subject, String body, long minutesAgo,
                     Set<String> labels, int score, boolean suspected, String... reasons) {
        messages.put(id, new EmailMessage(ACCOUNT.getAccountId(), id, sender,
                List.of(ACCOUNT.getEmailAddress()), subject, body,
                SAMPLE_TIME.minus(minutesAgo, ChronoUnit.MINUTES), labels));
        spamResults.put(id, new SpamResult(score, suspected, List.of(reasons)));
    }

    private void addSent(String id, String recipient, String subject, String body, long minutesAgo) {
        messages.put(id, new EmailMessage(ACCOUNT.getAccountId(), id, ACCOUNT.getEmailAddress(),
                List.of(recipient), subject, body,
                SAMPLE_TIME.minus(minutesAgo, ChronoUnit.MINUTES), Set.of("SENT")));
        spamResults.put(id, new SpamResult(0, false,
                List.of("A fictional sent message in the demo mailbox.")));
    }
}
```

APPENDIX A.3 — JAVA VERIFICATION SOURCES

src/test/java/com/emailapp/AppTest.java

```
package com.emailapp;

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class AppTest {
    @Test void teammateLauncherUsesProductionHelp() {
        PrintStream previous = System.out;
        ByteArrayOutputStream output = new ByteArrayOutputStream();
        try {
            System.setOut(new PrintStream(output, true, StandardCharsets.UTF_8));
            App.main(new String[] {"--help"});
        } finally { System.setOut(previous); }
        assertTrue(output.toString(StandardCharsets.UTF_8).contains("--check-setup"));
    }
}
```

src/test/java/com/emailapp/SpamClassifierTest.java

```
package com.emailapp;

import com.emailapp.spam.SpamClassifier;
import ems.model.EmailMessage;
import ems.model.SpamResult;
import java.time.Instant;
import java.util.List;
import java.util.Set;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class SpamClassifierTest {
    private final SpamClassifier classifier = new SpamClassifier();

    private EmailMessage message(String subject, String body, Set<String> labels) {
        return new EmailMessage(1, "message", "sender@example.com", List.of("me@example.com"),
                subject, body, Instant.parse("2026-10-20T10:15:00Z"), labels);
    }

    @Test void ordinaryMailNeedsNoDatabaseAndIsNotSpam() {
        SpamResult result = classifier.analyze(message("Meeting notes", "See you on Tuesday.",
Set.of("INBOX")));
        assertEquals(0, result.getScore());
        assertFalse(result.isSuspectedSpam());
    }

    @Test void oneCommonPhraseDoesNotFlagLegitimateAccountMail() {
        SpamResult result = classifier.analyze(message("Please verify your account", "Welcome to our service.",
Set.of("INBOX")));
        assertEquals(20, result.getScore());
        assertFalse(result.isSuspectedSpam());
    }

    @Test void multipleIndependentSignalsCanReachSpamThreshold() {
        SpamResult result = classifier.analyze(message("URGENT: Claim your prize", "You have won! Verify your
account now. "
                + "https://one.example https://two.example https://three.example https://four.example
https://five.example", Set.of("INBOX")));
        assertTrue(result.isSuspectedSpam());
        assertTrue(result.getScore() >= 50);
```

```
        assertTrue(result.getReasons().contains("Message contains many links"));
    }

    @Test void gmailSpamLabelDoesNotInfluenceOurClassifier() {
        SpamResult result = classifier.analyze(message("Meeting notes", "See you on Tuesday.", Set.of("SPAM")));
        assertEquals(0, result.getScore());
        assertFalse(result.isSuspectedSpam());
    }

    @Test void blockedSenderIsFlaggedWithReason() {
        SpamResult result = classifier.analyze(message("Hello", "Hello", Set.of("INBOX")), true);
        assertTrue(result.isSuspectedSpam());
        assertTrue(result.getReasons().contains("Sender is blocked"));
    }

    @Test void repeatedSinglePhraseDoesNotAccumulate() {
        SpamResult result = classifier.analyze(message("Hello", "you have won you have won you have won",
Set.of("INBOX")));
        assertEquals(20, result.getScore());
    }
}
```

src/test/java/ems/check/MemberOneChecks.java

```
package ems.check;

import ems.demo.DemoEmailService;
import ems.exception.AppException;
import ems.model.Account;
import ems.model.EmailMessage;
import ems.model.Mailbox;
import ems.model.SpamResult;
import ems.service.EmailService;

import java.time.Instant;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Set;
import java.util.stream.Collectors;

/** Dependency-free contract checks. No real Gmail account, database, or window is used. */
public final class MemberOneChecks {
    private static int assertions;

    private MemberOneChecks() {
    }

    public static void main(String[] args) throws Exception {
        checkModelBoundaries();
        checkDemoFlow();
        System.out.println("PASS: " + assertions + " contract/demo assertions (headless).");
    }

    private static void checkModelBoundaries() {
        List<String> recipients = new ArrayList<>(List.of("reader@example.test"));
        Set<String> labels = new HashSet<>(Set.of("INBOX", "UNREAD"));
        EmailMessage message = new EmailMessage(1L, "boundary-test", "writer@example.test",
                recipients, "Subject", "Body", Instant.parse("2026-01-01T12:00:00Z"), labels);
        recipients.clear();
        labels.clear();
        check(message.getRecipientEmails().equals(List.of("reader@example.test")),
                "Message recipients must not change when the caller mutates its input list.");
        check(message.getGmailLabelIds().equals(Set.of("INBOX", "UNREAD")),
                "Message labels must not change when the caller mutates its input set.");
        expectUnsupported(() -> message.getRecipientEmails().clear(),
```

```
                "Recipient lists must not permit callers to change cached state.");
        expectUnsupported(() -> message.getGmailLabelIds().clear(),
                "Label sets must not permit callers to change cached state.");


        List<String> reasons = new ArrayList<>(List.of("Blocked sender"));
        SpamResult result = new SpamResult(80, true, reasons);
        reasons.clear();
        check(result.getReasons().equals(List.of("Blocked sender")),
                "Spam reasons must be copied before they cross the UI/backend boundary.");
        expectUnsupported(() -> result.getReasons().clear(),
                "Spam reasons must not permit callers to change cached state.");
    }

    private static void checkDemoFlow() throws AppException {
        EmailService service = new DemoEmailService();
        expectAppException(() -> service.getMessages(Mailbox.INBOX, ""),
                "Mailbox access must require a connection.");
        expectAppException(service::refresh, "Refresh must require a connection.");


        Account account = service.connectGmail();
        check(account.getAccountId() > 0 && account.getEmailAddress().endsWith(".test"),
                "The demo must connect to a fictional account.");
        List<EmailMessage> inbox = service.getMessages(Mailbox.INBOX, "");
        check(!inbox.isEmpty(), "Connected demo must provide inbox fixtures.");
        checkNewestFirst(inbox);
        check(inbox.stream().allMatch(message -> message.getAccountId() == account.getAccountId()),
                "Inbox messages must belong to the connected account.");


        EmailMessage meeting = find(inbox, "demo-meeting");
        check(meeting.getGmailLabelIds().contains("UNREAD"), "Meeting fixture should begin unread.");
        check(contains(service.getMessages(Mailbox.INBOX,
                        meeting.getSubject().toUpperCase(Locale.ROOT)), meeting.getGmailMessageId()),
                "Search should match subjects without case sensitivity.");
        check(contains(service.getMessages(Mailbox.INBOX,
                        meeting.getSenderEmail().toUpperCase(Locale.ROOT)), meeting.getGmailMessageId()),
                "Search should match the sender.");
        check(contains(service.getMessages(Mailbox.INBOX, meeting.getBodyText()),
                        meeting.getGmailMessageId()),
                "Search should match the message body.");
        check(contains(service.getMessages(Mailbox.INBOX, meeting.getRecipientEmails().get(0)),
                        meeting.getGmailMessageId()),
                "Search should match recipients.");
        check(service.getMessages(Mailbox.INBOX, "this-search-has-no-match-90453").isEmpty(),
                "Search should return an empty list when nothing matches.");


        EmailMessage opened = service.openMessage(meeting.getGmailMessageId());
        check(!opened.getGmailLabelIds().contains("UNREAD"), "Opening an email must mark it read.");
        check(!find(service.getMessages(Mailbox.INBOX, ""), meeting.getGmailMessageId())
                        .getGmailLabelIds().contains("UNREAD"),
                "Read status must persist within the demo session.");


        int sentCount = service.getMessages(Mailbox.SENT, "").size();
        String sentId = service.sendMessage("outside@example.test", "Project update", "The UI is ready.");
        check(sentId != null && !sentId.isBlank(), "Sending must return a message ID.");
        List<EmailMessage> sent = service.getMessages(Mailbox.SENT, "");
        check(sent.size() == sentCount + 1, "Sending must add exactly one Sent entry.");
        EmailMessage delivered = find(sent, sentId);
        check(delivered.getSenderEmail().equals(account.getEmailAddress()),
                "The sender must be the connected account.");
        check(delivered.getRecipientEmails().contains("outside@example.test"),
                "The demo must accept an external recipient.");
        check(delivered.getSubject().equals("Project update")
                        && delivered.getBodyText().equals("The UI is ready."),
                "Sent content must match what the compose form supplied.");
        check(!contains(service.getMessages(Mailbox.INBOX, ""), sentId),
                "Sending to an external recipient should not add it to the demo Inbox.");


        EmailMessage untouched = find(service.getMessages(Mailbox.INBOX, ""), "demo-library");
        EmailMessage suspicious = find(service.getMessages(Mailbox.SUSPECTED, ""), "demo-invoice");
        SpamResult originalScore = service.getSpamResult(suspicious.getGmailMessageId());
```

```
    check(originalScore.isSuspectedSpam() && !originalScore.getReasons().isEmpty(),
            "Suspected messages must provide a classification and reasons.");
    check(contains(service.getMessages(Mailbox.SPAM, ""), "demo-prize"),
            "Gmail Spam fixtures must be available in the separate Spam mailbox.");
    check(!contains(service.getMessages(Mailbox.SUSPECTED, ""), "demo-prize"),
            "Confirmed Spam should not also be queued for local suspicion review.");


    service.markAsSpam(suspicious.getGmailMessageId());
    EmailMessage spam = find(service.getMessages(Mailbox.SPAM, ""), suspicious.getGmailMessageId());
    check(spam.getGmailLabelIds().contains("SPAM") && !spam.getGmailLabelIds().contains("INBOX"),
            "Spam correction must move the message from Inbox to Spam.");
    Set<String> otherLabels = new HashSet<>(suspicious.getGmailLabelIds());
    otherLabels.remove("INBOX");
    otherLabels.remove("SPAM");
    check(spam.getGmailLabelIds().containsAll(otherLabels),
            "Spam correction must preserve unrelated Gmail labels.");


    service.markAsNotSpam(suspicious.getGmailMessageId());
    EmailMessage restored = find(service.getMessages(Mailbox.INBOX, ""), suspicious.getGmailMessageId());
    check(restored.getGmailLabelIds().contains("INBOX") && !restored.getGmailLabelIds().contains("SPAM"),
            "Not-spam correction must return the message to Inbox.");
    check(!service.getSpamResult(suspicious.getGmailMessageId()).isSuspectedSpam(),
            "Not-spam correction must override local suspicion.");
    check(!contains(service.getMessages(Mailbox.SUSPECTED, ""), suspicious.getGmailMessageId()),
            "Corrected messages must leave the suspicion review queue.");
    check(contains(service.getMessages(Mailbox.SUSPECTED, ""), "demo-password"),
            "Correcting one message must not clear another message's classification.");
    check(find(service.getMessages(Mailbox.INBOX, ""), untouched.getGmailMessageId())
                    .getGmailLabelIds().equals(untouched.getGmailLabelIds()),
            "Correcting one message must preserve other messages' labels.");


    expectAppException(() -> service.openMessage("another-account-message"),
            "Opening an unknown or unavailable message must fail.");
    expectAppException(() -> service.getSpamResult("another-account-message"),
            "Spam details for an unavailable message must fail.");
    expectAppException(() -> service.markAsSpam("another-account-message"),
            "Spam corrections for an unavailable message must fail.");
    expectAppException(() -> service.markAsNotSpam("another-account-message"),
            "Not-spam corrections for an unavailable message must fail.");
    expectAppException(() -> service.markAsSpam(sentId),
            "A sent message must not be moved into received Spam.");


    Set<String> inboxBeforeRefresh = ids(service.getMessages(Mailbox.INBOX, ""));
    Set<String> sentBeforeRefresh = ids(service.getMessages(Mailbox.SENT, ""));
    service.refresh();
    List<EmailMessage> refreshedInbox = service.getMessages(Mailbox.INBOX, "");
    check(ids(refreshedInbox).equals(inboxBeforeRefresh)
                    && refreshedInbox.size() == inboxBeforeRefresh.size(),
            "Repeated demo refresh must not lose or duplicate inbox messages.");
    check(ids(service.getMessages(Mailbox.SENT, "")).equals(sentBeforeRefresh),
            "Refresh must preserve sent messages.");
    check(!service.getSpamResult(suspicious.getGmailMessageId()).isSuspectedSpam(),
            "Refresh must preserve a not-spam correction.");


    service.disconnect();
    expectAppException(() -> service.getMessages(Mailbox.INBOX, ""),
            "Disconnect must prevent subsequent mailbox access.");
    expectAppException(() -> service.openMessage(meeting.getGmailMessageId()),
            "Disconnect must prevent message opening.");
    expectAppException(() -> service.sendMessage("outside@example.test", "Subject", "Body"),
            "Disconnect must prevent sending.");
}

private static void checkNewestFirst(List<EmailMessage> messages) {
    for (int index = 1; index < messages.size(); index++) {
        check(!messages.get(index).getReceivedAt().isAfter(messages.get(index - 1).getReceivedAt()),
                "Mailbox messages must be ordered newest first.");
    }
}
```

```
    private static EmailMessage find(List<EmailMessage> messages, String messageId) {
        return messages.stream().filter(message -> message.getGmailMessageId().equals(messageId))
                .findFirst().orElseThrow(() -> new AssertionError("Missing message: " + messageId));
    }

    private static boolean contains(List<EmailMessage> messages, String messageId) {
        return messages.stream().anyMatch(message -> message.getGmailMessageId().equals(messageId));
    }

    private static Set<String> ids(List<EmailMessage> messages) {
        return messages.stream().map(EmailMessage::getGmailMessageId).collect(Collectors.toSet());
    }

    private static void check(boolean condition, String description) {
        assertions++;
        if (!condition) {
            throw new AssertionError(description);
        }
    }

    private static void expectAppException(CheckedAction action, String description) {
        assertions++;
        try {
            action.run();
        } catch (AppException expected) {
            return;
        }
        throw new AssertionError(description);
    }

    private static void expectUnsupported(Runnable action, String description) {
        assertions++;
        try {
            action.run();
        } catch (UnsupportedOperationException expected) {
            return;
        }
        throw new AssertionError(description);
    }

    @FunctionalInterface
    private interface CheckedAction {
        void run() throws AppException;
    }
}
```

src/test/java/ems/check/MemberTwoChecks.java

```
package ems.check;

import ems.exception.AppException;
import ems.gmail.SimpleJson;
import ems.model.Account;
import ems.model.EmailMessage;
import ems.model.Mailbox;
import ems.model.SpamResult;
import ems.service.EmailService;
import ems.service.GmailEmailService;

import java.time.Instant;
import java.util.List;
import java.util.Map;
import java.util.Set;

/**
 * Headless automated test suite verifying Member 2 implementation contracts.
 */
```

```
public final class MemberTwoChecks {

    private static int assertions = 0;

    private MemberTwoChecks() {
    }

    public static void main(String[] args) throws Exception {
        testModelContracts();
        testJsonParser();
        testServiceDisconnectSafety();
        System.out.println("PASS: " + assertions + " Member 2 contract assertions (headless).");
    }

    private static void testModelContracts() {
        // Account contract
        Account account = new Account(101L, "user@example.test");
        check(account.getAccountId() == 101L, "Account accountId should match constructor.");
        check("user@example.test".equals(account.getEmailAddress()), "Account emailAddress should match.");

        // EmailMessage contract
        Instant now = Instant.now();
        EmailMessage msg = new EmailMessage(
                101L,
                "msg-1",
                "sender@example.test",
                List.of("recipient@example.test"),
                "Hello World",
                "This is test body text",
                now,
                Set.of("INBOX", "UNREAD")
        );
        check(msg.getAccountId() == 101L, "EmailMessage accountId should match.");
        check("msg-1".equals(msg.getGmailMessageId()), "EmailMessage messageId should match.");
        check("sender@example.test".equals(msg.getSenderEmail()), "EmailMessage sender should match.");
        check(msg.getRecipientEmails().contains("recipient@example.test"), "EmailMessage recipient should
match.");
        check("Hello World".equals(msg.getSubject()), "EmailMessage subject should match.");
        check("This is test body text".equals(msg.getBodyText()), "EmailMessage body should match.");
        check(msg.getReceivedAt().equals(now), "EmailMessage receivedAt should match.");
        check(msg.getGmailLabelIds().contains("INBOX"), "EmailMessage labels should match.");

        // SpamResult contract
        SpamResult spam = new SpamResult(85, true, List.of("Phishing phrase detected"));
        check(spam.getScore() == 85, "SpamResult score should match.");
        check(spam.isSuspectedSpam(), "SpamResult suspectedSpam should match.");
        check(spam.getReasons().size() == 1, "SpamResult reasons should match.");

        // Mailbox enum
        check(Mailbox.valueOf("INBOX") == Mailbox.INBOX, "Mailbox INBOX should exist.");
        check(Mailbox.valueOf("SENT") == Mailbox.SENT, "Mailbox SENT should exist.");
        check(Mailbox.valueOf("SPAM") == Mailbox.SPAM, "Mailbox SPAM should exist.");
        check(Mailbox.valueOf("SUSPECTED") == Mailbox.SUSPECTED, "Mailbox SUSPECTED should exist.");
    }

    private static void testJsonParser() {
        String json = "{\"id\":\"12345\",\"labelIds\":[\"INBOX\",\"UNREAD\"],\"snippet\":\"Hi
there\",\"valid\":true,\"count\":42}";
        Map<String, Object> parsed = SimpleJson.parseObject(json);
        check("12345".equals(SimpleJson.getString(parsed, "id")), "Parsed string ID should match.");
        check("Hi there".equals(SimpleJson.getString(parsed, "snippet")), "Parsed snippet should match.");
        List<Object> labels = SimpleJson.getList(parsed, "labelIds");
        check(labels.size() == 2, "Parsed labels count should match.");
        check("INBOX".equals(labels.get(0)), "First label should be INBOX.");
        check(Boolean.TRUE.equals(parsed.get("valid")), "Parsed boolean should match.");
        check(Integer.valueOf(42).equals(((Number) parsed.get("count")).intValue()), "Parsed number should
match.");
    }
```

```
    private static void testServiceDisconnectSafety() throws AppException {
        EmailService service = GmailEmailService.createDefault();

        expectAppException(() -> service.getMessages(Mailbox.INBOX, ""),
                "Calls before connecting must fail with AppException.");
        expectAppException(() -> service.openMessage("msg-1"),
                "Opening message before connecting must fail with AppException.");
        expectAppException(() -> service.sendMessage("test@example.com", "Subj", "Body"),
                "Sending message before connecting must fail with AppException.");
        expectAppException(() -> service.getSpamResult("msg-1"),
                "Getting spam result before connecting must fail with AppException.");
        expectAppException(() -> service.markAsSpam("msg-1"),
                "Marking spam before connecting must fail with AppException.");
        expectAppException(() -> service.markAsNotSpam("msg-1"),
                "Marking not spam before connecting must fail with AppException.");
    }

    private static void check(boolean condition, String message) {
        assertions++;
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    private static void expectAppException(ThrowingRunnable action, String message) {
        assertions++;
        try {
            action.run();
            throw new AssertionError("Expected AppException, but none was thrown: " + message);
        } catch (AppException expected) {
            // Success
        } catch (Exception e) {
            throw new AssertionError("Expected AppException, but got: " + e.getClass().getName() + " - " +
message, e);
        }
    }

    @FunctionalInterface
    private interface ThrowingRunnable {
        void run() throws Exception;
    }
}
```

src/test/java/ems/check/MemberTwoContractTest.java

```
package ems.check;

import org.junit.jupiter.api.Test;

/** Runs the teammate's standalone contract checks in every Maven/CI test run. */
class MemberTwoContractTest {
    @Test void memberTwoContract() throws Exception {
        MemberTwoChecks.main(new String[0]);
    }
}
```

src/test/java/ems/check/UiChecks.java

```
package ems.check;

import ems.demo.DemoEmailService;
import ems.exception.AppException;
import ems.model.Mailbox;
import ems.service.EmailService;
import ems.ui.*;
import java.awt.*;
import java.awt.image.BufferedImage;
```

```
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Proxy;
import java.nio.file.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import javax.imageio.ImageIO;
import javax.swing.*;

/** Exercises real Swing actions headlessly, with slow/failing service responses. */
public final class UiChecks {
    private static MailAppPanel panel;
    private static int assertions;
    private static Path previewDirectory;

    public static void main(String[] args) throws Exception {
        if (args.length > 0 && args[0].equals("--render")) {
            previewDirectory = Path.of("build", "previews");
            Files.createDirectories(previewDirectory);
        }
        DemoEmailService demo = new DemoEmailService();
        AtomicBoolean onEdt = new AtomicBoolean();
        AtomicBoolean failRefresh = new AtomicBoolean(true);
        AtomicBoolean failOpen = new AtomicBoolean();
        AtomicReference<CountDownLatch> refreshGate = new AtomicReference<>();
        EmailService service = (EmailService) Proxy.newProxyInstance(EmailService.class.getClassLoader(),
                new Class<?>[] {EmailService.class}, (proxy, method, arguments) -> {
                    if (SwingUtilities.isEventDispatchThread()) onEdt.set(true);
                    if (method.getName().equals("refresh") || method.getName().equals("refreshRecent")) {
                        if (failRefresh.get()) throw new AppException("Test connection unavailable. Try
again.");
                        CountDownLatch gate = refreshGate.get();
                        if (gate != null && !gate.await(5, TimeUnit.SECONDS)) throw new AppException("Test timed
out.");
                    }
                    if (method.getName().equals("openMessage") && failOpen.get()) {
                        throw new AppException("Test message unavailable.");
                    }
                    try { return method.invoke(demo, arguments); }
                    catch (InvocationTargetException exception) { throw exception.getCause(); }
                });

        edt(() -> {
            UiTheme.install(); panel = new MailAppPanel(service, true);
            panel.setSize(1360, 860); layout(panel); render(panel, "connect.png");
        });
        click("connectButton");
        edt(() -> {
            check(button("refreshButton").isEnabled(), "Refresh retry available after initial sync failure");
            check(label("statusLabel").getText().contains("Test connection"), "Useful connection failure
shown");
        });
        check(demo.getMessages(Mailbox.INBOX, "").size() == 8,
                "Initial refresh failure preserves the signed-in account and cached messages");
        failRefresh.set(false); click("refreshButton");
        edt(() -> {
            check(table().getRowCount() == 8, "Connected Inbox shown");
            check(button("composeButton").isEnabled(), "Compose enabled after connect");
            layout(panel); render(panel, "inbox.png");
            table().setRowSelectionInterval(0, 0);
        });
        awaitIdle();
        edt(() -> {
            check(find(panel, "messageBody") != null, "Opening row displays body");
            check(!((MailTableModel) table().getModel()).isUnread(0), "Opened row updates read state");
            layout(panel); render(panel, "reader.png");
        });
        failOpen.set(true);
        edt(() -> table().setRowSelectionInterval(1, 1)); awaitIdle();
        edt(() -> {
            check(find(panel, "markSpamButton") == null, "Failed open cannot correct previous message");
            check(find(panel, "messageBody") == null, "Previous body cleared on failed open");
            check(label("statusLabel").getText().contains("unavailable"), "Message failure shown");
```

```
        });
        failOpen.set(false);
        edt(() -> table().setRowSelectionInterval(1, 1)); awaitIdle();
        edt(() -> check(find(panel, "messageBody") != null,
                "The same row can be retried after an opening failure"));
        click("suspectedButton");
        edt(() -> {
            check(table().getRowCount() == 2, "Suspected view shows local flags");
            table().setRowSelectionInterval(0, 0);
        }); awaitIdle();
        edt(() -> {
            check(find(panel, "spamReasons") != null, "Rule reasons displayed");
            layout(panel); render(panel, "suspected.png");
        });
        click("notSpamButton");
        edt(() -> check(table().getRowCount() == 1, "Not spam correction removes local review item"));
        click("spamButton");
        edt(() -> check(table().getRowCount() == 2, "Gmail Spam stays separate"));
        click("sentButton");
        edt(() -> {
            check(table().getRowCount() == 2, "Sent view works");
            check(table().getColumnName(0).equals("TO"), "Sent identifies recipient");
            table().setRowSelectionInterval(0, 0);
        }); awaitIdle();
        edt(() -> check(find(panel, "markSpamButton") == null, "Sent cannot trigger spam correction"));
        click("inboxButton");
        edt(() -> field("searchField").setText("no-such-message-1928")); click("searchButton");
        edt(() -> check(table().getRowCount() == 0, "Empty search result clears list"));
        edt(() -> field("searchField").setText("project")); click("searchButton");
        edt(() -> check(table().getRowCount() > 0, "Search recovers from empty results"));
        CountDownLatch gate = new CountDownLatch(1); refreshGate.set(gate);
        edt(() -> button("refreshButton").doClick());
        edt(() -> {
            check(panel.isBusy(), "Long-running refresh tracked");
            check(!button("disconnectButton").isEnabled(), "Session cannot change mid-operation");
            check(!button("refreshButton").isEnabled(), "Duplicate refresh disabled");
        });
        gate.countDown(); awaitIdle(); refreshGate.set(null);
        failRefresh.set(true); click("refreshButton"); failRefresh.set(false);
        edt(() -> {
            check(button("refreshButton").isEnabled(), "Refresh recovers after failure");
            check(label("statusLabel").getText().contains("unavailable"), "Refresh error shown");
            check(table().getRowCount() > 0, "Cached messages remain on refresh error");
        });
        composeChecks();
        readerLayoutChecks(demo);
        click("disconnectButton");
        edt(() -> {
            check(button("connectButton").isEnabled(), "Disconnect returns to connect screen");
            check(!button("composeButton").isEnabled(), "Disconnected compose disabled");
        });
        check(!onEdt.get(), "Every service operation stayed off the Swing EDT");
        System.out.println("PASS: " + assertions + " Swing interaction assertions (headless).");
        if (previewDirectory != null) System.out.println("Rendered previews to " +
previewDirectory.toAbsolutePath());
    }

    private static void composeChecks() throws Exception {
        AtomicReference<ComposePanel.Draft> submitted = new AtomicReference<>();
        AtomicInteger submissions = new AtomicInteger();
        AtomicInteger cancellations = new AtomicInteger();
        edt(() -> {
            ComposePanel form = new ComposePanel("student@example.test", true, draft -> {
                submitted.set(draft); submissions.incrementAndGet();
            }, cancellations::incrementAndGet);
            form.setSize(660, 620);
            JButton send = (JButton) find(form, "sendButton");
            send.doClick(); check(submissions.get() == 0, "Blank compose cannot send");
            ((JTextField) find(form, "composeTo")).setText("maya@campus.test");
            ((JTextField) find(form, "composeSubject")).setText("Project review");
            ((JTextArea) find(form, "composeBody")).setText("Hi Maya,\n\nThe mailbox screens are ready to
review.\n\nThanks!");
            form.showError(null);
```

```
            layout(form); render(form, "compose.png"); send.doClick();
            check(submissions.get() == 1 && submitted.get().subject().equals("Project review"), "Validated
compose delivers draft");
            form.setBusy(true);
            form.getActionMap().get("sendDraft").actionPerformed(null);
            form.getActionMap().get("cancelCompose").actionPerformed(null);
            check(submissions.get() == 1 && cancellations.get() == 0, "Busy compose blocks keyboard duplicates
and dismissal");
            form.setBusy(false); form.showError("Delivery failed. You can retry.");
            check(((JTextArea) find(form, "composeBody")).getText().contains("ready"), "Draft survives send
failure");
            ((JButton) find(form, "cancelComposeButton")).doClick();
            check(cancellations.get() == 1, "Compose cancel callback works");
        });
    }

    private static void readerLayoutChecks(DemoEmailService demo) throws Exception {
        String paragraph = "A longer paragraph must wrap at the reader width without losing its last line. ";
        String message = paragraph.repeat(95) + "\n\nFINAL SIGNATURE — visible at the end.";
        demo.sendMessage("student@example.test", "Long reader layout regression ".repeat(5), message);
        click("sentButton");
        edt(() -> table().setRowSelectionInterval(0, 0)); awaitIdle();
        edt(() -> {
            for (int width : new int[] {1360, 1100}) {
                panel.setSize(width, 740);
                layout(panel);
                JTextArea body = (JTextArea) find(panel, "messageBody");
                JScrollPane scroll = (JScrollPane) SwingUtilities.getAncestorOfClass(JScrollPane.class, body);
                try {
                    java.awt.geom.Rectangle2D end = body.modelToView2D(body.getDocument().getLength());
                    check(end != null && end.getMaxY() <= body.getHeight() - body.getInsets().bottom,
                            "All wrapped body lines have allocated height at width " + width);
                    Rectangle ending = SwingUtilities.convertRectangle(body, end.getBounds(),
scroll.getViewport().getView());
                    scroll.getVerticalScrollBar().setValue(scroll.getVerticalScrollBar().getMaximum());
                    layout(scroll);
                    JViewport viewport = scroll.getViewport();
                    check(viewport.getViewPosition().y + viewport.getExtentSize().height >= ending.getMaxY(),
                            "The final body line is reachable by scrolling at width " + width);
                    check(scroll.getVerticalScrollBar().getMaximum() > viewport.getExtentSize().height,
                            "Long mail produces a vertical scroll range at width " + width);
                } catch (javax.swing.text.BadLocationException exception) {
                    throw new AssertionError(exception);
                }
            }
            panel.setSize(1360, 860); layout(panel);
        });
    }

    private static void click(String name) throws Exception {
        edt(() -> button(name).doClick()); awaitIdle();
    }

    private static void awaitIdle() throws Exception {
        long deadline = System.nanoTime() + TimeUnit.SECONDS.toNanos(8);
        AtomicBoolean working = new AtomicBoolean();
        do {
            edt(() -> working.set(panel.isBusy()));
            if (!working.get()) return;
            Thread.sleep(20);
        } while (System.nanoTime() < deadline);
        throw new AssertionError("UI worker did not finish");
    }

    private static JButton button(String name) { return (JButton) required(name); }
    private static JLabel label(String name) { return (JLabel) required(name); }
    private static JTextField field(String name) { return (JTextField) required(name); }
    private static JTable table() { return (JTable) required("messageTable"); }
    private static Component required(String name) {
        Component value = find(panel, name);
        if (value == null) throw new AssertionError("Missing component: " + name);
        return value;
```

```
    }

    private static Component find(Component component, String name) {
        if (name.equals(component.getName())) return component;
        if (component instanceof Container container) {
            for (Component child : container.getComponents()) {
                Component match = find(child, name);
                if (match != null) return match;
            }
        }
        return null;
    }

    private static void layout(Container container) {
        container.doLayout();
        for (Component child : container.getComponents()) {
            if (child instanceof Container nested) layout(nested);
        }
    }

    private static void render(JComponent component, String filename) {
        if (previewDirectory == null) return;
        BufferedImage image = new BufferedImage(component.getWidth(), component.getHeight(),
BufferedImage.TYPE_INT_RGB);
        Graphics2D graphics = image.createGraphics();
        graphics.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        graphics.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        component.paint(graphics); graphics.dispose();
        try { ImageIO.write(image, "png", previewDirectory.resolve(filename).toFile()); }
        catch (java.io.IOException exception) { throw new RuntimeException(exception); }
    }

    private static void check(boolean condition, String description) {
        assertions++;
        if (!condition) throw new AssertionError(description);
    }

    private static void edt(Runnable work) throws Exception {
        if (SwingUtilities.isEventDispatchThread()) work.run();
        else SwingUtilities.invokeAndWait(work);
    }
}
```

src/test/java/ems/config/AppConfigTest.java

```
package ems.config;

import ems.exception.AppException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Map;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;
import static org.junit.jupiter.api.Assertions.*;

class AppConfigTest {
    @TempDir Path home;

    @Test void defaultsWorkWithoutSavingSecrets() throws Exception {
        AppConfig config = AppConfig.load(home, Map.of());
        assertTrue(config.getDatabaseUrl().contains("email_management"));
        assertEquals(home.resolve("config/credentials.json"), config.getGmailCredentialsPath());
        assertFalse(Files.exists(config.getConfigFile()));
    }

    @Test void environmentOverridesLocalFileIncludingEmptyPassword() throws Exception {
        Files.createDirectories(home.resolve("config"));
```

```
        Files.writeString(home.resolve("config/app.properties"),
                "db.user=local\ndb.password=local-secret\ngmail.credentials=custom/client.json\n");
        AppConfig config = AppConfig.load(home, Map.of("DB_USER", "environment", "DB_PASSWORD", ""));
        assertEquals("environment", config.getDatabaseUser());
        assertEquals("", config.getDatabasePassword());
        assertEquals(home.resolve("custom/client.json"), config.getGmailCredentialsPath());
    }

    @Test void invalidPropertiesBecomeActionableError() throws Exception {
        Files.createDirectories(home.resolve("config"));
        Files.writeString(home.resolve("config/app.properties"), "db.password=\\uBAD!\n");
        AppException exception = assertThrows(AppException.class, () -> AppConfig.load(home, Map.of()));
        assertFalse(exception.getMessage().contains("BAD!"));
        assertTrue(exception.getMessage().contains("config/app.properties"));
    }

    @Test void teammateCredentialFilenamesWorkWithDeterministicPrecedence() throws Exception {
        Files.writeString(home.resolve("client_secret.json"), "{}");
        assertEquals(home.resolve("client_secret.json"),
                AppConfig.load(home, Map.of()).getGmailCredentialsPath());
        Files.createDirectories(home.resolve("config"));
        Files.writeString(home.resolve("config/credentials.json"), "{}");
        assertEquals(home.resolve("config/credentials.json"),
                AppConfig.load(home, Map.of()).getGmailCredentialsPath());
        // A missing explicit path must report that path rather than use another client's grant.
        assertEquals(home.resolve("missing.json"), AppConfig.load(home,
                Map.of("GMAIL_CREDENTIALS", "missing.json")).getGmailCredentialsPath());
    }
}
```

src/test/java/ems/gmail/GmailApiClientTest.java

```
package ems.gmail;

import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import ems.exception.AppException;
import ems.model.EmailMessage;
import jakarta.mail.Session;
import jakarta.mail.internet.MimeMessage;
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.Base64;
import java.util.List;
import java.util.Properties;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class GmailApiClientTest {
    private HttpServer server;
    private GmailApiClient gateway;
    private final FakeTokens tokens = new FakeTokens();
    private final AtomicReference<Handler> handler = new AtomicReference<>();

    @BeforeEach void start() throws Exception {
        server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
        server.createContext("/", exchange -> {
            try { handler.get().handle(exchange); }
            catch (Throwable failure) { reply(exchange, 500, "{}"); }
```

```
        });
        server.start();
        gateway = new GmailApiClient(URI.create("http://127.0.0.1:" + server.getAddress().getPort() + "/"),
                HttpClient.newHttpClient(), tokens, milliseconds -> { });
    }

    @AfterEach void stop() { server.stop(0); }

    @Test void connectsToProfileWithoutExposingCredentials() throws Exception {
        handler.set(exchange -> {
            assertEquals("Bearer initial", exchange.getRequestHeaders().getFirst("Authorization"));
            assertEquals("/profile", exchange.getRequestURI().getPath());
            reply(exchange, 200, "{\"emailAddress\":\"alice@gmail.com\"}");
        });
        assertEquals("alice@gmail.com", gateway.connect());
        assertTrue(tokens.authorized);
        gateway.disconnect();
        assertTrue(tokens.disconnected);
    }

    @Test void loadsAllPagesDeduplicatesIdsAndSkipsOnlyDeletedMessages() throws Exception {
        AtomicInteger rawReads = new AtomicInteger();
        handler.set(exchange -> {
            String path = exchange.getRequestURI().getPath();
            if (path.equals("/messages")) {
                var query = GmailAuth.parseQuery(exchange.getRequestURI().getRawQuery());
                assertEquals("true", query.get("includeSpamTrash"));
                assertEquals("-in:trash -in:drafts", query.get("q"));
                reply(exchange, 200, query.containsKey("pageToken")
                        ? "{\"messages\":[{\"id\":\"first\"},{\"id\":\"deleted\"},{\"id\":\"second\"}]}"
                        : "{\"messages\":[{\"id\":\"first\"}],\"nextPageToken\":\"page 2\"}");
            } else if (path.endsWith("deleted")) reply(exchange, 404, "{}");
            else {
                rawReads.incrementAndGet();
                reply(exchange, 200, rawMessage("Subject: Hello\r\nContent-Type: text/plain; charset=UTF-
8\r\n\r\nReal body"));
            }
        });
        List<EmailMessage> messages = gateway.fetchMessages(42);
        assertEquals(List.of("first", "second"),
messages.stream().map(EmailMessage::getGmailMessageId).toList());
        assertEquals(2, rawReads.get());
        assertEquals(42, messages.get(0).getAccountId());
        assertEquals(Instant.ofEpochMilli(1700000000000L), messages.get(0).getReceivedAt());
        assertTrue(messages.get(0).getGmailLabelIds().contains("INBOX"));
    }

    @Test void doesNotReturnPartialSnapshotWhenLaterFetchFails() {
        handler.set(exchange -> {
            if (exchange.getRequestURI().getPath().equals("/messages")) {
                reply(exchange, 200, "{\"messages\":[{\"id\":\"first\"},{\"id\":\"second\"}]}");
            } else if (exchange.getRequestURI().getPath().endsWith("first")) {
                reply(exchange, 200, rawMessage("Subject: Fine\r\n\r\nFirst"));
            } else reply(exchange, 503, "{}");
        });
        assertThrows(AppException.class, () -> gateway.fetchMessages(1));
    }

    @Test void startupReadsBoundedRecentWindowsFromAllThreeFolders() throws Exception {
        java.util.Set<String> folders = new java.util.HashSet<>();
        AtomicInteger reads = new AtomicInteger();
        handler.set(exchange -> {
            if (exchange.getRequestURI().getPath().equals("/messages")) {
                var query = GmailAuth.parseQuery(exchange.getRequestURI().getRawQuery());
                assertEquals("6", query.get("maxResults"));
                assertFalse(query.containsKey("pageToken"));
                folders.add(query.get("labelIds"));
                reply(exchange, 200, "{\"messages\":[{\"id\":\"shared\"}],\"nextPageToken\":\"older-mail\"}");
            } else {
                reads.incrementAndGet();
                reply(exchange, 200, rawMessage("Subject: Recent\r\n\r\nBody"));
```

```
            }
        });
        assertEquals(1, gateway.fetchRecentMessages(42, 6).size());
        assertEquals(java.util.Set.of("INBOX", "SENT", "SPAM"), folders);
        assertEquals(1, reads.get(), "Messages in multiple folders are read once");
    }

    @Test void refreshesOnceOnUnauthorizedResponse() throws Exception {
        AtomicInteger calls = new AtomicInteger();
        handler.set(exchange -> {
            if (calls.incrementAndGet() == 1) reply(exchange, 401, "{}");
            else {
                assertEquals("Bearer refreshed", exchange.getRequestHeaders().getFirst("Authorization"));
                reply(exchange, 200, "{\"emailAddress\":\"alice@gmail.com\"}");
            }
        });
        assertEquals("alice@gmail.com", gateway.connect());
        assertEquals(1, tokens.refreshes);
        assertEquals(2, calls.get());
    }

    @Test void neverRetriesAnUnconfirmedSend() throws Exception {
        AtomicInteger calls = new AtomicInteger();
        handler.set(exchange -> {
            calls.incrementAndGet();
            reply(exchange, 503, "{\"error\":\"private provider information\"}");
        });
        AppException failure = assertThrows(AppException.class,
                () -> gateway.sendMessage("alice@gmail.com", "bob@example.org", "Hi", "Text"));
        assertEquals(1, calls.get());
        assertFalse(failure.getMessage().contains("private provider"));
        assertTrue(failure.getMessage().contains("Check Gmail Sent before retrying"));
    }

    @Test void sendsUnicodeMimeAndUpdatesReadAndSpamLabels() throws Exception {
        AtomicReference<JsonObject> sent = new AtomicReference<>();
        AtomicReference<JsonObject> modification = new AtomicReference<>();
        handler.set(exchange -> {
            JsonObject json = JsonParser.parseString(new String(exchange.getRequestBody().readAllBytes(),
                    StandardCharsets.UTF_8)).getAsJsonObject();
            if (exchange.getRequestURI().getPath().endsWith("/send")) sent.set(json);
            else modification.set(json);
            reply(exchange, 200, "{\"id\":\"newmail\"}");
        });
        assertEquals("newmail", gateway.sendMessage("alice@gmail.com", "bob@example.org", "Meeting – தமிழ்",
"Hello\nவணக்கம்"));
        MimeMessage mime = new MimeMessage(Session.getInstance(new Properties()),
                new ByteArrayInputStream(Base64.getUrlDecoder().decode(sent.get().get("raw").getAsString())));
        assertEquals("Meeting – தமிழ்", mime.getSubject());
        assertEquals("Hello\r\nவணக்கம்", mime.getContent().toString().replace("\r\n", "\n").replace("\n",
"\r\n"));
        assertEquals("bob@example.org", mime.getAllRecipients()[0].toString());
        gateway.setRead("newmail");
        assertTrue(modification.get().getAsJsonArray("removeLabelIds").toString().contains("UNREAD"));
        gateway.setSpam("newmail", true);
        assertTrue(modification.get().getAsJsonArray("addLabelIds").toString().contains("SPAM"));
        assertTrue(modification.get().getAsJsonArray("removeLabelIds").toString().contains("INBOX"));
        gateway.setSpam("newmail", false);
        assertTrue(modification.get().getAsJsonArray("addLabelIds").toString().contains("INBOX"));
        assertTrue(modification.get().getAsJsonArray("removeLabelIds").toString().contains("SPAM"));
    }

    @Test void rejectsHeaderInjectionAndPathInjectionBeforeNetwork() {
        assertThrows(AppException.class,
                () -> gateway.sendMessage("alice@gmail.com", "bob@example.org\r\nBcc: victim@example.org", "Hi",
"Text"));
        assertThrows(AppException.class,
                () -> gateway.sendMessage("alice@gmail.com", "bob@example.org", "Hi\r\nBcc: victim@example.org",
"Text"));
        assertThrows(AppException.class, () -> gateway.getMessage(1, "../profile"));
    }
```

```
    @Test void choosesPlainAlternativeAndExcludesAttachments() throws Exception {
        String raw = "From: Alice <alice@gmail.com>\r\nTo: bob@example.org\r\n"
                + "Subject: MIME test\r\nMIME-Version: 1.0\r\nContent-Type: multipart/mixed;
boundary=mix\r\n\r\n"
                + "--mix\r\nContent-Type: multipart/alternative; boundary=alt\r\n\r\n"
                + "--alt\r\nContent-Type: text/plain; charset=UTF-8\r\n\r\nPlain body\r\n"
                + "--alt\r\nContent-Type: text/html; charset=UTF-8\r\n\r\n<p>HTML duplicate</p>\r\n--alt--\r\n"
                + "--mix\r\nContent-Type: text/plain\r\nContent-Disposition: attachment;
filename=private.txt\r\n\r\n"
                + "ATTACHMENT SHOULD NOT APPEAR\r\n--mix--\r\n";
        EmailMessage message = MimeMessages.parse(1, "id", encode(raw), Instant.EPOCH, java.util.Set.of());
        assertEquals("alice@gmail.com", message.getSenderEmail());
        assertEquals(List.of("bob@example.org"), message.getRecipientEmails());
        assertEquals("Plain body", message.getBodyText().strip());
    }

    @Test void convertsHtmlToInertTextAndDecodesEntities() throws Exception {
        String raw = "Content-Type: text/html; charset=UTF-8\r\n\r\n"
                + "<html><body><p>Hello &amp; welcome</p><script>alert('bad')</script>"
                + "<img src='https://tracker.invalid/pixel'><p>Second line</p></body></html>";
        String body = MimeMessages.parse(1, "id", encode(raw), Instant.EPOCH, java.util.Set.of()).getBodyText();
        assertTrue(body.contains("Hello & welcome"));
        assertTrue(body.contains("Second line"));
        assertFalse(body.contains("alert"));
        assertFalse(body.contains("tracker"));
    }

    @Test void repeatedPageTokenAbortsInsteadOfLoopingForever() {
        handler.set(exchange -> reply(exchange, 200, "{\"nextPageToken\":\"same\"}"));
        assertThrows(AppException.class, () -> gateway.fetchMessages(1));
    }

    @Test void providerErrorsAreActionableAndDoNotEchoResponse() {
        handler.set(exchange -> reply(exchange, 403, "{\"error\":\"secret-token-value\"}"));
        AppException failure = assertThrows(AppException.class, gateway::connect);
        assertTrue(failure.getMessage().contains("Enable the Gmail API"));
        assertFalse(failure.getMessage().contains("secret-token-value"));
    }

    @Test void retriesRateLimitedReadsWithoutRefreshingAuthorization() throws Exception {
        AtomicInteger calls = new AtomicInteger();
        java.util.ArrayList<Long> delays = new java.util.ArrayList<>();
        gateway = new GmailApiClient(URI.create("http://127.0.0.1:" + server.getAddress().getPort() + "/"),
                HttpClient.newHttpClient(), tokens, delays::add);
        handler.set(exchange -> {
            if (calls.incrementAndGet() <= 2) reply(exchange, 403,
                    "{\"error\":{\"errors\":[{\"reason\":\"userRateLimitExceeded\"}]}}");
            else reply(exchange, 200, "{\"emailAddress\":\"alice@gmail.com\"}");
        });
        assertEquals("alice@gmail.com", gateway.connect());
        assertEquals(List.of(1000L, 2000L), delays);
        assertEquals(0, tokens.refreshes);
    }

    @Test void exhaustedQuotaIsNotMisreportedAsMissingGmailSetup() {
        AtomicInteger calls = new AtomicInteger();
        handler.set(exchange -> {
            calls.incrementAndGet();
            reply(exchange, 403, "{\"error\":{\"errors\":[{\"reason\":\"dailyLimitExceeded\"}],"
                    + "\"message\":\"private-account-information\"}}");
        });
        AppException error = assertThrows(AppException.class, gateway::connect);
        assertTrue(error.getMessage().contains("quota"));
        assertFalse(error.getMessage().contains("private-account"));
        assertEquals(1, calls.get());
    }

    @Test void rateLimitedSendIsNotAutomaticallyReplayed() {
        AtomicInteger calls = new AtomicInteger();
```

```
        handler.set(exchange -> {
            calls.incrementAndGet();
            reply(exchange, 403, "{\"error\":{\"errors\":[{\"reason\":\"userRateLimitExceeded\"}]}}");
        });
        assertThrows(AppException.class, () -> gateway.sendMessage("alice@gmail.com", "bob@example.org", "Hi",
"Text"));
        assertEquals(1, calls.get());
    }

    @Test void persistentRateLimitStopsAfterBoundedBackoffAndKeepsAuthorization() {
        java.util.ArrayList<Long> delays = new java.util.ArrayList<>();
        gateway = new GmailApiClient(URI.create("http://127.0.0.1:" + server.getAddress().getPort() + "/"),
                HttpClient.newHttpClient(), tokens, delays::add);
        handler.set(exchange -> reply(exchange, 403,
                "{\"error\":{\"errors\":[{\"reason\":\"rateLimitExceeded\"}]}}"));
        AppException error = assertThrows(AppException.class, gateway::connect);
        assertEquals(List.of(1000L, 2000L, 4000L, 8000L, 16000L, 32000L), delays);
        assertTrue(error.getMessage().contains("temporarily limited"));
        assertFalse(error.getMessage().contains("Enable the Gmail API"));
        assertFalse(tokens.disconnected);
        assertEquals(0, tokens.refreshes);
    }

    private static String rawMessage(String content) {
        JsonObject json = new JsonObject();
        json.addProperty("raw", encode(content));
        json.addProperty("internalDate", "1700000000000");
        json.add("labelIds", JsonParser.parseString("[\"INBOX\",\"UNREAD\"]"));
        return json.toString();
    }

    private static String encode(String raw) {
        return Base64.getUrlEncoder().withoutPadding().encodeToString(raw.getBytes(StandardCharsets.UTF_8));
    }

    private static void reply(HttpExchange exchange, int status, String value) throws IOException {
        byte[] bytes = value.getBytes(StandardCharsets.UTF_8);
        exchange.sendResponseHeaders(status, bytes.length);
        try (var output = exchange.getResponseBody()) { output.write(bytes); }
    }

    @FunctionalInterface private interface Handler { void handle(HttpExchange exchange) throws Exception; }

    private static final class FakeTokens implements AccessTokenProvider {
        boolean authorized;
        boolean disconnected;
        int refreshes;
        @Override public void authorize() { authorized = true; }
        @Override public String accessToken() { return refreshes == 0 ? "initial" : "refreshed"; }
        @Override public String refreshAccessToken() { refreshes++; return "refreshed"; }
        @Override public void disconnect() { disconnected = true; }
    }
}
```

src/test/java/ems/gmail/GmailAuthTest.java

```
package ems.gmail;

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import ems.exception.AppException;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
```

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.MessageDigest;
import java.time.Duration;
import java.util.Base64;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;
import static org.junit.jupiter.api.Assertions.*;


class GmailAuthTest {
    @TempDir Path temp;
    private HttpServer server;
    private URI tokenUri;
    private final HttpClient http = HttpClient.newHttpClient();
    private Path credentials;
    private Path tokens;
    private final AtomicReference<TokenHandler> tokenHandler = new AtomicReference<>();


    @BeforeEach void start() throws Exception {
        credentials = temp.resolve("credentials.json");
        tokens = temp.resolve("tokens");
        Files.writeString(credentials, "{\"installed\":{\"client_id\":\"desktop-test-
client\",\"client_secret\":\"test-secret\"}}");
        server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
        server.createContext("/token", exchange -> {
            try { tokenHandler.get().handle(exchange); }
            catch (Throwable failure) { reply(exchange, 500, "{}"); }
        });
        server.start();
        tokenUri = URI.create("http://127.0.0.1:" + server.getAddress().getPort() + "/token");
    }

    @AfterEach void stop() { server.stop(0); }


    @Test void authorizationUsesPkceRejectsWrongStateAndReusesProtectedCache() throws Exception {
        AtomicReference<Map<String, String>> authorization = new AtomicReference<>();
        AtomicInteger browserCalls = new AtomicInteger();
        tokenHandler.set(exchange -> {
            Map<String, String> form = GmailAuth.parseQuery(
                    new String(exchange.getRequestBody().readAllBytes(), StandardCharsets.UTF_8));
            assertEquals("authorization_code", form.get("grant_type"));
            assertEquals("auth-code", form.get("code"));
            assertEquals("desktop-test-client", form.get("client_id"));
            String challenge =
Base64.getUrlEncoder().withoutPadding().encodeToString(MessageDigest.getInstance("SHA-256")
                    .digest(form.get("code_verifier").getBytes(StandardCharsets.US_ASCII)));
            assertEquals(authorization.get().get("code_challenge"), challenge);
            reply(exchange, 200, "{\"access_token\":\"initial-access\",\"refresh_token\":\"refresh-
value\",\"expires_in\":3600}");
        });
        GmailAuth client = client(uri -> {
            browserCalls.incrementAndGet();
            Map<String, String> query = GmailAuth.parseQuery(uri.getRawQuery());
            authorization.set(query);
            assertEquals("S256", query.get("code_challenge_method"));
            assertEquals(GmailAuth.SCOPE, query.get("scope"));
            assertEquals("offline", query.get("access_type"));
            assertTrue(query.get("prompt").contains("select_account"));
            URI callback = URI.create(query.get("redirect_uri"));
            assertEquals("127.0.0.1", callback.getHost());
            assertEquals(400, callback(callback, "state=wrong&code=attack").statusCode());
            assertEquals(200, callback(callback, "state=" + query.get("state") + "&code=auth-
code").statusCode());
        });
        client.authorize();
        assertEquals("initial-access", client.accessToken());
        assertTrue(Files.exists(tokens.resolve("gmail-token.json")));
```

```
        GmailAuth restarted = client(uri -> fail("Valid stored token should not reopen browser"));
        restarted.authorize();
        assertEquals("initial-access", restarted.accessToken());
        assertEquals(1, browserCalls.get());
        tokenHandler.set(exchange -> {
            Map<String, String> form = GmailAuth.parseQuery(
                    new String(exchange.getRequestBody().readAllBytes(), StandardCharsets.UTF_8));
            assertEquals("refresh_token", form.get("grant_type"));
            assertEquals("refresh-value", form.get("refresh_token"));
            reply(exchange, 200, "{\"access_token\":\"renewed-access\",\"expires_in\":3600}");
        });
        assertEquals("renewed-access", restarted.refreshAccessToken());
        assertTrue(Files.readString(tokens.resolve("gmail-token.json")).contains("refresh-value"));
        restarted.disconnect();
        assertFalse(Files.exists(tokens.resolve("gmail-token.json")));
        assertThrows(AppException.class, restarted::accessToken);
    }

    @Test void deniedAuthorizationDoesNotSaveTokens() {
        GmailAuth client = client(uri -> {
            Map<String, String> query = GmailAuth.parseQuery(uri.getRawQuery());
            callback(URI.create(query.get("redirect_uri")), "state=" + query.get("state") +
"&error=access_denied");
        });
        AppException failure = assertThrows(AppException.class, client::authorize);
        assertTrue(failure.getMessage().contains("denied"));
        assertFalse(Files.exists(tokens.resolve("gmail-token.json")));
    }

    @Test void missingCredentialsExplainsDesktopSetup() throws Exception {
        Files.delete(credentials);
        AppException failure = assertThrows(AppException.class, () -> client(uri -> fail()).authorize());
        assertTrue(failure.getMessage().contains("Desktop app"));
        assertTrue(failure.getMessage().contains("README.md"));
    }

    @Test void rejectsWebClientCredentials() throws Exception {
        Files.writeString(credentials, "{\"web\":{\"client_id\":\"not-desktop\"}}");
        AppException failure = assertThrows(AppException.class, () -> client(uri -> fail()).authorize());
        assertTrue(failure.getMessage().contains("Desktop app"));
    }

    @Test void timesOutAndClosesLoopbackListener() {
        AtomicReference<URI> callback = new AtomicReference<>();
        GmailAuth client = new GmailAuth(credentials, tokens, http, tokenUri, tokenUri,
                uri -> callback.set(URI.create(GmailAuth.parseQuery(uri.getRawQuery()).get("redirect_uri"))),
                Duration.ofMillis(100));
        AppException failure = assertThrows(AppException.class, client::authorize);
        assertTrue(failure.getMessage().contains("timed out"));
        assertThrows(IOException.class, () -> http.send(HttpRequest.newBuilder(callback.get()).GET().build(),
                HttpResponse.BodyHandlers.ofString()));
    }

    @Test void revokedRefreshTokenIsForgottenWithoutEchoingProviderPayload() throws Exception {
        Files.createDirectories(tokens);
        Files.writeString(tokens.resolve("gmail-token.json"), "{\"client_id\":\"desktop-test-client\","
                + "\"access_token\":\"old\",\"refresh_token\":\"revoked\",\"expires_at\":0}");
        tokenHandler.set(exchange -> reply(exchange, 400, "{\"error\":\"invalid_grant\",\"private\":\"secret-
value\"}"));
        GmailAuth client = client(uri -> {
            assertFalse(Files.exists(tokens.resolve("gmail-token.json")));
            Map<String, String> query = GmailAuth.parseQuery(uri.getRawQuery());
            callback(URI.create(query.get("redirect_uri")), "state=" + query.get("state") +
"&error=access_denied");
        });
        AppException failure = assertThrows(AppException.class, client::authorize);
        assertFalse(failure.getMessage().contains("secret-value"));
        assertFalse(Files.exists(tokens.resolve("gmail-token.json")));
    }

    private GmailAuth client(GmailAuth.BrowserOpener browser) {
```

```
        return new GmailAuth(credentials, tokens, http, tokenUri, tokenUri, browser, Duration.ofSeconds(3));
    }

    private HttpResponse<String> callback(URI uri, String query) throws Exception {
        return http.send(HttpRequest.newBuilder(URI.create(uri + "?" + query)).GET().build(),
                HttpResponse.BodyHandlers.ofString());
    }

    private static void reply(HttpExchange exchange, int status, String text) throws IOException {
        byte[] bytes = text.getBytes(StandardCharsets.UTF_8);
        exchange.sendResponseHeaders(status, bytes.length);
        try (var out = exchange.getResponseBody()) { out.write(bytes); }
    }

    @FunctionalInterface private interface TokenHandler { void handle(HttpExchange exchange) throws Exception; }
}
```

src/test/java/ems/gmail/SimpleJsonTest.java

```
package ems.gmail;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class SimpleJsonTest {
    @Test void rejectsIncompleteOrTrailingJsonRatherThanReturningPartialData() {
        for (String invalid : new String[] {"{\"id\":\"one\"", "[1", "\"open", "{} trailing", "{id:1}"}) {
            assertThrows(RuntimeException.class, () -> SimpleJson.parse(invalid), invalid);
        }
    }

    @Test void retainsTeammateHelperContractForUnicodeAndNumbers() {
        var object = SimpleJson.parseObject("{\"subject\":\"Hello
\\u20ac\",\"count\":42,\"nested\":{\"ok\":true}}");
        assertEquals("Hello €", SimpleJson.getString(object, "subject"));
        assertEquals(42L, object.get("count"));
        assertEquals(Boolean.TRUE, SimpleJson.getObject(object, "nested").get("ok"));
        assertEquals("a\\n\\\"b", SimpleJson.escapeString("a\n\"b"));
    }
}
```

src/test/java/ems/service/GmailEmailServiceTest.java

```
package ems.service;

import ems.exception.AppException;
import ems.gmail.GmailGateway;
import ems.model.*;
import ems.storage.*;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Proxy;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.time.Instant;
import java.util.*;
import java.util.concurrent.atomic.AtomicBoolean;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

/** Service orchestration uses a real JDBC store and a deterministic remote mailbox. */
class GmailEmailServiceTest {
    private JdbcMailStore store;
    private FakeGmail gmail;
    private GmailEmailService service;
```

```
    @BeforeEach void setup() {
        String url = "jdbc:h2:mem:service-" + UUID.randomUUID() + ";MODE=MySQL;DB_CLOSE_DELAY=-
1;DATABASE_TO_LOWER=TRUE";
        store = new JdbcMailStore(() -> DriverManager.getConnection(url));
        gmail = new FakeGmail();
        gmail.messages.put("received", message(0, "received", Set.of("INBOX", "UNREAD")));
        service = new GmailEmailService(gmail, store);
    }

    @Test void receiveSearchReadSendAndCorrectThroughRealJdbc() throws Exception {
        service.connectGmail(); service.refresh();
        assertEquals(1, service.getMessages(Mailbox.INBOX, "prize").size());
        assertEquals(0, gmail.modifications, "Automatic spam analysis must not change Gmail labels");
        assertTrue(service.getSpamResult("received").isSuspectedSpam());
        assertFalse(service.openMessage("received").getGmailLabelIds().contains("UNREAD"));
        assertEquals(1, gmail.modifications);
        String sentId = service.sendMessage("friend@external.test", "Hello", "A real outbound message
contract.");
        assertEquals(sentId, service.getMessages(Mailbox.SENT, "Hello").get(0).getGmailMessageId());
        service.markAsNotSpam("received"); service.refresh();
        assertFalse(service.getSpamResult("received").isSuspectedSpam(), "User correction survives analysis on
refresh");
        service.markAsSpam("received");
        assertEquals(1, service.getMessages(Mailbox.SPAM, "").size());
        assertEquals(0, service.getMessages(Mailbox.INBOX, "").size());
    }

    @Test void networkFailurePreservesCacheAndSuccessfulSnapshotRemovesDeletedMessages() throws Exception {
        service.connectGmail(); service.refresh(); gmail.failFetch = true;
        assertThrows(AppException.class, service::refresh);
        assertEquals(1, service.getMessages(Mailbox.INBOX, "").size());
        gmail.failFetch = false; gmail.messages.clear(); service.refresh();
        assertTrue(service.getMessages(Mailbox.INBOX, "").isEmpty());
    }

    @Test void remoteFailuresDoNotSaveUnconfirmedCorrections() throws Exception {
        service.connectGmail(); service.refresh(); gmail.failModification = true;
        assertThrows(AppException.class, () -> service.markAsNotSpam("received"));
        assertTrue(service.getSpamResult("received").isSuspectedSpam());
        assertTrue(service.getMessages(Mailbox.SPAM, "").isEmpty());
    }

    @Test void refreshingSampleReplacesOnlyTheLocalSampleSnapshot() throws Exception {
        service.connectGmail(); service.refresh();
        gmail.messages.clear();
        gmail.messages.put("newer", message(0, "newer", Set.of("INBOX")));
        service.refreshRecent();
        assertEquals(1, service.getMessages(Mailbox.INBOX, "").size());
        assertFalse(store.findMessage(service.connectGmail().getAccountId(), "received").isPresent());
        assertEquals(0, gmail.modifications, "Replacing the local sample must not change Gmail labels");
    }

    @Test void sampleImportsNeverExceedEighteenMessages() throws Exception {
        for (int i = 0; i < 30; i++) gmail.messages.put("sample" + i, message(0, "sample" + i,
Set.of("INBOX")));
        service.connectGmail(); service.refresh();
        assertEquals(18, service.getMessages(Mailbox.INBOX, "").size());
        service.refreshRecent();
        assertEquals(18, service.getMessages(Mailbox.INBOX, "").size());
    }

    @Test void databaseFailureAfterDeliveryReturnsSuccessAndWarningWithoutAnotherSend() throws Exception {
        AtomicBoolean failWrites = new AtomicBoolean();
        MailStore unreliable = (MailStore) Proxy.newProxyInstance(MailStore.class.getClassLoader(),
                new Class<?>[] {MailStore.class}, (proxy, method, arguments) -> {
                    if (failWrites.get() && method.getName().equals("saveMessages")) throw new
SQLException("Test unavailable");
                    try { return method.invoke(store, arguments); }
                    catch (InvocationTargetException exception) { throw exception.getCause(); }
                });
        service = new GmailEmailService(gmail, unreliable);
```

```
        service.connectGmail(); service.refresh(); failWrites.set(true);
        assertEquals("sent1", service.sendMessage("friend@external.test", "Hello", "Sent once."));
        assertEquals(1, gmail.sends);
        assertTrue(service.getLastWarning().contains("was sent"));
    }

    @Test void missingMessagesAndAccountSwitchCannotAccessAnotherCachedAccount() throws Exception {
        Account first = service.connectGmail(); service.refresh();
        assertThrows(AppException.class, () -> service.openMessage("someone-elses-message"));
        assertEquals(0, gmail.modifications);
        service.disconnect();
        assertThrows(AppException.class, () -> service.getMessages(Mailbox.INBOX, ""));
        gmail.email = "second@external.test"; gmail.messages.clear();
        Account second = service.connectGmail(); service.refresh();
        assertNotEquals(first.getAccountId(), second.getAccountId());
        assertTrue(service.getMessages(Mailbox.INBOX, "").isEmpty());
        assertThrows(AppException.class, () -> service.openMessage("received"));
        assertTrue(store.findMessage(first.getAccountId(), "received").isPresent());
    }

    @Test void validatesRecipientAndHeaderInjectionBeforeSending() throws Exception {
        service.connectGmail();
        assertThrows(AppException.class, () -> service.sendMessage("a@test.test,b@test.test", "Hi", "body"));
        assertThrows(AppException.class, () -> service.sendMessage("a@test.test\r\nBcc: b@test.test", "Hi",
"body"));
        assertThrows(AppException.class, () -> service.sendMessage("a@test.test", "Hi\r\nBcc: b@test.test",
"body"));
        assertEquals(0, gmail.sends);
    }

    private static EmailMessage message(long accountId, String id, Set<String> labels) {
        return new EmailMessage(accountId, id, "sender@external.test", List.of("student@example.test"),
                "URGENT: Claim your prize", "Verify your account https://1.test https://2.test https://3.test
https://4.test https://5.test",
                Instant.parse("2026-10-20T06:00:00Z"), labels);
    }

    private static final class FakeGmail implements GmailGateway {
        String email = "student@example.test";
        final Map<String, EmailMessage> messages = new LinkedHashMap<>();
        int modifications;
        int sends;
        boolean failFetch;
        boolean failModification;
        @Override public String connect() { return email; }
        @Override public void disconnect() { }
        @Override public List<EmailMessage> fetchMessages(long accountId) throws AppException {
            if (failFetch) throw new AppException("Test network unavailable");
            return messages.values().stream().map(m -> copy(m, accountId, m.getGmailLabelIds())).toList();
        }
        @Override public EmailMessage getMessage(long accountId, String id) throws AppException {
            if (!messages.containsKey(id)) throw new AppException("Test message missing");
            EmailMessage message = messages.get(id);
            return copy(message, accountId, message.getGmailLabelIds());
        }
        @Override public String sendMessage(String sender, String recipient, String subject, String body) {
            String id = "sent" + ++sends;
            messages.put(id, new EmailMessage(0, id, sender, List.of(recipient), subject, body, Instant.now(),
Set.of("SENT")));
            return id;
        }
        @Override public void setRead(String id) throws AppException {
            checkModification();
            Set<String> labels = new HashSet<>(messages.get(id).getGmailLabelIds()); labels.remove("UNREAD");
            messages.put(id, copy(messages.get(id), 0, labels));
        }
        @Override public void setSpam(String id, boolean spam) throws AppException {
            checkModification();
            Set<String> labels = new HashSet<>(messages.get(id).getGmailLabelIds());
            if (spam) { labels.add("SPAM"); labels.remove("INBOX"); }
            else { labels.remove("SPAM"); labels.add("INBOX"); }
            messages.put(id, copy(messages.get(id), 0, labels));
```

```
        }
        private void checkModification() throws AppException {
            if (failModification) throw new AppException("Test change rejected");
            modifications++;
        }
        private static EmailMessage copy(EmailMessage message, long accountId, Set<String> labels) {
            return new EmailMessage(accountId, message.getGmailMessageId(), message.getSenderEmail(),
message.getRecipientEmails(),
                    message.getSubject(), message.getBodyText(), message.getReceivedAt(), labels);
        }
    }
}
```

src/test/java/ems/service/RealMySqlTest.java

```
package ems.service;

import ems.config.AppConfig;
import ems.exception.AppException;
import ems.gmail.GmailGateway;
import ems.model.*;
import ems.storage.JdbcMailStore;
import java.sql.*;
import java.time.Instant;
import java.util.*;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.condition.EnabledIfSystemProperty;
import static org.junit.jupiter.api.Assertions.*;

/** Opt-in real MySQL round trip; Gmail remains a fake so no real email is sent. */
@EnabledIfSystemProperty(named = "maildesk.mysqlTest", matches = "true")
class RealMySqlTest {
    @Test void realDatabasePersistsTheCompleteServiceFlow() throws Exception {
        AppConfig config = AppConfig.load();
        JdbcMailStore store = new JdbcMailStore(() -> connect(config));
        String email = "maildesk-check-" + UUID.randomUUID() + "@example.test";
        TestGateway gateway = new TestGateway(email);
        GmailEmailService service = new GmailEmailService(gateway, store);
        Account account = null;
        try {
            account = service.connectGmail();
            service.refresh();
            assertEquals(1, service.getMessages(Mailbox.INBOX, "café").size());
            assertTrue(service.getSpamResult("incoming").isSuspectedSpam());
            assertFalse(service.openMessage("incoming").getGmailLabelIds().contains("UNREAD"));
            assertEquals("outgoing", service.sendMessage("friend@external.test", "Project update café", "MySQL
integration fixture."));
            assertEquals(1, service.getMessages(Mailbox.SENT, "café").size());
            service.markAsNotSpam("incoming"); service.refresh();
            assertFalse(service.getSpamResult("incoming").isSuspectedSpam());
            service.markAsSpam("incoming");
            assertEquals(1, service.getMessages(Mailbox.SPAM, "").size());
            assertTrue(service.getMessages(Mailbox.SUSPECTED, "").isEmpty());
            JdbcMailStore reopened = new JdbcMailStore(() -> connect(config));
            assertEquals(Set.of("SPAM"), reopened.findMessage(account.getAccountId(),
"incoming").orElseThrow().getGmailLabelIds());
            assertTrue(reopened.getSpamResult(account.getAccountId(), "incoming").isSuspectedSpam());
            gateway.messages.remove("incoming"); service.refresh();
            assertTrue(service.getMessages(Mailbox.SPAM, "").isEmpty());
        } finally {
            service.disconnect();
            if (account != null) removeFixture(config, account, email);
        }
    }

    private static Connection connect(AppConfig config) throws SQLException {
        return DriverManager.getConnection(config.getDatabaseUrl(), config.getDatabaseUser(),
config.getDatabasePassword());
    }
```

```
    private static void removeFixture(AppConfig config, Account account, String expectedEmail) throws
SQLException {
        // Every cleanup statement is scoped to this newly created random fixture account.
        try (Connection connection = connect(config)) {
            connection.setAutoCommit(false);
            try {
                for (String table : List.of("spam_feedback", "blocked_senders", "messages")) {
                    try (PreparedStatement statement = connection.prepareStatement("DELETE FROM " + table + "
WHERE account_id=?")) {
                        statement.setLong(1, account.getAccountId()); statement.executeUpdate();
                    }
                }
                try (PreparedStatement statement = connection.prepareStatement("DELETE FROM accounts WHERE
account_id=? AND email_address=?")) {
                    statement.setLong(1, account.getAccountId()); statement.setString(2, expectedEmail);
statement.executeUpdate();
                }
                connection.commit();
            } catch (SQLException exception) { connection.rollback(); throw exception; }
        }
    }

    private static final class TestGateway implements GmailGateway {
        private final String email;
        private final Map<String, EmailMessage> messages = new LinkedHashMap<>();
        TestGateway(String email) {
            this.email = email;
            messages.put("incoming", new EmailMessage(0, "incoming", "sender@external.test", List.of(email),
                    "URGENT: Claim your prize café", "Verify your account https://a.test https://b.test
https://c.test https://d.test https://e.test",
                    Instant.parse("2026-10-20T06:00:00.123456Z"), Set.of("INBOX", "UNREAD")));
        }
        @Override public String connect() { return email; }
        @Override public void disconnect() { }
        @Override public List<EmailMessage> fetchMessages(long accountId) {
            return messages.values().stream().map(m -> copy(m, accountId, m.getGmailLabelIds())).toList();
        }
        @Override public EmailMessage getMessage(long accountId, String id) throws AppException {
            EmailMessage message = messages.get(id);
            if (message == null) throw new AppException("Fixture missing");
            return copy(message, accountId, message.getGmailLabelIds());
        }
        @Override public String sendMessage(String sender, String recipient, String subject, String body) {
            messages.put("outgoing", new EmailMessage(0, "outgoing", sender, List.of(recipient), subject, body,
Instant.now(), Set.of("SENT")));
            return "outgoing";
        }
        @Override public void setRead(String id) {
            Set<String> labels = new HashSet<>(messages.get(id).getGmailLabelIds()); labels.remove("UNREAD");
            messages.put(id, copy(messages.get(id), 0, labels));
        }
        @Override public void setSpam(String id, boolean spam) {
            Set<String> labels = new HashSet<>(messages.get(id).getGmailLabelIds());
            if (spam) { labels.add("SPAM"); labels.remove("INBOX"); } else { labels.remove("SPAM");
labels.add("INBOX"); }
            messages.put(id, copy(messages.get(id), 0, labels));
        }
        private static EmailMessage copy(EmailMessage message, long accountId, Set<String> labels) {
            return new EmailMessage(accountId, message.getGmailMessageId(), message.getSenderEmail(),
message.getRecipientEmails(),
                    message.getSubject(), message.getBodyText(), message.getReceivedAt(), labels);
        }
    }
}
```

src/test/java/ems/storage/JdbcMailStoreTest.java

```
package ems.storage;

import ems.model.Account;
import ems.model.EmailMessage;
```

```
import ems.model.Mailbox;
import ems.model.SpamResult;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.Instant;
import java.util.List;
import java.util.Set;
import java.util.UUID;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class JdbcMailStoreTest {
    private JdbcMailStore store;
    private String url;
    private Account alice;
    private Account bob;

    @BeforeEach void prepareDatabase() throws SQLException {
        url = "jdbc:h2:mem:" + UUID.randomUUID() + ";MODE=MySQL;DATABASE_TO_LOWER=TRUE;DB_CLOSE_DELAY=-1";
        store = new JdbcMailStore(() -> DriverManager.getConnection(url));
        store.initialize();
        alice = store.findOrCreateAccount("alice@example.com");
        bob = store.findOrCreateAccount("bob@example.com");
    }

    private EmailMessage message(Account account, String id, String subject, String body, Set<String> labels) {
        return new EmailMessage(account.getAccountId(), id, "sender@example.com",
List.of("recipient@example.com"),
                subject, body, Instant.parse("2026-10-20T10:15:00.123456Z"), labels);
    }

    private EmailMessage normal(Account account, String id) {
        return message(account, id, "Meeting notes", "Project update", Set.of("INBOX", "UNREAD"));
    }

    private EmailMessage suspicious(Account account, String id) {
        return message(account, id, "URGENT: Claim your prize", "You have won! Verify your account now.",
Set.of("INBOX"));
    }

    @Test void initializerIsIdempotentAndAccountAddressesAreNormalized() throws SQLException {
        store.initialize();
        Account same = store.findOrCreateAccount("  ALICE@EXAMPLE.COM ");
        assertEquals(alice.getAccountId(), same.getAccountId());
        assertEquals("alice@example.com", same.getEmailAddress());
        assertNotEquals(alice.getAccountId(), bob.getAccountId());
    }

    @Test void refreshUpsertHasNoDuplicatesAndReplacesRecipientsAndLabels() throws SQLException {
        EmailMessage original = normal(alice, "same");
        store.saveMessages(alice.getAccountId(), List.of(original));
        store.saveMessages(alice.getAccountId(), List.of(original));
        EmailMessage moved = new EmailMessage(alice.getAccountId(), "same", "sender@example.com",
                List.of("new@example.com", "new@example.com"), "Changed", "New body", original.getReceivedAt(),
Set.of("SPAM"));
        store.saveMessages(alice.getAccountId(), List.of(moved));

        assertEquals(1, store.findAll(alice.getAccountId()).size());
        EmailMessage saved = store.findMessage(alice.getAccountId(), "same").orElseThrow();
        assertEquals(List.of("new@example.com"), saved.getRecipientEmails());
        assertEquals(Set.of("SPAM"), saved.getGmailLabelIds());
        assertEquals("Changed", saved.getSubject());
        assertTrue(store.getMessages(alice.getAccountId(), Mailbox.INBOX, "").isEmpty());
        assertEquals(1, store.getMessages(alice.getAccountId(), Mailbox.SPAM, "").size());
        assertTrue(readColumn(alice.getAccountId(), "same"));
    }
```

```
    @Test void timestampRoundTripsAsAnInstant() throws SQLException {
        EmailMessage original = normal(alice, "time");
        store.saveMessages(alice.getAccountId(), List.of(original));
        assertEquals(original.getReceivedAt(), store.findMessage(alice.getAccountId(),
"time").orElseThrow().getReceivedAt());
    }

    @Test void repeatedRecipientsAcrossHeadersAreDeduplicatedIgnoringCase() throws SQLException {
        EmailMessage original = normal(alice, "recipients");
        EmailMessage repeated = new EmailMessage(alice.getAccountId(), original.getGmailMessageId(),
                original.getSenderEmail(), List.of("Person@example.com", "person@EXAMPLE.COM"),
                original.getSubject(), original.getBodyText(), original.getReceivedAt(),
original.getGmailLabelIds());
        store.saveMessages(alice.getAccountId(), List.of(repeated));
        assertEquals(List.of("Person@example.com"), store.findMessage(alice.getAccountId(), "recipients")
                .orElseThrow().getRecipientEmails());
    }

    @Test void incomingUnreadLabelControlsReadStatusAndLegacyMethodKeepsItConsistent() throws SQLException {
        store.saveMessages(alice.getAccountId(), List.of(normal(alice, "unread")));
        assertFalse(readColumn(alice.getAccountId(), "unread"));
        store.updateReadStatus(alice.getAccountId(), "unread", true);
        assertTrue(readColumn(alice.getAccountId(), "unread"));
        assertFalse(store.findMessage(alice.getAccountId(),
"unread").orElseThrow().getGmailLabelIds().contains("UNREAD"));
        store.updateReadStatus(alice.getAccountId(), "unread", false);
        store.updateReadStatus(alice.getAccountId(), "unread", false);
        assertFalse(readColumn(alice.getAccountId(), "unread"));
        assertTrue(store.findMessage(alice.getAccountId(),
"unread").orElseThrow().getGmailLabelIds().contains("UNREAD"));
    }

    @Test void accountIsolationAppliesToEveryReadWriteAndFeedback() throws SQLException {
        store.saveMessages(alice.getAccountId(), List.of(normal(alice, "same-id")));
        store.saveMessages(bob.getAccountId(), List.of(suspicious(bob, "same-id")));
        store.saveFeedback(alice.getAccountId(), "same-id", false);
        assertEquals("Meeting notes", store.findMessage(alice.getAccountId(), "same-
id").orElseThrow().getSubject());
        assertTrue(store.getSpamResult(bob.getAccountId(), "same-id").isSuspectedSpam());
        assertFalse(store.getSpamResult(alice.getAccountId(), "same-id").isSuspectedSpam());
        assertThrows(SQLException.class, () -> store.saveMessages(alice.getAccountId(), List.of(normal(bob,
"bad"))));
        store.deleteMessages(alice.getAccountId(), List.of("same-id"));
        assertTrue(store.findMessage(alice.getAccountId(), "same-id").isEmpty());
        assertTrue(store.findMessage(bob.getAccountId(), "same-id").isPresent());
        assertThrows(SQLException.class, () -> store.saveFeedback(alice.getAccountId(), "missing", true));
    }

    @Test void accountWithNoMessagesReturnsEmptyLists() throws SQLException {
        for (Mailbox mailbox : Mailbox.values()) assertTrue(store.getMessages(bob.getAccountId(), mailbox,
"").isEmpty());
        assertTrue(store.findMessage(bob.getAccountId(), "missing").isEmpty());
    }

    @Test void localSpamAndGmailSpamAreDistinct() throws SQLException {
        store.saveMessages(alice.getAccountId(), List.of(suspicious(alice, "local"),
                message(alice, "gmail", "Meeting notes", "Project update", Set.of("SPAM"))));
        assertEquals(List.of("local"), store.getMessages(alice.getAccountId(), Mailbox.SUSPECTED,
"").stream().map(EmailMessage::getGmailMessageId).toList());
        assertEquals(List.of("gmail"), store.getMessages(alice.getAccountId(), Mailbox.SPAM,
"").stream().map(EmailMessage::getGmailMessageId).toList());
        assertFalse(store.getSpamResult(alice.getAccountId(), "gmail").isSuspectedSpam());
    }

    @Test void suspectedReviewExcludesGmailSpamAndAllSentMailEvenWhenClassifierFlagsThem() throws SQLException {
        String subject = "URGENT: Claim your prize";
        String body = "You have won! Verify your account now.";
        store.saveMessages(alice.getAccountId(), List.of(suspicious(alice, "review"),
                message(alice, "gmail-spam", subject, body, Set.of("SPAM")),
```

```
                message(alice, "sent", subject, body, Set.of("SENT")),
                message(alice, "self-sent", subject, body, Set.of("SENT", "INBOX"))));
        assertTrue(store.getSpamResult(alice.getAccountId(), "gmail-spam").isSuspectedSpam());
        assertTrue(store.getSpamResult(alice.getAccountId(), "sent").isSuspectedSpam());
        assertTrue(store.getSpamResult(alice.getAccountId(), "self-sent").isSuspectedSpam());
        assertEquals(List.of("review"), store.getMessages(alice.getAccountId(), Mailbox.SUSPECTED, "")
                .stream().map(EmailMessage::getGmailMessageId).toList());
        assertEquals(1, store.getMessages(alice.getAccountId(), Mailbox.SPAM, "").size());
        assertEquals(2, store.getMessages(alice.getAccountId(), Mailbox.SENT, "").size());
        store.saveFeedback(alice.getAccountId(), "gmail-spam", true);
        assertEquals(1, store.getMessages(alice.getAccountId(), Mailbox.SUSPECTED, "").size());
    }

    @Test void userFeedbackSurvivesRefreshAndNewStoreInstance() throws SQLException {
        EmailMessage mail = suspicious(alice, "corrected");
        store.saveMessages(alice.getAccountId(), List.of(mail));
        store.saveFeedback(alice.getAccountId(), "corrected", false);
        store.saveMessages(alice.getAccountId(), List.of(mail));
        store = new JdbcMailStore(() -> DriverManager.getConnection(url));
        assertFalse(store.getSpamResult(alice.getAccountId(), "corrected").isSuspectedSpam());
        assertTrue(store.getMessages(alice.getAccountId(), Mailbox.SUSPECTED, "").isEmpty());
        store.saveFeedback(alice.getAccountId(), "corrected", true);
        store.saveMessages(alice.getAccountId(), List.of(normal(alice, "corrected")));
        assertTrue(store.getSpamResult(alice.getAccountId(), "corrected").isSuspectedSpam());
        assertEquals(1, store.getMessages(alice.getAccountId(), Mailbox.SUSPECTED, "").size());
    }

    @Test void snapshotPrunesRemovedMessagesAtomicallyAndFeedbackSurvivesRedownload() throws SQLException {
        store.saveMessages(alice.getAccountId(), List.of(normal(alice, "keep"), suspicious(alice, "remove")));
        store.saveMessages(bob.getAccountId(), List.of(normal(bob, "remove")));
        store.saveFeedback(alice.getAccountId(), "remove", false);
        store.replaceSnapshot(alice.getAccountId(), List.of(normal(alice, "keep")));
        assertTrue(store.findMessage(alice.getAccountId(), "remove").isEmpty());
        assertTrue(store.findMessage(bob.getAccountId(), "remove").isPresent());
        store.saveMessages(alice.getAccountId(), List.of(suspicious(alice, "remove")));
        assertFalse(store.getSpamResult(alice.getAccountId(), "remove").isSuspectedSpam());
        store.replaceSnapshot(alice.getAccountId(), List.of());
        assertTrue(store.findAll(alice.getAccountId()).isEmpty());
        assertFalse(store.findAll(bob.getAccountId()).isEmpty());
    }

    @Test void invalidBatchRollsBackContentRecipientsLabelsScoresAndSnapshotPruning() throws SQLException {
        store.saveMessages(alice.getAccountId(), List.of(normal(alice, "keep"), normal(alice, "existing")));
        EmailMessage invalid = new EmailMessage(alice.getAccountId(), "invalid", "sender@example.com",
                List.of("x".repeat(321)), "subject", "body", Instant.now(), Set.of("INBOX"));
        assertThrows(SQLException.class, () -> store.replaceSnapshot(alice.getAccountId(),
List.of(suspicious(alice, "existing"), invalid)));
        assertEquals("Meeting notes", store.findMessage(alice.getAccountId(),
"existing").orElseThrow().getSubject());
        assertEquals(Set.of("INBOX", "UNREAD"), store.findMessage(alice.getAccountId(),
"existing").orElseThrow().getGmailLabelIds());
        assertFalse(store.getSpamResult(alice.getAccountId(), "existing").isSuspectedSpam());
        assertTrue(store.findMessage(alice.getAccountId(), "invalid").isEmpty());
        assertTrue(store.findMessage(alice.getAccountId(), "keep").isPresent());
    }

    @Test void searchIsCaseInsensitiveMatchesAllFieldsAndEscapesLikeWildcards() throws SQLException {
        store.saveMessages(alice.getAccountId(), List.of(
                message(alice, "literal", "100%_done!", "Project update", Set.of("INBOX")),
                normal(alice, "other")));
        assertEquals(1, store.getMessages(alice.getAccountId(), Mailbox.INBOX, "%_done!").size());
        assertEquals(2, store.getMessages(alice.getAccountId(), Mailbox.INBOX, "PROJECT UPDATE").size());
        assertEquals(2, store.getMessages(alice.getAccountId(), Mailbox.INBOX, "RECIPIENT@EXAMPLE.COM").size());
        assertEquals(2, store.getMessages(alice.getAccountId(), Mailbox.INBOX, "SENDER@EXAMPLE.COM").size());
        assertTrue(store.getMessages(alice.getAccountId(), Mailbox.INBOX, "' OR 1=1 --").isEmpty());
    }

    @Test void sentAndTrashViewsFollowGmailLabels() throws SQLException {
        store.saveMessages(alice.getAccountId(), List.of(normal(alice, "inbox"),
                message(alice, "sent", "Sent", "Hello", Set.of("SENT")),
                message(alice, "trashed", "Trash", "Hello", Set.of("INBOX", "TRASH"))));
```

```
        assertEquals(1, store.getMessages(alice.getAccountId(), Mailbox.INBOX, "").size());
        assertEquals(1, store.getMessages(alice.getAccountId(), Mailbox.SENT, "").size());
        assertEquals("sent", store.getMessages(alice.getAccountId(), Mailbox.SENT,
"").get(0).getGmailMessageId());
    }

    @Test void blockedSendersAreNormalizedAndScopedToTheirAccount() throws SQLException {
        store.setSenderBlocked(alice.getAccountId(), " SENDER@EXAMPLE.COM ", true);
        store.setSenderBlocked(alice.getAccountId(), "sender@example.com", true);
        assertTrue(store.isSenderBlocked(alice.getAccountId(), "sender@example.com"));
        assertFalse(store.isSenderBlocked(bob.getAccountId(), "sender@example.com"));
        store.saveMessages(alice.getAccountId(), List.of(normal(alice, "blocked")));
        store.saveMessages(bob.getAccountId(), List.of(normal(bob, "normal")));
        assertTrue(store.getSpamResult(alice.getAccountId(), "blocked").isSuspectedSpam());
        assertFalse(store.getSpamResult(bob.getAccountId(), "normal").isSuspectedSpam());
        store.setSenderBlocked(alice.getAccountId(), "sender@example.com", false);
        store.saveMessages(alice.getAccountId(), List.of(normal(alice, "blocked")));
        assertFalse(store.getSpamResult(alice.getAccountId(), "blocked").isSuspectedSpam());
    }

    @Test void reasonJsonRoundTripsPunctuationControlCharactersAndUnicode() throws SQLException {
        store.saveMessages(alice.getAccountId(), List.of(normal(alice, "json")));
        List<String> reasons = List.of("A quote: \" and slash: \\", "line\nbreak\tand\u0001", "தமிழ் 🎉");
        store.saveSpamResult(alice.getAccountId(), "json", new SpamResult(60, true, reasons));
        assertEquals(reasons, store.getSpamResult(alice.getAccountId(), "json").getReasons());
    }

    @Test void messageBodiesMayExceedMysqlTextLimit() throws SQLException {
        String body = "Long message body. ".repeat(10_000);
        store.saveMessages(alice.getAccountId(), List.of(message(alice, "large", "Large", body,
Set.of("INBOX"))));
        assertEquals(body, store.findMessage(alice.getAccountId(), "large").orElseThrow().getBodyText());
    }

    private boolean readColumn(long accountId, String id) throws SQLException {
        try (Connection connection = DriverManager.getConnection(url);
                PreparedStatement query = connection.prepareStatement("SELECT is_read FROM messages WHERE
account_id=? AND gmail_message_id=?")) {
            query.setLong(1, accountId);
            query.setString(2, id);
            try (ResultSet rows = query.executeQuery()) { assertTrue(rows.next()); return rows.getBoolean(1); }
        }
    }
}
```

APPENDIX A.4 — DATABASE SCHEMA

sql/schema.sql

```
-- Email Management: MySQL 8.x cache schema. No OAuth tokens or passwords are stored here.
-- Use a dedicated database user with permission to create and read/write these tables.
CREATE DATABASE IF NOT EXISTS email_management CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE email_management;

CREATE TABLE IF NOT EXISTS accounts (
            account_id BIGINT AUTO_INCREMENT PRIMARY KEY,
            email_address VARCHAR(254) NOT NULL UNIQUE
        );

CREATE TABLE IF NOT EXISTS messages (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            sender_email VARCHAR(320) NOT NULL,
            subject TEXT NOT NULL,
            body_text LONGTEXT NOT NULL,
            received_at DATETIME(6) NOT NULL,
            is_read BOOLEAN NOT NULL DEFAULT TRUE,
            PRIMARY KEY (account_id, gmail_message_id),
```

```
            FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        );

CREATE TABLE IF NOT EXISTS message_recipients (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            recipient_email VARCHAR(320) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id, recipient_email),
            FOREIGN KEY (account_id, gmail_message_id) REFERENCES messages(account_id, gmail_message_id) ON
DELETE CASCADE
        );

CREATE TABLE IF NOT EXISTS message_labels (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            gmail_label_id VARCHAR(128) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id, gmail_label_id),
            FOREIGN KEY (account_id, gmail_message_id) REFERENCES messages(account_id, gmail_message_id) ON
DELETE CASCADE
        );

CREATE TABLE IF NOT EXISTS spam_results (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            score INT NOT NULL,
            suspected_spam BOOLEAN NOT NULL,
            reasons TEXT NOT NULL,
            analyzed_at DATETIME(6) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id),
            FOREIGN KEY (account_id, gmail_message_id) REFERENCES messages(account_id, gmail_message_id) ON
DELETE CASCADE
        );

CREATE TABLE IF NOT EXISTS spam_feedback (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            user_marked_spam BOOLEAN NOT NULL,
            corrected_at DATETIME(6) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id),
            FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        );

CREATE TABLE IF NOT EXISTS blocked_senders (
            account_id BIGINT NOT NULL,
            sender_email VARCHAR(320) NOT NULL,
            PRIMARY KEY (account_id, sender_email),
            FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        );
```

APPENDIX A.5 — WINDOWS BUILD AND LAUNCH SCRIPTS

scripts/build.ps1

```
[CmdletBinding()]
param([switch]$SkipTests)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
. (Join-Path $PSScriptRoot 'initialize-java.ps1')
$projectRoot = Split-Path -Parent $PSScriptRoot
Push-Location -LiteralPath $projectRoot
try {
    $mavenArguments = @('--batch-mode', '--no-transfer-progress', 'package')
    if ($SkipTests) { $mavenArguments += '-DskipTests' }
    & (Join-Path $projectRoot 'mvnw.cmd') @mavenArguments
    if ($LASTEXITCODE -ne 0) { throw "Build failed (exit code $LASTEXITCODE)." }
    Write-Host "Application: $(Join-Path $projectRoot 'target/maildesk.jar')"
} finally { Pop-Location }
```

scripts/check.ps1

```
[CmdletBinding()]
param([switch]$Render, [switch]$MySql)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
. (Join-Path $PSScriptRoot 'initialize-java.ps1')
$projectRoot = Split-Path -Parent $PSScriptRoot
Push-Location -LiteralPath $projectRoot
try {
    $mavenArguments = @('--batch-mode', '--no-transfer-progress', 'test')
    if ($Render) { $mavenArguments += '-Prender' }
    if ($MySql) { $mavenArguments += '-Dmaildesk.mysqlTest=true' }
    & (Join-Path $projectRoot 'mvnw.cmd') @mavenArguments
    if ($LASTEXITCODE -ne 0) { throw "Checks failed (exit code $LASTEXITCODE)." }
} finally { Pop-Location }
```

scripts/initialize-java.ps1

```
# Shared version of Member 2's JDK discovery for build, check, app, and demo launchers.
if (-not (Get-Command javac -CommandType Application -ErrorAction SilentlyContinue)) {
    $env:Path = [Environment]::GetEnvironmentVariable('Path', 'User') + ';' +
        [Environment]::GetEnvironmentVariable('Path', 'Machine') + ';' + $env:Path
}
$maildeskJdkCandidates = @(
    $env:JAVA_HOME
    (Join-Path $env:LOCALAPPDATA 'Programs/Eclipse Adoptium/jdk-17')
)
foreach ($maildeskJdk in $maildeskJdkCandidates) {
    if ($maildeskJdk -and (Test-Path -LiteralPath (Join-Path $maildeskJdk 'bin/javac.exe'))) {
        $env:JAVA_HOME = $maildeskJdk
        $env:Path = (Join-Path $maildeskJdk 'bin') + ';' + $env:Path
        break
    }
}
if (-not (Get-Command javac -CommandType Application -ErrorAction SilentlyContinue)) {
    throw 'Install JDK 17 or newer and set JAVA_HOME to its installation folder.'
}
```

scripts/package-windows.ps1

```
[CmdletBinding()]
param(
    [string]$JdkPath = $env:JAVA_HOME,
    [string]$OutputDirectory,
    [switch]$NoBuild,
    [switch]$UseLocalConfig
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
$projectRoot = Split-Path -Parent $PSScriptRoot
if ([Environment]::OSVersion.Platform -ne [PlatformID]::Win32NT) {
    throw 'Build the Windows package on Windows with an x64 JDK 17 or newer.'
}
if (-not $JdkPath) {
    $packager = Get-Command jpackage -CommandType Application -ErrorAction SilentlyContinue
    if ($packager) { $JdkPath = Split-Path -Parent (Split-Path -Parent $packager.Source) }
}
if (-not $JdkPath) { throw 'Pass -JdkPath with an x64 JDK containing jpackage, jlink, and jdeps.' }
$JdkPath = (Resolve-Path -LiteralPath $JdkPath).Path
foreach ($tool in @('java', 'javac', 'jpackage', 'jlink', 'jdeps')) {
    if (-not (Test-Path -LiteralPath (Join-Path $JdkPath "bin/$tool.exe"))) {
        throw "The selected JDK is missing $tool.exe."
    }
```

```
}
$env:JAVA_HOME = $JdkPath
$env:Path = (Join-Path $JdkPath 'bin') + ';' + $env:Path
if (-not $NoBuild) { & (Join-Path $PSScriptRoot 'build.ps1') }
$applicationJar = Join-Path $projectRoot 'target/maildesk.jar'
if (-not (Test-Path -LiteralPath $applicationJar)) { throw 'Build the application first.' }
if (-not $OutputDirectory) { $OutputDirectory = Join-Path $projectRoot 'package' }
$OutputDirectory = [IO.Path]::GetFullPath($OutputDirectory)
$applicationDirectory = Join-Path $OutputDirectory 'Maildesk'
$archive = Join-Path $OutputDirectory 'Maildesk-Windows-x64.zip'
if ((Test-Path -LiteralPath $applicationDirectory) -or (Test-Path -LiteralPath $archive)) {
    throw 'The package already exists. Choose another -OutputDirectory to preserve existing local settings.'
}

$stage = Join-Path $projectRoot ('build/windows-package-' + [Guid]::NewGuid().ToString('N'))
$inputDirectory = Join-Path $stage 'input'
$runtimeDirectory = Join-Path $stage 'runtime'
New-Item -ItemType Directory -Path $inputDirectory, $OutputDirectory -Force | Out-Null
Copy-Item -LiteralPath $applicationJar -Destination (Join-Path $inputDirectory 'maildesk.jar')
$dependencyModules = & (Join-Path $JdkPath 'bin/jdeps.exe') --multi-release 17 --ignore-missing-deps --print-
module-deps $applicationJar
if ($LASTEXITCODE -ne 0) { throw 'Could not determine Java runtime modules.' }
# Explicit providers retain TLS, international mail charsets, and locale support.
$modules = @((($dependencyModules -join '') -split ',') + @('jdk.crypto.ec', 'jdk.charsets', 'jdk.localedata'))
|
    Sort-Object -Unique
& (Join-Path $JdkPath 'bin/jlink.exe') --add-modules ($modules -join ',') --output $runtimeDirectory --strip-
debug --no-header-files --no-man-pages --compress=2
if ($LASTEXITCODE -ne 0) { throw 'Could not build the bundled Java runtime.' }

$packageArguments = @('--type', 'app-image', '--name', 'Maildesk', '--app-version', '1.0.0',
    '--vendor', 'Maildesk', '--description', 'Gmail client with MySQL persistence and spam review',
    '--input', $inputDirectory, '--main-jar', 'maildesk.jar', '--main-class', 'ems.App',
    '--runtime-image', $runtimeDirectory, '--dest', $OutputDirectory,
    '--java-options', '-Dmaildesk.home=$APPDIR/..')
& (Join-Path $JdkPath 'bin/jpackage.exe') @packageArguments
if ($LASTEXITCODE -ne 0) { throw 'Windows application packaging failed.' }

$configurationDirectory = Join-Path $applicationDirectory 'config'
$schemaDirectory = Join-Path $applicationDirectory 'sql'
New-Item -ItemType Directory -Path $configurationDirectory, $schemaDirectory -Force | Out-Null
Copy-Item -LiteralPath (Join-Path $projectRoot 'config/app.properties.example') -Destination
$configurationDirectory
Copy-Item -LiteralPath (Join-Path $projectRoot 'sql/schema.sql') -Destination $schemaDirectory
Copy-Item -LiteralPath (Join-Path $projectRoot 'docs/SETUP.md') -Destination (Join-Path $applicationDirectory
'SETUP.md')
@'
MAILDESK FOR WINDOWS x64

Extract the entire ZIP, then open Maildesk.exe. Keep app/ and runtime/ beside it.
A separate Java installation is unnecessary. MySQL must be running separately.

SETUP
1. Copy config/app.properties.example to config/app.properties and set the MySQL login.
2. Save your Google Desktop OAuth JSON as config/credentials.json.
3. Open Maildesk.exe and click Connect Gmail. See SETUP.md for detailed instructions.

Imports are capped at 18 messages: up to 6 each from Inbox, Sent, and Spam.
The distribution ZIP contains no personal settings, OAuth credentials, or tokens.
User configuration is resolved beside Maildesk.exe, independently of the working directory.

The bundled Java runtime includes its notices and licenses under runtime/legal.
For Eclipse Temurin runtime sources, see https://adoptium.net/temurin/releases/.
'@ | Set-Content -LiteralPath (Join-Path $applicationDirectory 'README.txt') -Encoding UTF8

# Archive the clean distribution before adding any user-specific configuration.
Compress-Archive -LiteralPath $applicationDirectory -DestinationPath $archive -CompressionLevel Optimal
if ($UseLocalConfig) {
    foreach ($name in @('app.properties', 'credentials.json')) {
        $source = Join-Path $projectRoot "config/$name"
```

```
        if (Test-Path -LiteralPath $source) { Copy-Item -LiteralPath $source -Destination
$configurationDirectory }
    }
    $sourceTokens = Join-Path $projectRoot 'config/tokens'
    if (Test-Path -LiteralPath $sourceTokens) {
        $destinationTokens = Join-Path $configurationDirectory 'tokens'
        New-Item -ItemType Directory -Path $destinationTokens -Force | Out-Null
        $identity = [Security.Principal.WindowsIdentity]::GetCurrent().Name
        & icacls.exe $destinationTokens /inheritance:r /grant:r ('{0}:(OI)(CI)F' -f $identity) | Out-Null
        if ($LASTEXITCODE -ne 0) { throw 'Could not protect the local token directory.' }
        Get-ChildItem -LiteralPath $sourceTokens -File | ForEach-Object {
            Copy-Item -LiteralPath $_.FullName -Destination $destinationTokens
        }
    }
}
Write-Host "Windows executable: $(Join-Path $applicationDirectory 'Maildesk.exe')"
Write-Host "Clean distribution: $archive"
```

scripts/run-app.ps1

```
[CmdletBinding()]
param(
    [switch]$NoBuild,
    [Parameter(ValueFromRemainingArguments = $true)]
    [string[]]$AppArguments
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
. (Join-Path $PSScriptRoot 'initialize-java.ps1')
$projectRoot = Split-Path -Parent $PSScriptRoot
if (-not $NoBuild) { & (Join-Path $PSScriptRoot 'build.ps1') -SkipTests }
$applicationJar = Join-Path $projectRoot 'target/maildesk.jar'
if (-not (Test-Path -LiteralPath $applicationJar)) { throw 'Build first with .\scripts\build.ps1.' }
$javaRuntime = Get-Command java -CommandType Application -ErrorAction Stop
Push-Location -LiteralPath $projectRoot
try {
    & $javaRuntime.Source -jar $applicationJar @AppArguments
    if ($LASTEXITCODE -ne 0) { throw "Maildesk exited with code $LASTEXITCODE." }
} finally { Pop-Location }
```

scripts/run-demo.ps1

```
[CmdletBinding()]
param([switch]$NoBuild)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
. (Join-Path $PSScriptRoot 'initialize-java.ps1')
$projectRoot = Split-Path -Parent $PSScriptRoot
if (-not $NoBuild) { & (Join-Path $PSScriptRoot 'build.ps1') -SkipTests }
$applicationJar = Join-Path $projectRoot 'target/maildesk.jar'
if (-not (Test-Path -LiteralPath $applicationJar)) { throw 'Build first with .\scripts\build.ps1.' }
$javaRuntime = Get-Command java -CommandType Application -ErrorAction Stop
Push-Location -LiteralPath $projectRoot
try {
    & $javaRuntime.Source -cp $applicationJar ems.demo.DemoApp
    if ($LASTEXITCODE -ne 0) { throw "The demo exited with code $LASTEXITCODE." }
} finally { Pop-Location }
```

APPENDIX A.6 — MAVEN BUILD CONFIGURATION

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>ems</groupId>
  <artifactId>maildesk</artifactId>
  <version>1.0.0</version>
  <name>Maildesk</name>
  <description>Java Swing Gmail client with a MySQL cache and explainable spam checks.</description>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <project.build.outputTimestamp>2026-10-20T00:00:00Z</project.build.outputTimestamp>
    <junit.version>5.11.4</junit.version>
    <skipTests>false</skipTests>
    <maildesk.uiRender></maildesk.uiRender>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.google.code.gson</groupId><artifactId>gson</artifactId><version>2.12.1</version>
    </dependency>
    <dependency>
      <groupId>jakarta.mail</groupId><artifactId>jakarta.mail-api</artifactId><version>2.1.3</version>
    </dependency>
    <dependency>
      <groupId>org.eclipse.angus</groupId><artifactId>angus-
mail</artifactId><version>2.0.3</version><scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>8.4.0</version><scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId><artifactId>h2</artifactId><version>2.3.232</version><scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId><artifactId>junit-
jupiter</artifactId><version>${junit.version}</version><scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>maildesk</finalName>
    <resources>
      <resource><directory>src/main/resources</directory></resource>
      <resource><directory>sql</directory><targetPath>sql</targetPath></resource>
    </resources>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId><artifactId>maven-clean-
plugin</artifactId><version>3.4.1</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId><artifactId>maven-resources-
plugin</artifactId><version>3.3.1</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId><artifactId>maven-compiler-
plugin</artifactId><version>3.14.0</version>
      </plugin>
      <plugin>
        <groupId>org.codehaus.mojo</groupId><artifactId>build-helper-maven-
plugin</artifactId><version>3.6.0</version>
        <executions>
          <execution>
            <id>include-demo</id><phase>generate-sources</phase><goals><goal>add-source</goal></goals>
            <configuration><sources><source>src/demo/java</source></sources></configuration>
          </execution>
        </executions>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId><artifactId>maven-surefire-
plugin</artifactId><version>3.5.2</version>
        <configuration>
          <systemPropertyVariables><java.awt.headless>true</java.awt.headless></systemPropertyVariables>
```

```
        </configuration>
      </plugin>
      <plugin>
        <groupId>org.codehaus.mojo</groupId><artifactId>exec-maven-plugin</artifactId><version>3.5.0</version>
        <executions>
          <execution>
            <id>member-one-contract-checks</id><phase>test</phase><goals><goal>exec</goal></goals>
            <configuration>
              <executable>java</executable><classpathScope>test</classpathScope>
              <skip>${skipTests}</skip>
              <arguments><argument>-Djava.awt.headless=true</argument><argument>-
classpath</argument><classpath/><argument>ems.check.MemberOneChecks</argument></arguments>
            </configuration>
          </execution>
          <execution>
            <id>swing-interface-checks</id><phase>test</phase><goals><goal>exec</goal></goals>
            <configuration>
              <executable>java</executable><classpathScope>test</classpathScope>
              <skip>${skipTests}</skip>
              <arguments><argument>-Djava.awt.headless=true</argument><argument>-
classpath</argument><classpath/><argument>ems.check.UiChecks</argument><argument>${maildesk.uiRender}</argument>
</arguments>
            </configuration>
          </execution>
        </executions>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId><artifactId>maven-jar-
plugin</artifactId><version>3.4.2</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId><artifactId>maven-shade-
plugin</artifactId><version>3.6.0</version>
        <executions>
          <execution>
            <phase>package</phase><goals><goal>shade</goal></goals>
            <configuration>
              <createDependencyReducedPom>false</createDependencyReducedPom>
              <filters>
                <filter>
                  <artifact>*:*</artifact>
                  <excludes><exclude>META-INF/*.SF</exclude><exclude>META-INF/*.RSA</exclude><exclude>META-
INF/*.DSA</exclude><exclude>module-info.class</exclude><exclude>META-INF/versions/*/module-
info.class</exclude></excludes>
                </filter>
              </filters>
              <transformers>
                <transformer
implementation="org.apache.maven.plugins.shade.resource.ServicesResourceTransformer"/>
                <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer"><mainClass>ems.App</mainCla
ss></transformer>
              </transformers>
            </configuration>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
  <profiles>
    <profile><id>render</id><properties><maildesk.uiRender>--render</maildesk.uiRender></properties></profile>
  </profiles>
</project>
```

APPENDIX A.7 — WRAPPER AND CONTINUOUS-INTEGRATION CONFIGURATION

mvnw

```
#!/bin/sh
# ----------------------------------------------------------------------------
# Licensed to the Apache Software Foundation (ASF) under one
# or more contributor license agreements.  See the NOTICE file
# distributed with this work for additional information
# regarding copyright ownership.  The ASF licenses this file
# to you under the Apache License, Version 2.0 (the
# "License"); you may not use this file except in compliance
# with the License.  You may obtain a copy of the License at
#
#    http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing,
# software distributed under the License is distributed on an
# "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
# KIND, either express or implied.  See the License for the
# specific language governing permissions and limitations
# under the License.
# ----------------------------------------------------------------------------

# ----------------------------------------------------------------------------
# Apache Maven Wrapper startup batch script, version 3.3.4
#
# Optional ENV vars
# -----------------
#   JAVA_HOME - location of a JDK home dir, required when download maven via java source
#   MVNW_REPOURL - repo url base for downloading maven distribution
#   MVNW_USERNAME/MVNW_PASSWORD - user and password for downloading maven
#   MVNW_VERBOSE - true: enable verbose log; debug: trace the mvnw script; others: silence the output
# ----------------------------------------------------------------------------

set -euf
[ "${MVNW_VERBOSE-}" != debug ] || set -x

# OS specific support.
native_path() { printf %s\\n "$1"; }
case "$(uname)" in
CYGWIN* | MINGW*)
  [ -z "${JAVA_HOME-}" ] || JAVA_HOME="$(cygpath --unix "$JAVA_HOME")"
  native_path() { cygpath --path --windows "$1"; }
  ;;
esac

# set JAVACMD and JAVACCMD
set_java_home() {
  # For Cygwin and MinGW, ensure paths are in Unix format before anything is touched
  if [ -n "${JAVA_HOME-}" ]; then
    if [ -x "$JAVA_HOME/jre/sh/java" ]; then
      # IBM's JDK on AIX uses strange locations for the executables
      JAVACMD="$JAVA_HOME/jre/sh/java"
      JAVACCMD="$JAVA_HOME/jre/sh/javac"
    else
      JAVACMD="$JAVA_HOME/bin/java"
      JAVACCMD="$JAVA_HOME/bin/javac"

      if [ ! -x "$JAVACMD" ] || [ ! -x "$JAVACCMD" ]; then
        echo "The JAVA_HOME environment variable is not defined correctly, so mvnw cannot run." >&2
        echo "JAVA_HOME is set to \"$JAVA_HOME\", but \"\$JAVA_HOME/bin/java\" or \"\$JAVA_HOME/bin/javac\" does
not exist." >&2
        return 1
      fi
    fi
  else
```

```
    JAVACMD="$(
      'set' +e
      'unset' -f command 2>/dev/null
      'command' -v java
    )" || :
    JAVACCMD="$(
      'set' +e
      'unset' -f command 2>/dev/null
      'command' -v javac
    )" || :

    if [ ! -x "${JAVACMD-}" ] || [ ! -x "${JAVACCMD-}" ]; then
      echo "The java/javac command does not exist in PATH nor is JAVA_HOME set, so mvnw cannot run." >&2
      return 1
    fi
  fi
}

# hash string like Java String::hashCode
hash_string() {
  str="${1:-}" h=0
  while [ -n "$str" ]; do
    char="${str%"${str#?}"}"
    h=$(((h * 31 + $(LC_CTYPE=C printf %d "'$char")) % 4294967296))
    str="${str#?}"
  done
  printf %x\\n $h
}

verbose() { :; }
[ "${MVNW_VERBOSE-}" != true ] || verbose() { printf %s\\n "${1-}"; }

die() {
  printf %s\\n "$1" >&2
  exit 1
}

trim() {
  # MWRAPPER-139:
  #   Trims trailing and leading whitespace, carriage returns, tabs, and linefeeds.
  #   Needed for removing poorly interpreted newline sequences when running in more
  #   exotic environments such as mingw bash on Windows.
  printf "%s" "${1}" | tr -d '[:space:]'
}

scriptDir="$(dirname "$0")"
scriptName="$(basename "$0")"

# parse distributionUrl and optional distributionSha256Sum, requires .mvn/wrapper/maven-wrapper.properties
while IFS="=" read -r key value; do
  case "${key-}" in
  distributionUrl) distributionUrl=$(trim "${value-}") ;;
  distributionSha256Sum) distributionSha256Sum=$(trim "${value-}") ;;
  esac
done <"$scriptDir/.mvn/wrapper/maven-wrapper.properties"
[ -n "${distributionUrl-}" ] || die "cannot read distributionUrl property in $scriptDir/.mvn/wrapper/maven-
wrapper.properties"

case "${distributionUrl##*/}" in
maven-mvnd-*bin.*)
  MVN_CMD=mvnd.sh _MVNW_REPO_PATTERN=/maven/mvnd/
  case "${PROCESSOR_ARCHITECTURE-}${PROCESSOR_ARCHITEW6432-}:$(uname -a)" in
  *AMD64:CYGWIN* | *AMD64:MINGW*) distributionPlatform=windows-amd64 ;;
  :Darwin*x86_64) distributionPlatform=darwin-amd64 ;;
  :Darwin*arm64) distributionPlatform=darwin-aarch64 ;;
  :Linux*x86_64*) distributionPlatform=linux-amd64 ;;
  *)
    echo "Cannot detect native platform for mvnd on $(uname)-$(uname -m), use pure java version" >&2
    distributionPlatform=linux-amd64
    ;;
```

```
  esac
  distributionUrl="${distributionUrl%-bin.*}-$distributionPlatform.zip"
  ;;
maven-mvnd-*) MVN_CMD=mvnd.sh _MVNW_REPO_PATTERN=/maven/mvnd/ ;;
*) MVN_CMD="mvn${scriptName#mvnw}" _MVNW_REPO_PATTERN=/org/apache/maven/ ;;
esac

# apply MVNW_REPOURL and calculate MAVEN_HOME
# maven home pattern: ~/.m2/wrapper/dists/{apache-maven-<version>,maven-mvnd-<version>-<platform>}/<hash>
[ -z "${MVNW_REPOURL-}" ] ||
distributionUrl="$MVNW_REPOURL$_MVNW_REPO_PATTERN${distributionUrl#*"$_MVNW_REPO_PATTERN"}"
distributionUrlName="${distributionUrl##*/}"
distributionUrlNameMain="${distributionUrlName%.*}"
distributionUrlNameMain="${distributionUrlNameMain%-bin}"
MAVEN_USER_HOME="${MAVEN_USER_HOME:-${HOME}/.m2}"
MAVEN_HOME="${MAVEN_USER_HOME}/wrapper/dists/${distributionUrlNameMain-}/$(hash_string "$distributionUrl")"

exec_maven() {
  unset MVNW_VERBOSE MVNW_USERNAME MVNW_PASSWORD MVNW_REPOURL || :
  exec "$MAVEN_HOME/bin/$MVN_CMD" "$@" || die "cannot exec $MAVEN_HOME/bin/$MVN_CMD"
}

if [ -d "$MAVEN_HOME" ]; then
  verbose "found existing MAVEN_HOME at $MAVEN_HOME"
  exec_maven "$@"
fi

case "${distributionUrl-}" in
*?-bin.zip | *?maven-mvnd-?*-?*.zip) ;;
*) die "distributionUrl is not valid, must match *-bin.zip or maven-mvnd-*.zip, but found '${distributionUrl-}'"
;;
esac

# prepare tmp dir
if TMP_DOWNLOAD_DIR="$(mktemp -d)" && [ -d "$TMP_DOWNLOAD_DIR" ]; then
  clean() { rm -rf -- "$TMP_DOWNLOAD_DIR"; }
  trap clean HUP INT TERM EXIT
else
  die "cannot create temp dir"
fi

mkdir -p -- "${MAVEN_HOME%/*}"

# Download and Install Apache Maven
verbose "Couldn't find MAVEN_HOME, downloading and installing it ..."
verbose "Downloading from: $distributionUrl"
verbose "Downloading to: $TMP_DOWNLOAD_DIR/$distributionUrlName"

# select .zip or .tar.gz
if ! command -v unzip >/dev/null; then
  distributionUrl="${distributionUrl%.zip}.tar.gz"
  distributionUrlName="${distributionUrl##*/}"
fi

# verbose opt
__MVNW_QUIET_WGET=--quiet __MVNW_QUIET_CURL=--silent __MVNW_QUIET_UNZIP=-q __MVNW_QUIET_TAR=''
[ "${MVNW_VERBOSE-}" != true ] || __MVNW_QUIET_WGET='' __MVNW_QUIET_CURL='' __MVNW_QUIET_UNZIP=''
__MVNW_QUIET_TAR=v

# normalize http auth
case "${MVNW_PASSWORD:+has-password}" in
'') MVNW_USERNAME='' MVNW_PASSWORD='' ;;
has-password) [ -n "${MVNW_USERNAME-}" ] || MVNW_USERNAME='' MVNW_PASSWORD='' ;;
esac

if [ -z "${MVNW_USERNAME-}" ] && command -v wget >/dev/null; then
  verbose "Found wget ... using wget"
```

```
  wget ${__MVNW_QUIET_WGET:+"$__MVNW_QUIET_WGET"} "$distributionUrl" -O "$TMP_DOWNLOAD_DIR/$distributionUrlName"
|| die "wget: Failed to fetch $distributionUrl"
elif [ -z "${MVNW_USERNAME-}" ] && command -v curl >/dev/null; then
  verbose "Found curl ... using curl"
  curl ${__MVNW_QUIET_CURL:+"$__MVNW_QUIET_CURL"} -f -L -o "$TMP_DOWNLOAD_DIR/$distributionUrlName"
"$distributionUrl" || die "curl: Failed to fetch $distributionUrl"
elif set_java_home; then
  verbose "Falling back to use Java to download"
  javaSource="$TMP_DOWNLOAD_DIR/Downloader.java"
  targetZip="$TMP_DOWNLOAD_DIR/$distributionUrlName"
  cat >"$javaSource" <<-END
        public class Downloader extends java.net.Authenticator
        {
          protected java.net.PasswordAuthentication getPasswordAuthentication()
          {
            return new java.net.PasswordAuthentication( System.getenv( "MVNW_USERNAME" ), System.getenv(
"MVNW_PASSWORD" ).toCharArray() );
          }
          public static void main( String[] args ) throws Exception
          {
            setDefault( new Downloader() );
            java.nio.file.Files.copy( java.net.URI.create( args[0] ).toURL().openStream(),
java.nio.file.Paths.get( args[1] ).toAbsolutePath().normalize() );
          }
        }
        END
  # For Cygwin/MinGW, switch paths to Windows format before running javac and java
  verbose " - Compiling Downloader.java ..."
  "$(native_path "$JAVACCMD")" "$(native_path "$javaSource")" || die "Failed to compile Downloader.java"
  verbose " - Running Downloader.java ..."
  "$(native_path "$JAVACMD")" -cp "$(native_path "$TMP_DOWNLOAD_DIR")" Downloader "$distributionUrl"
"$(native_path "$targetZip")"
fi


# If specified, validate the SHA-256 sum of the Maven distribution zip file
if [ -n "${distributionSha256Sum-}" ]; then
  distributionSha256Result=false
  if [ "$MVN_CMD" = mvnd.sh ]; then
    echo "Checksum validation is not supported for maven-mvnd." >&2
    echo "Please disable validation by removing 'distributionSha256Sum' from your maven-wrapper.properties." >&2
    exit 1
  elif command -v sha256sum >/dev/null; then
    if echo "$distributionSha256Sum  $TMP_DOWNLOAD_DIR/$distributionUrlName" | sha256sum -c - >/dev/null 2>&1;
then
      distributionSha256Result=true
    fi
  elif command -v shasum >/dev/null; then
    if echo "$distributionSha256Sum  $TMP_DOWNLOAD_DIR/$distributionUrlName" | shasum -a 256 -c >/dev/null 2>&1;
then
      distributionSha256Result=true
    fi
  else
    echo "Checksum validation was requested but neither 'sha256sum' or 'shasum' are available." >&2
    echo "Please install either command, or disable validation by removing 'distributionSha256Sum' from your
maven-wrapper.properties." >&2
    exit 1
  fi
  if [ $distributionSha256Result = false ]; then
    echo "Error: Failed to validate Maven distribution SHA-256, your Maven distribution might be compromised."
>&2
    echo "If you updated your Maven version, you need to update the specified distributionSha256Sum property."
>&2
    exit 1
  fi
fi


# unzip and move
if command -v unzip >/dev/null; then
  unzip ${__MVNW_QUIET_UNZIP:+"$__MVNW_QUIET_UNZIP"} "$TMP_DOWNLOAD_DIR/$distributionUrlName" -d
"$TMP_DOWNLOAD_DIR" || die "failed to unzip"
else
  tar xzf${__MVNW_QUIET_TAR:+"$__MVNW_QUIET_TAR"} "$TMP_DOWNLOAD_DIR/$distributionUrlName" -C
"$TMP_DOWNLOAD_DIR" || die "failed to untar"
```

```
fi

# Find the actual extracted directory name (handles snapshots where filename != directory name)
actualDistributionDir=""

# First try the expected directory name (for regular distributions)
if [ -d "$TMP_DOWNLOAD_DIR/$distributionUrlNameMain" ]; then
  if [ -f "$TMP_DOWNLOAD_DIR/$distributionUrlNameMain/bin/$MVN_CMD" ]; then
    actualDistributionDir="$distributionUrlNameMain"
  fi
fi

# If not found, search for any directory with the Maven executable (for snapshots)
if [ -z "$actualDistributionDir" ]; then
  # enable globbing to iterate over items
  set +f
  for dir in "$TMP_DOWNLOAD_DIR"/*; do
    if [ -d "$dir" ]; then
      if [ -f "$dir/bin/$MVN_CMD" ]; then
        actualDistributionDir="$(basename "$dir")"
        break
      fi
    fi
  done
  set -f
fi

if [ -z "$actualDistributionDir" ]; then
  verbose "Contents of $TMP_DOWNLOAD_DIR:"
  verbose "$(ls -la "$TMP_DOWNLOAD_DIR")"
  die "Could not find Maven distribution directory in extracted archive"
fi

verbose "Found extracted Maven distribution directory: $actualDistributionDir"
printf %s\\n "$distributionUrl" >"$TMP_DOWNLOAD_DIR/$actualDistributionDir/mvnw.url"
mv -- "$TMP_DOWNLOAD_DIR/$actualDistributionDir" "$MAVEN_HOME" || [ -d "$MAVEN_HOME" ] || die "fail to move
MAVEN_HOME"

clean || :
exec_maven "$@"
```

mvnw.cmd

```
<# : batch portion
@REM ----------------------------------------------------------------------------
@REM Licensed to the Apache Software Foundation (ASF) under one
@REM or more contributor license agreements.  See the NOTICE file
@REM distributed with this work for additional information
@REM regarding copyright ownership.  The ASF licenses this file
@REM to you under the Apache License, Version 2.0 (the
@REM "License"); you may not use this file except in compliance
@REM with the License.  You may obtain a copy of the License at
@REM
@REM    http://www.apache.org/licenses/LICENSE-2.0
@REM
@REM Unless required by applicable law or agreed to in writing,
@REM software distributed under the License is distributed on an
@REM "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
@REM KIND, either express or implied.  See the License for the
@REM specific language governing permissions and limitations
@REM under the License.
@REM ----------------------------------------------------------------------------

@REM ----------------------------------------------------------------------------
@REM Apache Maven Wrapper startup batch script, version 3.3.4
@REM
@REM Optional ENV vars
@REM   MVNW_REPOURL - repo url base for downloading maven distribution
```

```
@REM   MVNW_USERNAME/MVNW_PASSWORD - user and password for downloading maven
@REM   MVNW_VERBOSE - true: enable verbose log; others: silence the output
@REM ----------------------------------------------------------------------------

@IF "%__MVNW_ARG0_NAME__%"=="" (SET __MVNW_ARG0_NAME__=%~nx0)
@SET __MVNW_CMD__=
@SET __MVNW_ERROR__=
@SET __MVNW_PSMODULEP_SAVE=%PSModulePath%
@SET PSModulePath=
@FOR /F "usebackq tokens=1* delims==" %%A IN (`powershell -noprofile "& {$scriptDir='%~dp0';
$script='%__MVNW_ARG0_NAME__%'; icm -ScriptBlock ([Scriptblock]::Create((Get-Content -Raw '%~f0'))) -
NoNewScope}"`) DO @(
  IF "%%A"=="MVN_CMD" (set __MVNW_CMD__=%%B) ELSE IF "%%B"=="" (echo %%A) ELSE (echo %%A=%%B)
)
@SET PSModulePath=%__MVNW_PSMODULEP_SAVE%
@SET __MVNW_PSMODULEP_SAVE=
@SET __MVNW_ARG0_NAME__=
@SET MVNW_USERNAME=
@SET MVNW_PASSWORD=
@IF NOT "%__MVNW_CMD__%"=="" ("%__MVNW_CMD__%" %*)
@echo Cannot start maven from wrapper >&2 && exit /b 1
@GOTO :EOF
: end batch / begin powershell #>

$ErrorActionPreference = "Stop"
if ($env:MVNW_VERBOSE -eq "true") {
  $VerbosePreference = "Continue"
}

# calculate distributionUrl, requires .mvn/wrapper/maven-wrapper.properties
$distributionUrl = (Get-Content -Raw "$scriptDir/.mvn/wrapper/maven-wrapper.properties" | ConvertFrom-
StringData).distributionUrl
if (!$distributionUrl) {
  Write-Error "cannot read distributionUrl property in $scriptDir/.mvn/wrapper/maven-wrapper.properties"
}

switch -wildcard -casesensitive ( $($distributionUrl -replace '^.*/','') ) {
  "maven-mvnd-*" {
    $USE_MVND = $true
    $distributionUrl = $distributionUrl -replace '-bin\.[^.]*$',"-windows-amd64.zip"
    $MVN_CMD = "mvnd.cmd"
    break
  }
  default {
    $USE_MVND = $false
    $MVN_CMD = $script -replace '^mvnw','mvn'
    break
  }
}

# apply MVNW_REPOURL and calculate MAVEN_HOME
# maven home pattern: ~/.m2/wrapper/dists/{apache-maven-<version>,maven-mvnd-<version>-<platform>}/<hash>
if ($env:MVNW_REPOURL) {
  $MVNW_REPO_PATTERN = if ($USE_MVND -eq $False) { "/org/apache/maven/" } else { "/maven/mvnd/" }
  $distributionUrl = "$env:MVNW_REPOURL$MVNW_REPO_PATTERN$($distributionUrl -replace
"^.*$MVNW_REPO_PATTERN",'')"
}
$distributionUrlName = $distributionUrl -replace '^.*/',''
$distributionUrlNameMain = $distributionUrlName -replace '\.[^.]*$','' -replace '-bin$',''

$MAVEN_M2_PATH = "$HOME/.m2"
if ($env:MAVEN_USER_HOME) {
  $MAVEN_M2_PATH = "$env:MAVEN_USER_HOME"
}

if (-not (Test-Path -Path $MAVEN_M2_PATH)) {
    New-Item -Path $MAVEN_M2_PATH -ItemType Directory | Out-Null
}

$MAVEN_WRAPPER_DISTS = $null
```

```
if ((Get-Item $MAVEN_M2_PATH).Target[0] -eq $null) {
  $MAVEN_WRAPPER_DISTS = "$MAVEN_M2_PATH/wrapper/dists"
} else {
  $MAVEN_WRAPPER_DISTS = (Get-Item $MAVEN_M2_PATH).Target[0] + "/wrapper/dists"
}

$MAVEN_HOME_PARENT = "$MAVEN_WRAPPER_DISTS/$distributionUrlNameMain"
$MAVEN_HOME_NAME =
([System.Security.Cryptography.SHA256]::Create().ComputeHash([byte[]][char[]]$distributionUrl) | ForEach-Object
{$_.ToString("x2")}) -join ''
$MAVEN_HOME = "$MAVEN_HOME_PARENT/$MAVEN_HOME_NAME"

if (Test-Path -Path "$MAVEN_HOME" -PathType Container) {
  Write-Verbose "found existing MAVEN_HOME at $MAVEN_HOME"
  Write-Output "MVN_CMD=$MAVEN_HOME/bin/$MVN_CMD"
  exit $?
}

if (! $distributionUrlNameMain -or ($distributionUrlName -eq $distributionUrlNameMain)) {
  Write-Error "distributionUrl is not valid, must end with *-bin.zip, but found $distributionUrl"
}

# prepare tmp dir
$TMP_DOWNLOAD_DIR_HOLDER = New-TemporaryFile
$TMP_DOWNLOAD_DIR = New-Item -Itemtype Directory -Path "$TMP_DOWNLOAD_DIR_HOLDER.dir"
$TMP_DOWNLOAD_DIR_HOLDER.Delete() | Out-Null
trap {
  if ($TMP_DOWNLOAD_DIR.Exists) {
    try { Remove-Item $TMP_DOWNLOAD_DIR -Recurse -Force | Out-Null }
    catch { Write-Warning "Cannot remove $TMP_DOWNLOAD_DIR" }
  }
}

New-Item -Itemtype Directory -Path "$MAVEN_HOME_PARENT" -Force | Out-Null

# Download and Install Apache Maven
Write-Verbose "Couldn't find MAVEN_HOME, downloading and installing it ..."
Write-Verbose "Downloading from: $distributionUrl"
Write-Verbose "Downloading to: $TMP_DOWNLOAD_DIR/$distributionUrlName"

$webclient = New-Object System.Net.WebClient
if ($env:MVNW_USERNAME -and $env:MVNW_PASSWORD) {
  $webclient.Credentials = New-Object System.Net.NetworkCredential($env:MVNW_USERNAME, $env:MVNW_PASSWORD)
}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
$webclient.DownloadFile($distributionUrl, "$TMP_DOWNLOAD_DIR/$distributionUrlName") | Out-Null

# If specified, validate the SHA-256 sum of the Maven distribution zip file
$distributionSha256Sum = (Get-Content -Raw "$scriptDir/.mvn/wrapper/maven-wrapper.properties" | ConvertFrom-
StringData).distributionSha256Sum
if ($distributionSha256Sum) {
  if ($USE_MVND) {
    Write-Error "Checksum validation is not supported for maven-mvnd. `nPlease disable validation by removing
'distributionSha256Sum' from your maven-wrapper.properties."
  }
  Import-Module $PSHOME\Modules\Microsoft.PowerShell.Utility -Function Get-FileHash
  if ((Get-FileHash "$TMP_DOWNLOAD_DIR/$distributionUrlName" -Algorithm SHA256).Hash.ToLower() -ne
$distributionSha256Sum) {
    Write-Error "Error: Failed to validate Maven distribution SHA-256, your Maven distribution might be
compromised. If you updated your Maven version, you need to update the specified distributionSha256Sum
property."
  }
}

# unzip and move
Expand-Archive "$TMP_DOWNLOAD_DIR/$distributionUrlName" -DestinationPath "$TMP_DOWNLOAD_DIR" | Out-Null

# Find the actual extracted directory name (handles snapshots where filename != directory name)
$actualDistributionDir = ""
```

```
# First try the expected directory name (for regular distributions)
$expectedPath = Join-Path "$TMP_DOWNLOAD_DIR" "$distributionUrlNameMain"
$expectedMvnPath = Join-Path "$expectedPath" "bin/$MVN_CMD"
if ((Test-Path -Path $expectedPath -PathType Container) -and (Test-Path -Path $expectedMvnPath -PathType Leaf))
{
  $actualDistributionDir = $distributionUrlNameMain
}

# If not found, search for any directory with the Maven executable (for snapshots)
if (!$actualDistributionDir) {
  Get-ChildItem -Path "$TMP_DOWNLOAD_DIR" -Directory | ForEach-Object {
    $testPath = Join-Path $_.FullName "bin/$MVN_CMD"
    if (Test-Path -Path $testPath -PathType Leaf) {
      $actualDistributionDir = $_.Name
    }
  }
}

if (!$actualDistributionDir) {
  Write-Error "Could not find Maven distribution directory in extracted archive"
}

Write-Verbose "Found extracted Maven distribution directory: $actualDistributionDir"
Rename-Item -Path "$TMP_DOWNLOAD_DIR/$actualDistributionDir" -NewName $MAVEN_HOME_NAME | Out-Null
try {
  Move-Item -Path "$TMP_DOWNLOAD_DIR/$MAVEN_HOME_NAME" -Destination $MAVEN_HOME_PARENT | Out-Null
} catch {
  if (! (Test-Path -Path "$MAVEN_HOME" -PathType Container)) {
    Write-Error "fail to move MAVEN_HOME"
  }
} finally {
  try { Remove-Item $TMP_DOWNLOAD_DIR -Recurse -Force | Out-Null }
  catch { Write-Warning "Cannot remove $TMP_DOWNLOAD_DIR" }
}

Write-Output "MVN_CMD=$MAVEN_HOME/bin/$MVN_CMD"
```

.mvn/wrapper/maven-wrapper.properties

```
wrapperVersion=3.3.4
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3.9.11-
bin.zip
distributionSha256Sum=0d7125e8c91097b36edb990ea5934e6c68b4440eef4ea96510a0f6815e7eeadb
```

.github/workflows/verify.yml

```
name: Verify Maildesk
on:
  push:
  pull_request:
permissions:
  contents: read
jobs:
  test:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
          cache: maven
      - name: Build and test on Linux
```

```
        if: runner.os != 'Windows'
        run: sh ./mvnw --batch-mode --no-transfer-progress verify
      - name: Build and test on Windows
        if: runner.os == 'Windows'
        shell: pwsh
        run: ./scripts/build.ps1
      - uses: actions/upload-artifact@v4
        with:
          name: maildesk-${{ runner.os }}
          path: target/maildesk.jar
          if-no-files-found: error
      - name: Package Windows executable
        if: runner.os == 'Windows'
        shell: pwsh
        run: ./scripts/package-windows.ps1 -NoBuild -JdkPath $env:JAVA_HOME
      - name: Check Windows launcher
        if: runner.os == 'Windows'
        shell: pwsh
        run: |
          $launcher = Start-Process -FilePath (Resolve-Path 'package/Maildesk/Maildesk.exe') -ArgumentList '--
help' -WindowStyle Hidden -Wait -PassThru
          if ($launcher.ExitCode -ne 0) { throw 'The packaged Windows launcher failed.' }
      - uses: actions/upload-artifact@v4
        if: runner.os == 'Windows'
        with:
          name: Maildesk-Windows-x64
          path: package/Maildesk-Windows-x64.zip
          if-no-files-found: error
```